Applied Python Programming:

From Foundations to Advanced Architectures

Table of Contents

CHAPTER 1 — Python Fundamentals and Data Representation

1. PYTHON FUNDAMENTALS AND DATA REPRESENTATION

1.1 Features and History of Python

Origins and Evolution:

Python was created by **Guido van Rossum** at the **Centrum Wiskunde & Informatica (CWI), Netherlands**, during the late 1980s and first released in **1991**.

Historical Milestones:

Version	Key Features	Remarks
Python 1.x	Basic modules, functional core	Foundation phase
Python 2.x	Unicode, improved libraries	Long-term adoption: support ended 2020
Python 3.x	Clean syntax, improved Unicode, modernized language model	Not backward compatible with 2.x

Key Features of Python:

(a) High-Level Language

Python abstracts away hardware complexities:

- No manual memory management
- Automatic garbage collection
- Human-readable syntax

(b) Freeware & Open Source

- Managed by the **Python Software Foundation (PSF)** • Licensed under OSI-approved terms
- Source code is openly available.

(c) Dynamic Typing

Variables do *not* declare types; types are inferred at runtime.

$x = 10$ *# integer*

$x =$ "*hello*" *# now a string*

(d) Extensibility

Python can call external modules written in:

- **C**
- **C++**
- **Java (via Jython)**
- **.NET languages (via IronPython)**

This makes Python suitable for high-performance systems.

(e) Embeddability

Python can be embedded into larger systems:

- Game engines
- Robotics controllers
- Scientific simulations

Python Implementations:

Implementation	Platform	Purpose
CPython	C-based	Official, most widely used
PyPy	RPython	JIT compiler for speed
Jython	JVM	Interoperable with Java
IronPython	.NET	Interoperable with C#/VB.NET
Anaconda	Multi-platform	Scientific computing, data science

Binary, Octal, Hexadecimal & High-Level Conversion

Python hides low-level memory representation, but supports:

$x = 0b1010 \quad \# \, binary$

$y = 0o17 \quad \# \, octal$

$z = 0xFF \quad \# \, hexadecimal$

Internally, Python converts these into **base-2 machine instructions**, but programmers interact with clean, abstracted data structures.

1.2 Variables, Literals, and Basic Data Types

Variables & Identifiers

A **variable** is a symbolic name referencing an object in memory.

Rules for Identifiers

The book now shows bright symbols around each pouch and writes new rules:

- **A variable name can contain:**
  Alphabets (A–Z, a–z)
  Digits (0–9)
  Underscore (_)
  Example: hero_name, level1_score, has_key_2
- **A variable name must start with:**
  An alphabet, or
  An underscore _
  Examples:
  Good → name, _secret, goldCoins
  Bad → 1name, @value, #score
- **No special symbols allowed except underscore.**
  Not allowed: hero-name, hero$name, hero@123
  Allowed: hero_name, _heroName
- **You cannot use Python keywords as variable names.**
  Keywords are special reserved words in Python, such as: if, else, for, while, class, def, return, etc.
  They already have important meanings and cannot be used as containers.

- **Variables are case-sensitive.**
  This means name, Name, and NAME are all different variables.

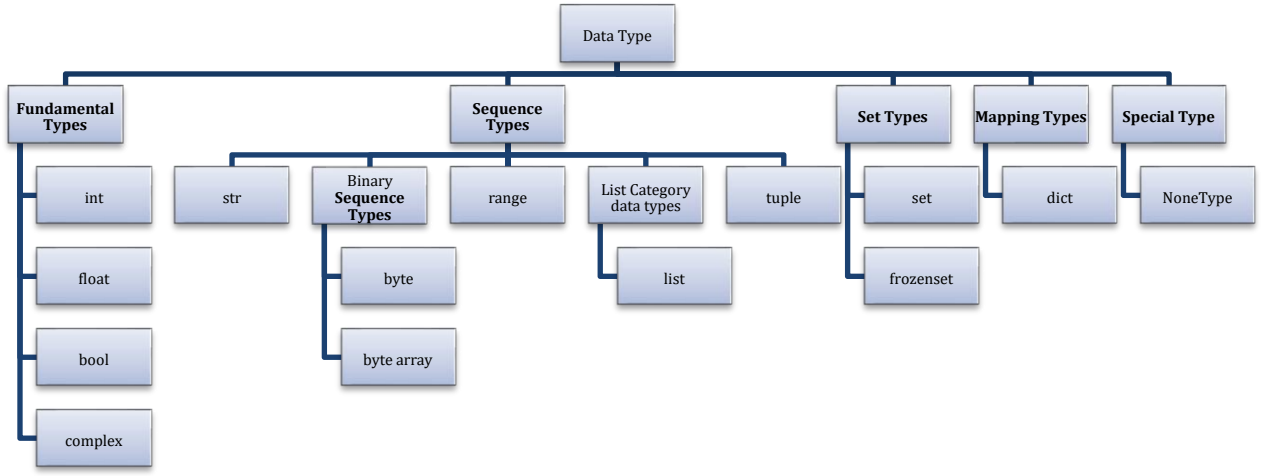
Why They Are Called Variables

Because their associated values can be **reassigned** during execution.

Memory Allocation in Python

- Python uses a **heap-based memory model**:
- *Variable Name* $--> points\ to\ an\ Object$
- Objects are allocated on the **heap**, while references live in the **namespace**.
- Example: a = 10 b = a
- Both a and b point to the same memory address initially.

1.3 Categories of Data Types



Fundamental Category data type:

The purpose of the fundamentals category data type is that "To store the single value".

Integer:

Properties:

- **int** is the one of the predefine class name and treated as fundamental datatype.
- The purpose of the **int datatype** is that to store the integer data type or whole numbers or Integral data(Number without Decimal Places)
- With int data type, In Python Programming, along with Integer data, we can also store Different. Number System Values. In Any Programming Lang, we have 4 Types of Number Systems. They are:

a. Decimal Number System:

- Decimal Number System is one the Default Number System in Programming language.
- This Number System Contains Digits: 0 1 2 3 4 5 6 7 8 9 →Total digits=10 which base: 10
- base 10 Literals are called Decimal Number System Values

b. Binary Number System:

- Binary Number System is Understandable by OS and Processor.
- This Number System Contains Digits: 0 1 →Total digits=2 which base: 2
- base 2 Literals are called Binary Number System Values. In python programming, To Store Binary Data, Binary data must be preceded by letter either 0b OR 0B.
- Syntax: $varname = 0b\ Binary\ data$

  (OR)

  $varname = 0B\ Binary\ data$
- Even we store Binary data in Python Programming, Python Programming Execution environment automatically / Implicitly Converted into Decimal Number System Types.

  Examples:

>>> $a = 0b1101$

>>> $print(a, type(a))$ → 13 <class 'int'>

>>> $bin(46)$ → b101110'

NOTE: bin() is used converting any number system data into Binary Number System Type. **c.**

Octal Number System

- Octal Number System is Understandable by Microprocessor Kits --8086
- This Number System Contains Digits: 0 1 2 3 4 5 6 7 →Total digits=8 which base:8
- Base 8 Literals are called Octal Number System Values.
- In python programming, To Store Octal Data, Octal data must be preceded by letter either 0o OR 0O.
- Syntax: $varname = 0o\ Octal\ data$

  (OR)

  $varname = 0O\ Octal\ data$
- Even we store Octal data in Python Programming, Python Programming Execution environment automatically / Implicitly Converted into Decimal Number System Types.

  Examples:

>>> $a = 0o33$

>>> $print(a, type(a))$ → 27 <class 'int'>

>>> $a = 0o128$ → SyntaxError: invalid digit '8' in octal literal

>>> $oct(27)$ →'0o33'

NOTE: oct() is used converting any number system data into Octal Number System Type.

d. Hexa Decimal Number System

- Hexa Decimal Number System used in development OSes.
- This Number System Contains

  Digits: 0 1 2 3 4 5 6 7 8 9

  Alphabet: A(10) B(11) C(12) D(13) E(14) F(15)-----Total ---16
- Which Base 16 Literals are called Hexa Decimal Number System Values.

- In python programming, To Store Hexa Decimal Data, Hexa Decimal data must be preceded by letter either 0x OR 0X.
- Syntax: $varname = 0x\ Hexa\ Decimal\ Data$

  (OR)

  $varname = 0X\ Hexa\ Decimal\ \ data$
- Even we store Hexa Decimal data in Python Programming, Python Programming Execution environment automatically / Implicitly Converted into Decimal Number System Types.

Examples:

$>>> a = 0XFaCe$

$>>> print(a, type(a))$ → 64206 <class 'int'>

$>>> a = 0xACCER$ → SyntaxError: invalid hexadecimal literal

$>>> hex(172)$ → '0xac'

NOTE: hex() is used converting any number system data into Hexa Decimal Number System Type.

Float:

- 'Float' is one of the pre-defined class names and treated as Fundamental data Type. The purpose of float data type is that" To store Real Constant values OR Float OR Double Pointing (Number with Decimal Places)".
- Float data type never allows us to represent / Store Binary, Octal and Hexa Decimal Values
- Float data type allows us to store Scientific Notation data. The Advantage of Scientific is that to take Less Memory space for bigger floating-point Values.

Example:

$>>> a = 12.34$

$>>> print(a, type(a))$ → 12.34 <class 'float'>

$>>> a = 10$

$>>> b = 1.2$

$>>> c = a + b$

$>>> print(c, type(c))$ → 11.2 <class 'float'>

$>>> a = 0b1010.0b1111$ → SyntaxError: invalid decimal literal $>>> a = 0o13.0o23$ → SyntaxError: invalid decimal literal $>>> a = 0xAC.0XBEE$ → SyntaxError: invalid decimal literal

$>>> a = 0b1010.0xAC$ → SyntaxError: invalid decimal literal

$>>> a = 0o12.0b1010$ → SyntaxError: invalid decimal literal

$>>> a = 3e2$

$>>> print(a, type(a))$ → 300.0 <class 'float'>

$>>> a = 0.00000000000000000000000000000000000000000000006$

>>> $print(a, type(a))$ → 6e-47 <class 'float'>

Bool:

- 'bool' is one of the pre-defined class names and treated as Fundamental data Type.
- The purpose of bool data type is that "To Store True False Values".
- In Python Programming True, False are Called Keywords and It is treated as values of bool data type.
- In Python Programming, Internally, the value of True is treated as 1 and False is treated as 0.

Example:

```
>>> a=True
>>> print(a, type(a))                    → True <class 'bool'>
>>> b=False
>>> print(b, type(b))                    → False <class 'bool'>
>>> a=True
>>> b=False
>>> print(a+b)                  → 1
>>> print(a-b)                  → 1
>>> print(2*True+True+False)    → 3
>>> print(2+True*3+False)       → 5
>>> print(0b1010-True+2*True)   → 11
>>> print(True/True)                 → 1.0
>>> print(True//True)           → 1
>>> print(False//True)  → 0
# Most Imp
>>> print(True//False)  → ZeroDivisionError: integer division or modulo by zero
```

Complex:

- 'complex' is one of the pre-defined classes and treated as Fundamental data type.
- The purpose of complex data type is that "To store Complex Values / Imaginary values".
- The General Notation of complex data type is given bellow.

  $x + yj$ or $x - yj$

  ○ Here 'x' is called Real Part ○ Here 'y' is called Imaginary part. ○ and j Represents $sqrt(-1)$ or $sqr(j) = -1$
- Internally, Real and Imaginary parts of Complex Object are treated as float data type.
- To Extract the real and Imaginary parts of Complex object, we have 2 pre-defined attributes. They are.

1) real

2) imag

Syntax: complexobj.real → Gives Real Part of Complex object

complexobj.imag → Gives Imaginary Part of Complex Object On the object of complex data type, we can perform various Arithmetic Operation such as Addition, Subtraction, Multiplication. Etc **Example:**

```
>>> a=2+3j
>>> print(a, type(a))          → (2+3j) <class 'complex'>
>>> b=-2-4j
>>> print(b, type(b))          → (-2-4j) <class 'complex'>
>>> a=1.2+3.4j
>>> print(a, type(a))          → (1.2+3.4j) <class 'complex'>
>>> b=-2.3-4.5j
>>> print(b, type(b))          → (-2.3-4.5j) <class 'complex'>
>>> a=12+4.5j
>>> print(a, type(a))          → (12+4.5j) <class 'complex'>
>>> b=-12.5-4j
>>> print(b, type(b))          → (-12.5-4j) <class 'complex'>
>>> a=2j
>>> print(a, type(a))          → 2j <class 'complex'>
>>> b=-5j
>>> print(b, type(b))          → (-0-5j) <class 'complex'>
>>> a=2.3j
>>> print(a, type(a))          → 2.3j <class 'complex'>
>>> b=-2.6j
>>> print(b, type(b))          →(-0-2.6j) <class 'complex'>
>>> a=10+3j
>>> print(a, type(a))          → (10+3j) <class 'complex'>
>>> print(a.real)              → 10.0
>>> print(a.imag)              → 3.0
>>> b=2.5+4.6j
>>> print(b, type(b))          → (2.5+4.6j) <class 'complex'>
>>> print(b.real)              → 2.5
>>> print(b.imag)              → 4.6
>>> a=-3.5j
>>> print(a, type(a))          → (-0-3.5j) <class 'complex'>
>>> print(a.real)              → 0.0
>>> a=2.5j
>>> print(a, type(a))          → 2.5j <class 'complex'>
>>> print(a.real)              → 0.0
>>> print(a.imag)              → 2.5
>>> a=-1+4.5j
```

```
>>> print(a, type(a))        → (-1+4.5j) <class 'complex'>
>>> print(a.real)            → -1.0
>>> print(a.imag)            → 4.5
>>> a=2+3j
>>> b=3+8j
>>> c=a+b
>>> print(c, type(c))        → (5+11j) <class 'complex'>
>>> d=a-b
>>> print(d, type(d))        → (-1-5j) <class 'complex'>
>>> e=a*b
>>> print(e, type(e))        → (-18+25j) <class 'complex'>
```

Sequence Category data type:

- The purpose of Sequence Category Data Types is that "To store Sequence of values".
- In Python programming, we have 4 data types in Sequence Category . They are:

Str:

- 'str' is one of the pre-defined classes and treated as Sequence Data Type.
- The purpose of str data type is that "To store String data or text data or Alphanumeric data or numeric data or special symbols within double Quotes or single quotes or triple double quotes and triple single quotes. "

Definition: Str is a collection of Characters or Alphanumeric data or numeric data, or any type of data enclosed within double Quotes or single quotes or triple double quotes and triple single quotes.

Type of str data:

In Python Programming, we have two types of Str Data. They are. **a.**

Single Line String Data:

Syntax1:- varname=" Single Line String Data "

(OR)

Syntax2:- varname=' Single Line String Data '

With the help double Quotes (" ") and single Quotes (' ') we can store single line str data only but not possible to store multi line string data. **b. Multi Line String Data:**

Syntax1:- $varname = $ " " " *This is the multiple line examples.* " " "

(OR)

Syntax2:- $varname = ''' $ *This is the multiple line examples.* $'''$

With the help OF triple double Quotes (" " " " " ") and Tripple single Quotes (' ' ' ' ' ') we can store single line str data and multi-line string data.

Example:

```
>>> s1="Python"
>>> print(s1,type(s1))          → Python <class 'str'> >>>
s2='Python'
>>> print(s2,type(s2))                → Python <class 'str'>
>>> s3='A'
>>> print(s3,type(s3))                → A <class 'str'>
>>> s4="A"
>>> print(s4,type(s4))                → A <class 'str'>
>>> s5="Python3.14"
>>> print(s5,type(s5))                → Python3.14 <class 'str'>
>>> s6="123456"
>>> print(s6,type(s6))                → 123456 <class 'str'>
>>> s7="Python Programming 1234$Rossum_Guido"
>>> print(s7,type(s7))              →
Python Programming 1234$Rossum_Guido <class 'str'>
>>> addr1= "Guido van Rossum          →
SyntaxError: unterminated string literal (detected at line 1)
>>> addr1= ' Guido van Rossum          →
SyntaxError: unterminated string literal (detected at line 1)
>>> addr1=" " "Guido van Rossum
... HNO:3-4,Hill Side
... Python Software Foundation
... Nether Lands-56"""
>>> print(addr1,type(addr1))
                        Guido van Rossum
                        HNO:3-4,Hill Side
                        Python Software Foundation
                        Nether Lands-56 <class 'str'>
>>> addr2=' ' 'Travis Oliphant
... HNO:23-45, Seaside
... NumPy Organization
... nethrlands-67 '''
>>> print(addr2,type(addr2))
                        Travis Oliphant
                        HNO:23-45, Seaside
        NumPy Organization
                        nethrlands-67  <class 'str'>
>>> s7=" " " Python " " "
```

```
>>> print(s7,type(s7))                → Python <class 'str'>
>>> s8=' ' ' Python ' ' '
>>> print(s8,type(s8))                → Python <class 'str'>
>>> s9=" " " Z " " "
>>> print(s9,type(s9))                → Z <class 'str'>
>>> s10=' ' ' K ' ' '
>>> print(s10,type(s10))              → K <class 'str'>
>>> s1="Python Programming"
>>> print(s1,type(s1))            → Python Programming <class 'str'>
>>> s1              → ' Python Programming '
>>> s7=" " " Python " " "
>>> print(s7,type(s7))            → Python <class 'str'>
>>> s7              → ' Python '
>>> s1=" " "Python is an oop lang ...
python is also fun Prog lang.
... Python is also Modular Prog lang" " ".
>>> print(s1,type(s1))            →
                        Python is an oop lang.
                        python is also fun Prog lang.
                        Python is also Modular Prog lang <class 'str'>
>>> s1          → ' Python is an oop lang \n python is also fun Prog lang \n Python is also
Modular Prog lang'.
```

Operation on str Data:

On str data, we can perform Two Types of Operations. They are.

a. Indexing

b. Slicing

a. Indexing:

- The Process of Obtaining Single Value from given Main Str object is called Indexing.
- Syntax: strobj[Index]
- Here strobj is an object of <class,'str'>
- Index represents Either +Ve Index or -Ve Index
- If we enter Valid Index value, then we get Corresponding Indexed Value.
- If we enter Invalid Index value, then we get Index Error.

Examples:

```
>>> s="PYTHON"
>>> print(s, type(s))          → PYTHON <class 'str'>
>>> print(s[0])                → P
>>> print(s[-6])               → P
>>> print(s[-1])               → N
```

```
>>> print(s[5])          → N
>>> print(s[3])          → H
>>> print(s[-3])         → H
>>> print(s[-2])         → O
>>> print(s[-5])         → Y
>>> print(s[2])          → T
>>> s[2]                 → 'T'
>>> s[-2]                → 'O'
>>> print(s[8])          → IndexError: string index out of range
>>> print(s[-12])        → IndexError: string index out of range
>>> s="123453456"
>>> print(s, type(s))    → 123453456 <class 'str'>
>>> s[2]                 → '3'
>>> s[-1]                → '6'
>>> s[0]                 → '1'
>>> s[len(s)-1]          → '6'
>>> s[-len(s)]           → '1'
>>> s="JAVA"
>>> print(s, type(s))    → JAVA <class 'str'>
>>> len(s)               → 4
>>> s[len(s)-1]          → 'A'
>>> s[-len(s)]           → 'J'
>>> s[len(s)-len(s)]     → 'J'
>>> s[-2]                → 'V'
>>> s[2]                 → 'V'
>>> s[len(s)]            → IndexError: string index out of range
>>> s="MISSISSIPPI"
>>> s[len(s)-1]          → 'I'
>>> s[-len(s)]           → 'M'
>>> s[len(s)]            → IndexError: string index out of range
```

b. Slicing:

- The Process obtaining Range of Values OR Sub String from Given main Str Object is called Slicing.
- To Perform Slicing Operations, we have 5 Syntaxes. They are.

→*Syntax*1: *strobj*[*BEGIN*: *END*]:

This Syntax generates Range of Values OR Sub String from BEGIN Index to END-1 Index provided BEGIN<END Otherwise we never get any result OR Space OR ' ' as a Result

Examples:

```
>>> s="PYTHON"
>>> print(s, type(s))    → PYTHON <class 'str'>
```

```
>>> s[1:-2]          → 'YTH'
>>> s[2:-1]          → 'THO'
>>> s[-3:1]          → '  '
```

→***Syntax*2**: ***strobj*[*BEGIN*:]:**

- In This Syntax, we are Specifying BEGIN Index and we did not Specify END Index.
- If we do not specify END Index, then PVM Takes END Index as len(strobj)

(OR)

- If we do not specify END Index, then PVM Takes BEGIN INDEX Character to Last Character.

Examples:

```
>>> s="PYTHON"
>>> print(s, type(s))    → PYTHON <class 'str'>
>>> s[2:]                → 'THON'
>>> s[0:]                → 'PYTHON'
>>> s[3:]                → 'HON'
>>> s[4:]                → 'ON'
>>> s[1:]                → 'YTHON'
>>> s[-3:]               → 'HON'
>>> s[-2:]               → 'ON'
>>> s[-6:]               → 'PYTHON'
>>> s[-5:]               → 'YTHON'
>>> s[-4:]               → 'THON'
>>> s[-1:]               → 'N'
```

Special Points:

```
>>> s="PYTHON"
>>> print(s, type(s))    → PYTHON <class 'str'>
>>> s[-12:]              → 'PYTHON'
>>> s[-23:]              → 'PYTHON'
>>> s[23:]               → ''
>>> s[2:4]               → 'TH'
>>> s[12:44]             → ''
>>> s[-66:-1]            → 'PYTHO'
>>> s[2:122]             → 'THON'
>>> s[2:-122]            → ''
>>> s[2:-1]              → 'THO'
>>> s[2:-22]             → ''
```

→***Syntax*3**: ***strobj*[: *END*]:**

- In This Syntax, we are Specifying END Index, and we did not Specify BEGIN Index.
- If we do not specify BEGIN Index, then PVM Takes BEGIN Index as 0 OR -len(strobj)

(OR)

- If we do not specify BEGIN Index, then PVM Takes First Character to END-1 Index.

Examples:

```
>>> s="PYTHON"
>>> print(s, type(s))        → PYTHON <class 'str'>
>>> s[:3]                    → 'PYT'
>>> s[:6]                    → 'PYTHON'
>>> s[:5]                    → 'PYTHO'
>>> s[:4]                    → 'PYTH'
>>> s="PYTHON"
>>> print(s, type(s))        → PYTHON <class 'str'>
>>> s[:-3]                   → 'PYT'
>>> s[:-2]                   → 'PYTH'
>>> s[:-1]                   → 'PYTHO'
>>> s[:-4]                   → 'PY'
>>> s[:-5]                   → 'P'
```

→*Syntax*4: *strobj*[:]:

- In This Syntax, we are not Specifying BEGIN Index and END Index
- If we do not specify BEGIN Index and END Index, then PVM Takes BEGIN Index as 0 OR len(strobj) and END Index as len(strobj) or -1.

(OR)

- If we do not specify BEGIN Index and END Index, then PVM Takes from First Character(0th index or -len(strobj) index) to last Character (len(strobj) or -1).
- Hence This Syntax always gives Total String Data. **Examples:**

```
>>> s="PYTHON"
>>> print(s, type(s))        → PYTHON <class 'str'>
>>> s[:]                     → 'PYTHON'
>>> s="JAVA PROG"
>>> print(s, type(s))        → JAVA PROG <class 'str'>
>>> s[:]                     → 'JAVA PROG'
>>> s[0:]                    → 'JAVA PROG'
>>> s[:len(s)]               → 'JAVA PROG'
>>> s[-len(s):]              → 'JAVA PROG'
```

NOTE: All the above Syntaxes are Extracting the data from strobj in Forward Direction with Default Step value 1.

→*Syntax*5: *strobj*[*BEGIN*: *END*: *STEP*]:

RULE-1: Here BEGIN, END and STEP Values can be either +VE or -VE.

RULE-2: If STEP Value is +VE then PVM Takes the Range of Characters from BEGIN Index to END-1 Index in FORWARD Direction with Step Value provided BEGIN INDEX < END INDEX Otherwise we get Space or ' ' as a result.

RULE-3: If STEP Value is -VE then PVM Takes the Range of Characters from BEGIN Index to END+1 Index in BACKWARD Direction with Step Value provided BEGIN INDEX > END INDEX Otherwise we get Space or ' ' as a result.

RULE-4: In FORWARD Direction, If END Value is 0 Then we get Space or ' ' as a result.

RULE-5: In BACKWARD Direction, If END Value is -1 Then we get Space or ' ' as a result.

Examples: RULE-3:

```
>>> s="PYTHON"
>>> print(s)          → PYTHON
>>> s[ : : ]          → 'PYTHON'
>>> s[ : :1]          → 'PYTHON'
>>> s[ : :-1]         → 'NOHTYP'
>>> s[ : :-2]         → 'NHY'
>>> s[0:6:-2]         → ''
>>> s[5:0:-1]         → 'NOHTY'
>>> s[ : :-3]         → 'NT'
>>> s[4: :-2]         → 'OTP'
>>> s[: :2]           → 'PTO'
>>> s[: :2][ : :-1]   → 'OTP'
>>> s[-1:-7:-1]       → 'NOHTYP'
>>> s[-1 : :-1]  → 'NOHTYP'
>>> s[-1: :-2]   → 'NHY'
>>> s[-2: :-2]   → 'OTP'
>>> s[2:6:][ : :-1] →'NOHT' >>>
s[2:6:]           → 'THON'
```

Examples:RULE-4:

```
>>> s="PYTHON"
>>> print(s)   → PYTHON
>>> s[2:0:1]   → ' '
>>> s[:0:1]        → ' '
>>> s="JAVA PROG"
>>> print(s)   → JAVA PROG
>>> s[:0:1]        → ''
>>> s[:0]      → ' ' >>> s[:0:3]
 → ' ' Examples: RULE-5:
>>> s="PYTHON"
>>> print(s)   → PYTHON
>>> s[ : :-1]      → 'NOHTYP'
>>> s[ :-1:-1]   → ' '
```

>>> s[:-1:-3] → ' ' >>> s[:-1:-4] → ' ' **Special Points:**

>>> "PYTHON"[: :-1] → 'NOHTYP'

>>> "PYTHON"[: :-1]=="PYTHON" →False

>>> "PYTHON"[: :2]=="PYTHON"[: :-1][: :2] →False

>>> "MOM"[: :-1]=="MOM" → True >>> "MOM"[: :-1]=="MOM"[: :] → True

>>> "LIRIL"[: :-1]=="LIRIl"[: :-1] → False

Binary Sequence Types:

Byte:

- 'Bytes' is one of the pre-defined class names and treated as Sequence Data Type.
- The purpose of bytes data type is that "To organize the Numerical Integer values ranges from (0,256) for the implementation of End-to-End Encryption".
- Bytes data type does not contain any symbolic notations for organizing (0,256) data. But we can convert any type of Value(s) into bytes type by using bytes()
- $Syntax$: $varname = bytes(object)$
- An object of bytes belongs to immutable because bytes object does not support Item assignment.
- On the object of bytes, we can perform both Indexing and Slicing Operations.
- An object of bytes maintains Insertion Order (Which is nothing but, whatever the order we insert the data, In the same order data will be displayed).

Examples:

lst=[10,34,56,100,256,0,102]

>>> print(lst, type(lst)) → [10, 34, 56, 100, 256, 0, 102] <class 'list'>

>>> b=bytes(lst) → ValueError: bytes must be in range(0, 256)

>>> lst=[10,-34,56,100,255,0,102]

>>> print(lst, type(lst)) → [10, -34, 56, 100, 255, 0, 102] <class 'list'>

>>> b=bytes(lst) → ValueError: bytes must be in range(0, 256)

>>> lst=[10,34,56,100,255,0,102]

>>> print(lst, type(lst)) → [10, 34, 56, 100, 255, 0, 102] <class 'list'>

>>> b=bytes(lst)

>>> print(b,type(b)) → b'\n"8d\xff\x00f' <class 'bytes'>

>>> print(b,type(b)) → b'\n"8d\xff\x00f' <class 'bytes'>

>>> print(b[0]) → 10

>>> print(b[-1]) → 102

>>> print(b[0:4]) → b'\n"8d'

>>> b[0]=123 → TypeError: 'bytes' object does not support item assignment

Bytes array:

- 'Byte array' is one of the pre-defined class names and treated as Sequence Data Type.
- The purpose of byte array data type is that "To organize the Numerical Integer values ranges from (0,256) for the implementation of End-to-End Encryption".
- Byte array data type does not contain any symbolic notations for organizing (0,256) data. But we can convert any type of Value(s) into byte array type by using bytearray()
- Syntax: varname=bytearray(object)
- An object of byte array belongs to mutable because byte array object supports Item Assignment.
- On the object of byte array, we can perform both Indexing and Slicing Operations.
- An object of byte array maintains Insertion Order (Which is nothing but, whatever the order we insert the data, In the same order data will be displayed).

NOTE: The Functionality of bytes and bytearray are exactly same but bytes object belongs to Immutable because bytes object does not support Item assignment where bytearray object belongs to Mutable because bytearray object supports Item Assignment.

Example:

```
>>> tpl=(10,34,56,100,256,0,102)
>>> print(tpl, type(tpl))          → (10, 34, 56, 100, 256, 0, 102) <class 'tuple'>
>>> ba=bytearray(tpl)              → ValueError: byte must be in range(0, 256)
>>> tpl=(-10,34,56,100,255,0,102)
>>> print(tpl, type(tpl))          → (-10, 34, 56, 100, 255, 0, 102) <class 'tuple'>
>>> ba=bytearray(tpl)          → ValueError: byte must be in range(0, 256) >>>
tpl=(10,34,56,100,255,0,102)
>>> print(tpl, type(tpl))          → (10, 34, 56, 100, 255, 0, 102) <class 'tuple'>
>>> ba=bytearray(tpl)
>>> print(ba, type(ba))        → bytearray(b'\n"8d\xff\x00f') <class 'bytearray'> >>>
tpl=(10,34,56,100,255,0,102)
>>> print(tpl, type(tpl))          → (10, 34, 56, 100, 255, 0, 102) <class 'tuple'>
>>> ba=bytearray(tpl)
>>> print(ba, type(ba))        → bytearray(b'\n"8d\xff\x00f') <class 'bytearray'> >>>
print(ba, type(ba),id(ba))  → bytearray(b'\n"8d\xff\x00f') <class 'bytearray'>
2149788291504
>>> ba[0]                          → 10
>>> ba[-1]                         → 102
>>> ba[0]=123                  → # Updating / item assignment on bytearray object--
allowed >>> print(ba, type(ba),id(ba))       → bytearray(b'{"8d\xff\x00f') <class
'bytearray'> 2149788291504
>>> b=bytes(ba)                    → # Converting bytearray into bytes
>>> print(b,type(b))               → b'{"8d\xff\x00f' <class 'bytes'>
```

```
>>> b[0]                          → 123
>>> b[0]=12  → TypeError: 'bytes' object does not support item assignment
>>> x=bytearray(b)                → # Converting bytes into bytearray
>>> print(x, type(x),id(x))       → bytearray(b'{"8d\xff\x00f') <class 'bytearray'>
2149788291888
>>> x[1]                          → 34
>>> x[1]=234
>>> print(x, type(x),id(x))       → bytearray(b'{\xea8d\xff\x00f') <class 'bytearray'>
2149788291888
```

Range:

- 'Range' is one of the pre-defined classes and treated as Sequence data type.
- The purpose of range data type is that "To Store Sequence of Numerical Integer Values with Equal Interval of Value."
- An object of range belongs to Immutable because range object does not support Item Assignment.
- On the object of range, we can perform Both Indexing Slicing Operations • The range of Values can be stored either in forward or Backward Directions.
- range data type contains 3 syntaxes. They are:

→ $\boldsymbol{Syntax-1}$: $\boldsymbol{varname = range(Value)}$

This Syntax generates range of values from 0 to Value-1 **Examples:**

```
>>> r=range(6) >>>
print(r, type(r)) range(0, 6)
<class 'range'> >>> for v
in r:
...         print(v)
...
                0
                1
                2
                3
                4
                5
```

→$\boldsymbol{Syntax-2}$: $\boldsymbol{varname = range(Begin, End)}$ This Syntax generates range of values from Begin to End-1

Examples:

```
>>> r=range(10,16)
>>> print(r, type(r)) range(10,
16) <class 'range'> >>> for val
in r:
```

```
...             print(val)
...
                10
                11
                12
                13
                14
                15
>>> for val in range(10,16):
...     print(val)
...
                10
                11
                12
                13
                14
                15
>>> for val in range(10,16):
...             print(val, end=" ")   #    10 11 12 13 14 15
```

NOTE: The Syntax-1 and Syntax-2 generates range of values in FORWARD DIRECTION with Default Interval 1.

$\rightarrow Syntax-3$: $varname = range(Begin, End, Step)$:

This Syntax generates range of values from Begin to End-1 by maintaining Equal Interval value in the form Step either in Forward Direction or Back direction. 'range' object does not support item assignment.

Examples:

```
>>> r=range(10,21,2)
>>> for val in r: ...
  print(val)
...
                10
                12
                14
                16
                18
                20
>>> for val in range(10,21,2):
...             print(val)
...
                10
```

12
14
16
18
20

List Category data types (Collection or Data Structure):

- The purpose of List Category Data Types is that "To store Multiple Values either of same Type OR Different Type OR Both the Types in Single Object with Unique and Duplicate Values".
- In Python Programming, we have 2 data types in List Category. They are.

List (mutable):

- 'list' is one of the pre-defined classes and treated as list data type.
- The purpose of list data type is that " To store Multiple Values either of same Type OR Different Type OR Both the Types in Single Object with Unique and Duplicate Values".
- The Values / Elements of list must be stored / Organized with Square Brackets [ ] and Values must be separated by comma.
- An object of list maintains Insertion order.
- On object of list, we can perform Both Indexing and Slicing Operations.
- An object of list belongs to Mutable.
- With respect to list class, we can create 2 types of list objects. They are. **a. Empty List:**
- An empty list is one, which does not contain any Elements and whose length is 0.
- Syntax: varname=[]
  (OR)
  varname=list()

b. Non-Empty List:

- A non-empty list is one, which contains Elements and whose length is >0.
- Syntax: varname=[Val1,Val2....Val-n]
  (OR)
  varname=list(object)

Examples:

```
>>> l2=[20,"Rossum",34.56,True,2+3j]
```

```
>>> print(l2,type(l2),id(l2))          → [20, 'Rossum', 34.56, True, (2+3j)] <class
'list'> 2705927635008
>>> l2[0]=30
>>> print(l2,type(l2),id(l2))          → [30, 'Rossum', 34.56, True, (2+3j)] <class
'list'> 2705927635008
>>> l2[2:4]=[44.55,False]
>>> print(l2,type(l2),id(l2))          → [30, 'Rossum', 44.55, False, (2+3j)] <class
'list'> 2705927635008
>>> s="MISSISSIPPI"
>>> l3=list(s)
>>> print(l3,type(l3))                 → ['M', 'I', 'S', 'S', 'I', 'S', 'S', 'I', 'P', 'P', 'I'] <class
'list'>
```

Pre-defined functions in list:

We know that, on the object of list, we can perform Both Indexing and Slicing Operations. Along with Indexing and Slicing Operations, we can Perform Various Operations by using Pre-Defined Functions present in list object. They are. **a. append():**

- Syntax: listobj.append(Value)
- This Function is used for adding the values to list object at end. **Example:**

```
>>> lst=[10,"Rossum",34.56]
>>> print(lst, id(lst))                → [10, 'Rossum', 34.56] 2705927639808
>>> lst.append("PYTHON")
>>> print(lst, id(lst))                → [10, 'Rossum', 34.56, 'PYTHON'] 2705927639808
>>> lst.append(True)
>>> lst.append(1+2.5j)
>>> print(lst, id(lst))                → [10, 'Rossum', 34.56, 'PYTHON', True, (1+2.5j)]
2705927639808
```

b. insert():

- Syntax: listobj.insert(Index, Value)
- This Function is used for adding the value to list object at Specified Index
- When we enter Invalid Positive Index then the value inserted at Last/End of List object • When we enter Invalid Negative Index then the value inserted at First of List object

Examples:

```
>>> lst=[10,"Rossum",34.56]
>>> print(lst, id(lst))                → [10, 'Rossum', 34.56] 2705923376704
>>> lst.insert(2,"PYTHON")
>>> print(lst, id(lst))                → [10, 'Rossum', 'PYTHON', 34.56] 2705923376704
>>> lst[-1]=44.56
>>> print(lst, id(lst))                → [10, 'Rossum', 'PYTHON', 44.56] 2705923376704
>>> lst.insert(2,True)
```

```
>>> print(lst, id(lst))          → [10, 'Rossum', True, 'PYTHON', 44.56]
2705923376704
>>> lst.insert(-1,2+3j)
>>> print(lst, id(lst))          → [10, 'Rossum', True, 'PYTHON', (2+3j), 44.56]
2705923376704
>>> lst=[10,"Rossum",34.56]
>>> print(lst, id(lst))          → [10, 'Rossum', 34.56] 2705927639296
>>> lst.insert(10,"PYTHON")
>>> print(lst, id(lst))          → [10, 'Rossum', 34.56, 'PYTHON'] 2705927639296
>>> lst.insert(-10,"HYD")
>>> print(lst, id(lst))          → ['HYD', 10, 'Rossum', 34.56, 'PYTHON']
2705927639296
```

c. clear():

- Syntax: listobj.clear()
- This Function is used for Removing all the elements of Non-Empty List object • When we call clear() on empty list object then we get No Output / None **Examples:**

```
>>> lst=[10,"Rossum",34.56]
>>> print(lst, id(lst), len(lst))      → [10, 'Rossum', 34.56] 2705927639808 3
>>> lst.clear()
>>> print(lst, id(lst),len(lst))       → [] 2705927639808 0
>>> lst.clear()                        → No Output
                (OR)
>>> print(lst.clear())                 → None
>>> [].clear()                         → No Output
                (OR)
>>> print([].clear())
None
>>> print(list().clear())              → None
```

d. remove() - Based on Value:

- Syntax: listobj.remove(Value)
- This Function is used for Removing the First Occurrence of Specified Element of list object.
- If the Specified Element does not exist in list objects, then we get ValueError.

Examples:

```
>>> lst=[10,"Rossum",34.56,"Python"]
>>> print(lst, len(lst))          → [10, 'Rossum', 34.56, 'Python'] 4
>>> lst.remove("Rossum")
>>> print(lst, len(lst))          → [10, 34.56, 'Python'] 3
>>> lst.remove(34.56)
```

```
>>> print(lst, len(lst))          → [10, 'Python'] 2
>>> lst.remove("Python")
>>> print(lst, len(lst))          → [10] 1
>>> lst.remove(10)
>>> print(lst,len(lst))           → [] 0
>>> lst.remove("Python")          → ValueError: list.remove(x): x not in list
>>> list().remove(100)            → ValueError: list.remove(x): x not in list
>>> lst1=[10,20,30,10,30,"Python",True]
>>> print(lst1,len(lst1))         → [10, 20, 30, 10, 30, 'Python', True] 7
>>> lst1.remove(10)
>>> print(lst1,len(lst1))         → [20, 30, 10, 30, 'Python', True] 6
>>> lst1.remove(10)
>>> print(lst1,len(lst1))         → [20, 30, 30, 'Python', True] 5
>>> lst1.remove(30)
>>> print(lst1,len(lst1))         → [20, 30, 'Python', True] 4
>>> lst1.remove(30)
>>> print(lst1,len(lst1))         → [20, 'Python', True] 3
```

e. pop(index) - Index Based:

- Syntax: listobj.pop(index)
- This Function is used for Removing the Element of list obj based on Index.
- If the Index is invalid, then we get IndexError **Examples:**

```
>>> lst1=[10,20,30,10,30,"Python",True]
>>> print(lst1)          → [10, 20, 30, 10, 30, 'Python', True]
>>> lst1.pop(3)          → 10
>>> print(lst1)          → [10, 20, 30, 30, 'Python', True]
>>> lst1.pop(-4)         → 30
>>> print(lst1)          → [10, 20, 30, 'Python', True]
>>> lst1.pop(0)          → 10
>>> print(lst1)          → [20, 30, 'Python', True]
>>> lst1.pop(0)          → 20
>>> print(lst1)          → [30, 'Python', True]
>>> lst1.pop(0)          → 30
>>> print(lst1)          → ['Python', True]
>>> lst1.pop(0)          → 'Python'
>>> print(lst1)          → [True]
>>> lst1.pop(0)          → True
>>> print(lst1)          → []
>>> lst1.pop(0)          → IndexError: pop from empty list
>>> list().pop(-4)       → IndexError: pop from empty list
>>> list().pop(4)        → IndexError: pop from empty list
```

f. pop():

- Syntax: listobj.pop()
- This Function is used for Removing always Last Element of List object.
- If we call pop() on empty list object then we get IndexError **Examples:**

```
>>> lst=[10,"Rossum",34.56,"Python"]
>>> print(lst)          → [10, 'Rossum', 34.56, 'Python']
>>> lst.pop()          → 'Python'
>>> print(lst)          → [10, 'Rossum', 34.56]
>>> lst.pop()          → 34.56
>>> print(lst)          → [10, 'Rossum']
>>> lst.pop()          → 'Rossum'
>>> print(lst)          → [10]
>>> lst.pop()          → 10
>>> print(lst)          → []
>>> lst.pop()          → IndexError: pop from empty list
>>> list().pop()       → IndexError: pop from empty list
>>> [].pop()           → IndexError: pop from empty list
```

NOTE: del operator--Most Imp

Syntax1: del objname[Index] → Removing the element based on Index

Syntax2: del objname[Begin : End : Step] → Removing the Elements Based in Slicing Operations

Syntax3: del objname → Removing the entire object **Examples:**

```
>>> lst=[10,"Sagatika",66.66,"OUCET","HYD"]
>>> print(lst, type(lst),id(lst))          → [10, 'Sagatika', 66.66, 'OUCET', 'HYD'] <class 'list'> 2302481486656
>>> del lst[-2]
>>> print(lst, type(lst),id(lst))          → [10, 'Sagatika', 66.66, 'HYD'] <class 'list'> 2302481486656
>>> del lst[0:2]
>>> print(lst, type(lst),id(lst))          → [66.66, 'HYD'] <class 'list'> 2302481486656
>>> lst=[10,"Sagatika",66.66,"OUCET","HYD"] >>>
del lst[::2]
>>> print(lst, type(lst),id(lst))          → ['Sagatika', 'OUCET'] <class 'list'> 2302485776384
>>> del lst
>>> print(lst, type(lst),id(lst))          → NameError: name 'lst' is not defined.
```

g. index():

- Syntax: listobj.index(Value)
- This Function is used for Finding Index of First Occurrence of Specified Element in List object. If the Specified Element does not present in List object, then we get ValueError.

Examples:

```
>>> lst=[10,20,30,40,10,60,70,10,30]
>>> print(lst)              → [10, 20, 30, 40, 10, 60, 70, 10, 30]
>>> lst.index(10)          → 0
>>> lst.index(20)          → 1
>>> lst.index(30)          → 2
>>> lst.index(300)         → ValueError: 300 is not in list
>>> list().index(10)       → ValueError: 10 is not in list
>>> [].index(-12)          → ValueError: -12 is not in list
```

h. copy() – Shallow Copy:

- This Function is Used for Copying the content of One Object into another Object (Implements Shallow Copy).
- Syntax: Listobject2=listobj1.copy() **Examples:**

```
>>> lst1=[10,"Rossum",34.56]
>>> print(lst1,type(lst1),id(lst1)) → [10, 'Rossum', 34.56] <class 'list'>
2302481827584 >>> lst2=lst1.copy()  # Shallow Copy
>>> print(lst2,type(lst2),id(lst2)) → [10, 'Rossum', 34.56] <class 'list'> 2302481486656
>>> lst1.append("PYTHON")
>>> print(lst1,type(lst1),id(lst1)) → [10, 'Rossum', 34.56, 'PYTHON'] <class 'list'>
2302481827584
>>> print(lst2,type(lst2),id(lst2)) → [10, 'Rossum', 34.56] <class 'list'> 2302481486656
>>> lst2.append("NL")
>>> print(lst2,type(lst2),id(lst2)) → [10, 'Rossum', 34.56, 'NL'] <class 'list'>
2302481486656
>>> print(lst1,type(lst1),id(lst1)) → [10, 'Rossum', 34.56, 'PYTHON'] <class 'list'>
2302481827584
```

Deep Copy—Examples:

```
>>> lst1=[10,"Rossum",34.56]
>>> print(lst1,type(lst1),id(lst1)) → [10, 'Rossum', 34.56] <class 'list'>
2302481517760 >>> lst2=lst1 # Deep Copy
>>> print(lst2,type(lst2),id(lst2)) → [10, 'Rossum', 34.56] <class 'list'>
2302481517760 >>> lst1.append("HYD")
>>> print(lst1,type(lst1),id(lst1)) → [10, 'Rossum', 34.56, 'HYD'] <class 'list'>
2302481517760
>>> print(lst2,type(lst2),id(lst2)) → [10, 'Rossum', 34.56, 'HYD'] <class 'list'>
2302481517760
>>> lst2.remove("Rossum")
>>> print(lst1,type(lst1),id(lst1)) → [10, 34.56, 'HYD'] <class 'list'> 2302481517760 >>>
print(lst2,type(lst2),id(lst2)) → [10, 34.56, 'HYD'] <class 'list'> 2302481517760
```

i. count():

- Syntax: listobj1.count(Value)
- This Function is used for finding / Counting Number of occurrences of Specified Value of List object. If the Specified Value does not exist in list object then we get zero as a Result.

Examples:

```
>>> lst=[10,20,30,10,20,50,60,70,10]
>>> print(lst)              → [10, 20, 30, 10, 20, 50, 60, 70, 10]
>>> lst.count(10)           → 3
>>> lst.count(20)           → 2
>>> lst.count(30)           → 1
>>> lst.count(40)           → 0
>>> lst.count("PYTHON" )    → 0
>>> list().count(10)    → 0 >>>
[].count(10)     → 0
```

j. reverse():

- Syntax: listobj.reverse()
- This Function is used for reversing(Front elements to back and back elements to Front) the elements of list object in same list itself.

Examples:

```
>>> lst1=[10,"Rossum",34.56,"PYTHON"]
>>> print(lst1,id(lst1))          → [10, 'Rossum', 34.56, 'PYTHON'] 2302485781376
>>> lst1.reverse()
>>> print(lst1,id(lst1))          → ['PYTHON', 34.56, 'Rossum', 10] 2302485781376
>>> lst2=[10,20,30,100,200,300]
>>> print(lst2,id(lst2))   → [10, 20, 30, 100, 200, 300] 2302485776384 >>>
lst2.reverse()
>>> print(lst2,id(lst2))          → [300, 200, 100, 30, 20, 10] 2302485776384
>>> lst2=[10,20,30,100,200,300]
>>> lst3=lst2.reverse()
>>> print(lst2,id(lst2))          → [300, 200, 100, 30, 20, 10] 2302481517760
>>> print(lst3)                   → None
>>> list().reverse()   # No Output
                  (OR)
>>> print(list().reverse())       → None
```

k. extend():

- Syntax: listobj1.extend(listobj2)
- This Function is used for Merging / Combining The values of listobj2 with ListObj1. Hence Listobj1 contains Its Own Elements and Elements of listobj2.
- Syntax: listobj1=listobj1+ listobj2+ .....+listobj-n
- By using + Operator also we can Merge OR Combine Multiple elements of list objects

Examples:

```
>>> lst1=[10,20,30,40]
>>> lst2=["Python", "Java"]
>>> lst3=["Rossum", "Gosling"]
>>> lst1.extend(lst2,lst3)          → TypeError: list.extend() takes exactly one argument (2 given)
#### TO Solve the above Error
>>> lst1.extend(lst2)
>>> lst1.extend(lst3)
>>> print(lst1)                     → [10, 20, 30, 40, 'Python', 'Java', 'Rossum', 'Gosling']
        OR
>>> lst1=[10,20,30,40]
>>> lst2=["Python", "Java"]
>>> lst3=["Rossum", "Gosling"]
>>> lst1=lst1+lst2+lst3  # Used + Operator for Merging
>>> print(lst1)                     → [10, 20, 30, 40, 'Python', 'Java', 'Rossum', 'Gosling']
```

l. sort():

Syntax1: listobj.sort()----->Sorts the given List data in Ascending Order
Syntax2: listobj.sort(revserse=False)----->Sorts the given List data in Ascending Order
Syntax3: listobj.sort(reverse=True)--->Sort the given List data in Descending Order

Examples:

```
>>> lst=[10,-2,12,56,13,-7,45,6]
>>> print(lst, id(lst))             → [10, -2, 12, 56, 13, -7, 45, 6] 2302485781696 >>> lst.sort()
>>> print(lst, id(lst))   → [-7, -2, 6, 10, 12, 13, 45, 56] 2302485781696 >>> lst=[10,-2,12,56,13,-7,45,6]
>>> print(lst, id(lst))             → [10, -2, 12, 56, 13, -7, 45, 6] 2302481486656 >>> lst.sort()
>>> print(lst, id(lst))   → [-7, -2, 6, 10, 12, 13, 45, 56] 2302481486656 >>> lst.reverse()
>>> print(lst, id(lst))             → [56, 45, 13, 12, 10, 6, -2, -7] 2302481486656
>>> lst=[10,-2,12,56,13,-7,45,6]
>>> print(lst, id(lst))   → [10, -2, 12, 56, 13, -7, 45, 6] 2302485781696 >>> lst.sort(reverse=True)
```

>>> print(lst, id(lst)) → [56, 45, 13, 12, 10, 6, -2, -7] 2302485781696 >>> lst=[10,-2,12,56,13,-7,45,6]
>>> print(lst, id(lst)) → [10, -2, 12, 56, 13, -7, 45, 6] 2302481486656
>>> lst.sort(reverse=False)
>>> print(lst, id(lst)) → [-7, -2, 6, 10, 12, 13, 45, 56] 2302481486656
>>> lst=["Trump", "Zaki", "Biden", "Putin", "Rossum", "Alen"]
>>> print(lst) → ['Trump', 'Zaki', 'Biden', 'Putin', 'Rossum', 'Alen'] >>> lst.sort(reverse=True)
>>> print(lst) → ['Zaki', 'Trump', 'Rossum', 'Putin', 'Biden', 'Alen'] >>> lst=["Trump", "Zaki", "Biden", "Putin", "Rossum", "Alen"]
>>> print(lst) → ['Trump', 'Zaki', 'Biden', 'Putin', 'Rossum', 'Alen']
>>> lst.sort()
>>> print(lst) → ['Alen', 'Biden', 'Putin', 'Rossum', 'Trump', 'Zaki']
>>> lst=[10,"Trump",33.33,2+3j,True]
>>> print(lst) → [10, 'Trump', 33.33, (2+3j), True]
>>> lst.sort() → TypeError: '<' not supported between instances of 'str' and 'int'

Copy Technique in Python:

In Python, we have Two Types of Copy Techniques. They are.

a. Shallow Copy

b. Deep Copy

a. Shallow Copy:

- The Properties of Shallow Copy are:
  a) Initial Content of Both the Objects are Same.
  b) The Memory Address of Both Objects are Different.
  c) The Modifications are Independent
  (Whatever the Changes we do on one object, will not Reflect on Other Object)
- To Implement Shallow Copy, we use copy() .

Syntax: Listobject2=Listobject1.copy() **Examples:**

>>> lst1=[10,"Rossum",34.56]
>>> print(lst1, type(lst1), id(lst1)) → [10, 'Rossum', 34.56] <class 'list'> 2302481827584
>>> lst2=lst1.copy() # Shallow Copy
>>> print(lst2, type(lst2), id(lst2)) → [10, 'Rossum', 34.56] <class 'list'> 2302481486656
>>> lst1.append("PYTHON")
>>> print(lst1,type(lst1),id(lst1)) →[10, 'Rossum', 34.56, 'PYTHON'] <class 'list'> 2302481827584

```
>>> print(lst2,type(lst2),id(lst2))          → [10, 'Rossum', 34.56] <class 'list'>
2302481486656
>>> lst2.append("NL")
>>> print(lst2,type(lst2),id(lst2))          → [10, 'Rossum', 34.56, 'NL'] <class 'list'>
2302481486656
>>> print(lst1,type(lst1),id(lst1))          → [10, 'Rossum', 34.56, 'PYTHON'] <class 'list'>
2302481827584
```

b. Deep Copy:

- The Properties of Deep Copy are
  a) Initial Content of Both the Objects are Same.
  b) The Memory Address of Both the Objects are SAME.
  c) The Modifications are Dependent
  (Whatever the Changes we do on one object, will Reflect on Other Object)
- To Implement Deep Copy, we use Assignment Operator (=) .

Syntax: ListObject2 = ListObject1 **Examples:**

```
>>> lst1=[10,"Rossum",34.56]
>>> print(lst1,type(lst1),id(lst1))          → [10, 'Rossum', 34.56] <class 'list'>
2302481517760
>>> lst2=lst1 # Deep Copy
>>> print(lst2,type(lst2),id(lst2))          → [10, 'Rossum', 34.56] <class 'list'>
2302481517760
>>> lst1.append("HYD")
>>> print(lst1,type(lst1),id(lst1))          → [10, 'Rossum', 34.56, 'HYD'] <class 'list'>
2302481517760
>>> print(lst2,type(lst2),id(lst2))          → [10, 'Rossum', 34.56, 'HYD'] <class 'list'>
2302481517760
>>> lst2.remove("Rossum")
>>> print(lst1,type(lst1),id(lst1))          → [10, 34.56, 'HYD'] <class 'list'>
2302481517760
>>> print(lst2,type(lst2),id(lst2))          →[10, 34.56, 'HYD'] <class 'list'>
2302481517760
```

Inner or Nested List:

The Process of Defining one list inside of another list is called Inner or Nested List.

Syntax:- lst=[Val_1, Val_2...[Val_{11},Val_{12}...Val_{1n}], [Val_{21},Val_{22}…Val_{2n}], Val_n] Here

Val_1,Val_2...Val_n are called Values of Outer List.

Here Val_{11},Val_{12}...Val_{1n} are called Values of one Inner List.

Here Val_{21},Val_{22}...Val_{2n} are called Values of another Inner List.

In inner list we can perform Both Indexing and Slicing Operations.

On Inner List, we can apply all pre-defined function of list.

Examples:

```
>>> lst=[100,"Karthik",[18,16,20],[76,75,66],"OUCET"]
>>> print(lst)                → [100, 'Karthik', [18, 16, 20], [76, 75, 66], 'OUCET']
>>> lst[-1]                   →  'OUCET'
>>> lst[-2]                   → [76, 75, 66]
>>> lst[-3]                   → [18, 16, 20]
>>> lst[1]                    → 'Karthik'
>>> lst[0]                    → 100
>>> print(lst[2],type(lst[2])) → [18, 16, 20] <class 'list'>
>>> print(lst[-2],type(lst[-2]))→[76, 75, 66] <class 'list'>
>>> lst[2][0]                     → 18
>>> lst[2][1]                     →  16
>>> lst[2][1]=17
>>> print(lst)                    → [100, 'Karthik', [18, 17, 20], [76, 75, 66], 'OUCET']
        >>>
lst[2].append(16)
>>> print(lst)                    → [100, 'Karthik', [18, 17, 20, 16], [76, 75, 66], 'OUCET']
>>> lst[-2].insert(-2,80)
>>> print(lst)                    → [100, 'Karthik', [18, 17, 20, 16], [76, 80, 75, 66], 'OUCET']
>>> del lst[-2]
>>> print(lst)                    → [100, 'Karthik', [18, 17, 20, 16], 'OUCET']
>>> lst[2].clear()
>>> print(lst)                    → [100, 'Karthik', [], 'OUCET']
>>> del lst[2]
>>> print(lst)                    → [100, 'Karthik', 'OUCET']
>>> lst.insert(2,[16,15,18,17])
>>> print(lst)                → [100, 'Karthik', [16, 15, 18, 17], 'OUCET']
>>> lst.insert(-1,[77,66,78,55])
>>> print(lst)            → [100, 'Karthik', [16, 15, 18, 17], [77, 66, 78, 55], 'OUCET'] >>>
lst[2].sort()
>>> print(lst)   → [100, 'Karthik', [15, 16, 17, 18], [77, 66, 78, 55], 'OUCET'] >>> lst[-
2].sort(reverse=True)
>>> print(lst)                → [100, 'Karthik', [15, 16, 17, 18], [78, 77, 66, 55], 'OUCET']

>>> studlist=[100,"DLPrince",[18,16,19],[78,66,79],"OUCET"]
>>> print(studlist,type(studlist))→[100, 'DLPrince', [18, 16, 19], [78, 66, 79], 'OUCET']
                                          <class 'list'>
>>> print(studlist[2],type(studlist[2]))→[18, 16, 19] <class 'list'>
>>> print(studlist[-2],type(studlist[-2]))→[78, 66, 79] <class 'list'>
>>> studlist[-3].append(17)
>>> print(studlist,type(studlist))→[100, 'DLPrince', [18, 16, 19, 17], [78, 66, 79],'OUCET']
```

```
                                        <class 'list'>
>>> studlist[3].append(68)
>>> print(studlist,type(studlist))→[100, 'DLPrince', [18, 16, 19, 17], [78, 66, 79, 68],
                                        'OUCET'] <class 'list'>
>>> studlist[2].sort()
>>> print(studlist,type(studlist))→[100, 'DLPrince', [16, 17, 18, 19], [78, 66, 79, 68],
                                        'OUCET'] <class 'list'>
>>> studlist[3].sort(reverse=True)
>>> print(studlist,type(studlist))→[100, 'DLPrince', [16, 17, 18, 19], [79, 78, 68, 66],
                                        'OUCET'] <class 'list'>
>>> studlist[2].pop(-2)             →18
>>> studlist[-2].remove(68)
>>> print(studlist,type(studlist))→[100, 'DLPrince', [16, 17, 19], [79, 78, 66], 'OUCET']
                                        <class 'list'>
>>> studlist[2].clear()
>>> print(studlist,type(studlist))→[100, 'DLPrince', [], [79, 78, 66], 'OUCET'] <class 'list'>
>>> studlist[2].append(14)
>>> print(studlist,type(studlist))→[100,'DLPrince', [14], [79, 78, 66], 'OUCET'] <class 'list'>
>>> del studlist[2]
>>> del studlist[-2]
>>> print(studlist,type(studlist))→[100, 'DLPrince', 'OUCET'] <class 'list'>
>>> studlist.insert(2,[17,15,14,19])
>>> print(studlist,type(studlist))→[100, 'DLPrince', [17, 15, 14, 19], 'OUCET'] <class
                                        'list'>
>>> studlist.append([66,78,65,76])
>>> print(studlist,type(studlist))→[100, 'DLPrince', [17, 15, 14, 19], 'OUCET', [66, 78,
                                         65, 76]] <class 'list'>
>>> studlist[-2]                →'OUCET'
>>> len(studlist)               →5
>>> len(studlist[2])            →4
>>> len(studlist[-1])           →4
```

Tuple (immutable):

Properties:

- 'tuple' is one of the pre-defined classes and treated as list data type.
- The purpose of tuple data type is that " To store Multiple Values either of same Type or different Type OR Both the Types in Single Object with Unique and Duplicate Values".
- The Values / Elements of tuple must be stored / Organized with Braces ( ) and Values must be separated by comma.

- An object of tuple maintains Insertion order.
- On object of tuple, we can perform Both Indexing and Slicing Operations.
- An object of tuple belongs to Immutable.
- With respect to tuple class, we can create 2 types of tuple objects. They are:
    a) Empty tuple
    b) Non-Empty tuple

a) Empty tuple:

• An empty tuple is one, which does not contain any Elements and whose length is 0.

Syntax: varname=()

(OR)

varname=tuple()

b) Non-Empty tuple:

• A non-empty tuple is one, which contains Elements and whose length is >0.

Syntax: varname=(Val1,Val2....Val-n)

(OR)

Syntax: varname=Val1,Val2....Val-n

(OR)

varname=tuple(object)

NOTE: The Functionality of tuple is exactly like Functionality of list, but list object belongs to Mutable and tuple object belongs to Immutable.

Examples:

```
>>> t1=(10,20,30,10,50,60,70)
>>> print(t1,type(t1))          → (10, 20, 30, 10, 50, 60, 70) <class 'tuple'>
>>> t2=(100,"Travis",33.33,"HYD")
>>> print(t2,type(t2))          → (100, 'Travis', 33.33, 'HYD') <class 'tuple'>
>>> len(t1)                     → 7
>>> len(t2)                     → 4
>>> t3=100,"Rossum",44.44,"PYTHON"
>>> print(t3,type(t3))          → (100, 'Rossum', 44.44, 'PYTHON') <class 'tuple'>
>>> t1=()
>>> print(t1, type(t1))         → () <class 'tuple'>
>>> len(t1)                     → 0
>>> t2=tuple()
>>> print(t2,type(t2))          → () <class 'tuple'>
>>> len(t2)                     → 0
>>> t=(100,"Rossum",44.44,"PYTHON")
>>> print(t, type(t),id(t))     → (100, 'Rossum', 44.44, 'PYTHON') <class 'tuple'>
2605398318096
>>> t[0]                        → 100
```

```
>>> t[1]                    → 'Rossum'
>>> t[-1]                   → 'PYTHON'
>>> t[0]=200          → TypeError: 'tuple' object does not support item assignment >>>
l1=[100,"Rossum",44.44,"PYTHON"]
>>> print(l1,type(l1))      → [100, 'Rossum', 44.44, 'PYTHON'] <class 'list'>
>>> t1=tuple(l1)
>>> print(t1,type(t1))      → (100, 'Rossum', 44.44, 'PYTHON') <class 'tuple'>
>>> l2=list(t1)
>>> print(l2,type(l2))      → [100, 'Rossum', 44.44, 'PYTHON'] <class 'list'>
>>> t=(100,"Rossum",44.44,"PYTHON")
>>> print(t, type(t))           → (100, 'Rossum', 44.44, 'PYTHON') <class 'tuple'>
>>> t[0:3]                  → (100, 'Rossum', 44.44) >>> t[ : : -1]
 → ('PYTHON', 44.44, 'Rossum', 100)
```

Special Points:

```
>>> a=10
>>> t=tuple(a)                  → TypeError: 'int' object is not iterable
>>> t=tuple(a,)             → TypeError: 'int' object is not iterable >>>
t=tuple((a))          → TypeError: 'int' object is not iterable
>>> t=tuple([a])
>>> print(t, type(t))   → (10,) <class 'tuple'> OR
>>> b=100
>>> t=(b,)
>>> print(t, type(t))           → (100,) <class 'tuple'>

>>> b=12.34
>>> t=(b)
>>> print(t, type(t))       → 12.34 <class 'float'> >>> t=(b,)
>>> print(t, type(t))           → (12.34,) <class 'tuple'>
```

Pre-defined Function in tuple:

- We know that on the object of tuple we can perform Both Indexing & Slicing Operations.
- Along with these operations, we can also perform other operations by using following pre-defined Functions.

1)index()
2)count()

Examples:

```
>>> t1=(10,"RS",45.67)
>>> print(t1,type(t1))          → (10, 'RS', 45.67) <class 'tuple'>
>>> t1.index(10)                → 0
```

```
>>> t1.index("RS")                  → 1
>>> t1=(10,"RS",45.67)
>>> print(t1,type(t1))              → (10, 'RS', 45.67) <class 'tuple'>
>>> t1.count(10)                    → 1
>>> t1.count(100)                   → 0
>>> t1=(10,0,10,10,20,0,10)
>>> print(t1,type(t1))              → (10, 0, 10, 10, 20, 0, 10) <class 'tuple'>
>>> t1.count(10)                    → 4
>>> t1.count(0)                     → 2
>>> t1.count(100)                   → 0
```

- The Functions do not present in tuple append(), insert(), remove(), clear(), pop(index), pop(), reverse(), sort(), copy(), extend().

NOTE:- By Using del Operator, we can't delete values of tuple object by using Indexing and slicing but we can delete entire object.

Examples:

```
>>> t1=(10,-34,0,10,23,56,76,21)
>>> print(t1,type(t1))          → (10, -34, 0, 10, 23, 56, 76, 21) <class 'tuple'> >>> del
t1[0]    → TypeError: 'tuple' object doesn't support item deletion.
>>> del t1[0:4]                → TypeError: 'tuple' object does not support item deletion.
>>> del t1  # Here we are removing complete object.
>>> print(t1,type(t1))              → NameError: name 't1' is not defined.
```

MOST IMP: sorted(): This Function is used for Sorting the data of immutable object tuple and gives. the sorted data in the form of list.

Syntax: listobj=sorted(tuple object)

Examples:

```
>>> t1=(10,23,-56,-1,13,15,6,-2)
>>> print(t1,type(t1))              →  (10, 23, -56, -1, 13, 15, 6, -2) <class 'tuple'>
>>> t1.sort()                       → AttributeError: 'tuple' object has no attribute 'sort'.
>>> x=sorted(t1)
>>> print(x, type(x))               → [-56, -2, -1, 6, 10, 13, 15, 23] <class 'list'>
>>> print(t1,type(t1))              → (10, 23, -56, -1, 13, 15, 6, -2) <class 'tuple'>
>>> t1=tuple(x) # Converted  sorted list into tuple
>>> print(t1,type(t1))                  → (-56, -2, -1, 6, 10, 13, 15, 23) <class 'tuple'>
>>> t2=t1[ : : -1]
>>> print(t2,type(t2))                  → (23, 15, 13, 10, 6, -1, -2, -56) <class 'tuple'>
                    OR
>>> t1=(10,-4,12,34,16,-6,0,15)
```

```
>>> print(t1,type(t1))          → (10, -4, 12, 34, 16, -6, 0, 15) <class 'tuple'>
>>> l1=list(t1)
>>> print(l1,type(l1))          → [10, -4, 12, 34, 16, -6, 0, 15] <class 'list'>
>>> l1.sort()
>>> print(l1,type(l1))          → [-6, -4, 0, 10, 12, 15, 16, 34] <class 'list'>
>>> t1=tuple(l1)
>>> print(t1,type(t1))          → (-6, -4, 0, 10, 12, 15, 16, 34) <class 'tuple'>
>>>t1=t1[ : : -1]
>>> print(t1,type(t1))          → (34, 16, 15, 12, 10, 0, -4, -6) <class 'tuple'>
```

Inner (OR) Nested tuple:

- The Process of Defining One tuple in another tuple is called Inner or Nested tuple.
- Syntax:- tplobj1=(Val1,Val2....(Val11,Val12....Val1n).....(Val21,Val22...Val2n).....Val-n)
- Here (Val11,Val12....Val1n) is called One Inner OR Nested tuple.
  (Val21,Val22...Val2n) is called another Inner OR Nested tuple.
- (Val1,Val2....(Val11,Val12....Val1n)....(Val21,Val22...Val2n)....Val-n) is called Outer tuple.
- All the pre-defined Functions of tuple can be applied on Inner or Nested tuple.
- On Inner or Nested tuple, we can perform Index and Slicing Operations.

Examples:

```
>>> sf=(10,"RS",(17,18,16),(78,66,79),"OUCET")
>>> print(sf, type(sf))  → (10, 'RS', (17, 18, 16), (78, 66, 79), 'OUCET') <class 'tuple'>
>>> print(sf[0])          → 10
>>> print(sf[2],type(sf[2]),type(sf)) → (17, 18, 16) <class 'tuple'> <class 'tuple'> >>>
print(sf[0:3])          → (10, 'RS', (17, 18, 16))

>>> sf=(10,"RS",[17,18,16],(78,66,79),"OUCET")
>>> print(sf, type(sf))  → (10, 'RS', [17, 18, 16], (78, 66, 79), 'OUCET') <class 'tuple'>
>>> print(sf[2],type(sf[2]),type(sf)) → [17, 18, 16] <class 'list'> <class 'tuple'>
>>> sf[2].append(12)
>>> print(sf[2],type(sf[2]),type(sf))  → [17, 18, 16, 12] <class 'list'> <class 'tuple'> >>>
print(sf,type(sf))      → (10, 'RS', [17, 18, 16, 12], (78, 66, 79), 'OUCET') <class 'tuple'> >>>
sf[3].append(12)  → AttributeError: 'tuple' object has no attribute 'append'.

>>> sf=[10,"RS",[17,18,16],(78,66,79),"OUCET"]
>>> print(sf,type(sf))  → [10, 'RS', [17, 18, 16], (78, 66, 79), 'OUCET'] <class 'list'>
>>> print(sf[2],type(sf[2]),type(sf))          →  [17, 18, 16] <class 'list'> <class 'list'> >>>
print(sf[3],type(sf[3]),type(sf))      → (78, 66, 79) <class 'tuple'> <class 'list'>
```

NOTE:

- One can define One List in another List.
- One can define One Tuple in another Tuple.
- One can define One List in another Tuple (tuple of lists) • One can define One tuple in another List (list of tuples)

```
>>> print(t1,type(t1))          → (10, 'Rossum', [16, 18, 17], ('CSE', 'AI', 'DS'), 'OUCET')
                                  <class 'tuple'>
>>> print(t1[2],type(t1[2]))    → [16, 18, 17] <class 'list'>
>>> print(t1[3],type(t1[3]))    → ('CSE', 'AI', 'DS') <class 'tuple'>
>>> t1[2].append("KVR")
>>> print(t1,type(t1))          → (10,'Rossum',[16,18,17,'KVR'],('CSE','AI','DS'),
                                  'OUCET') <class 'tuple'>
>>> t1[3].append("Python")      → AttributeError: 'tuple' object has no attribute 'append'.
>>> t1[2][1]  → 18 >>> t1[2][-
2]  → 17
>>> t1[2][-3]  → 18
>>> k=sorted(t1[-2])
>>> k                → ['AI', 'CSE', 'DS']
>>> t1[3]=k          → TypeError: 'tuple' object does not support item assignment
>>> l1=list(t1)
>>> l1           → [10, 'Rossum', [16, 18, 17, 'KVR'], ('CSE', 'AI', 'DS'), 'OUCET'] >>>
l1[3]=k
>>> l1               → [10, 'Rossum', [16, 18, 17, 'KVR'], ['AI', 'CSE', 'DS'], 'OUCET']
>>> y=tuple(l1[3])
>>> l1[3]=y
>>> l1  → [10, 'Rossum', [16, 18, 17, 'KVR'], ('AI', 'CSE', 'DS'), 'OUCET'] >>> t2=tuple(l1)
>>> t2               → (10, 'Rossum', [16, 18, 17, 'KVR'], ('AI', 'CSE', 'DS'), 'OUCET')

>>> t1=('AI', 'CSE', 'DS')
>>> k=sorted(t1)
>>> k                → ['AI', 'DS', 'CSE']
```

Set Types

Set:

Properties:

- 'set' is one of the pre-defined classes and treated as set data type.
- The purpose of set data type is that " To store Multiple Values either of Same Type or Different Type or
- Both the Types in Single Object with Unique Values (No Duplicates are allowed).
- The Elements of set must be stored within Curly Braces { } and the elements must be separated by comma.
- An object of set does not maintain Insertion Order bcoz PVM displays any possibility of set elements.

- On the object set, we can't perform Both Indexing and Slicing Operations bcoz set never maintains Insertion order.
- An object of set belongs to Both Mutable in the case of Add and immutable in the case of Item Assignment.
- With respect to set , we can create Two Types of set objects. They are.
    a) Empty Set
    b) Non-Empty Set

a) Empty Set:

• An Empty Set is One, which does not contain any Elements and whose length is 0. • Syntax: varname=set()

b) Non-Empty Set:

- An Nn-Empty Set is One, which contains Elements and whose length is>0.
- Syntax1: varname={Val1,Val2...Val-n} • Syntax2: varname=set(object) **Examples:**

```
>>> s1={10,20,30,40,10,50,60,20}
```

→

```
>>> print(s1,type(s1))                    {50, 20, 40, 10, 60, 30} <class 'set'> >>>
print(s1,type(s1))      → {50, 20, 40, 10, 60, 30} <class 'set'>
>>> s2={10,"Travis",44.44,True,"Numpy"}
>>> print(s2,type(s2))   → {'Numpy', True, 'Travis', 10, 44.44} <class 'set'> >>>
s2={10,"Travis",44.44,True,"Numpy"}
>>> print(s2,type(s2))          → {'Numpy', True, 'Travis', 10, 44.44} <class 'set'> >>>
s2[0]    → TypeError: 'set' object is not subscriptable. >>> s2[0:3]   → TypeError:
'set' object is not subscriptable.

>>> s2={10,"Travis",44.44,True,"Numpy"}
>>> print(s2,type(s2))              → {'Numpy', True, 'Travis', 10, 44.44} <class 'set'>
>>> s2[0]=100   → TypeError: 'set' object does not support item assignment→IMMUTABLE

>>> s2={10,"Travis",44.44,True,"Numpy"}
>>> print(s2,type(s2),id(s2))  → {'Numpy', True, 'Travis', 10, 44.44} <class 'set'>
                                          2485370854016
>>> s2.add("HYD")  #  MUTABLE
 >>> print(s2,type(s2),id(s2))        → {'Numpy', True, 'Travis', 10, 44.44, 'HYD'} <class 'set'>
                                          2485370854016
>>> x={}
>>> print(x, type(x))                    → {} <class 'dict'>
>>> s=set()
>>> print(s, type(s),len(s))             →  set() <class 'set'>   0

>>> l1=[10,20,30,10]
>>> print(l1,type(l1))                   → [10, 20, 30, 10] <class 'list'>
>>> s1=set(l1)
>>> print(s1,type(s1))                   → {10, 20, 30} <class 'set'>

>>> s="MISSISSIPPI"
>>> s1=set(s)
>>> print(s1)                            → {'M', 'S', 'P', 'I'}
```

Pre-Functions in set:

- On the object of set, we can perform different type of Operations by using Functions present in setobject. They are.

1) clear()

This Function is used for Removing all the values of set object.

Syntax: setobj.clear() **Examples:**

```
>>> s1={10,20,30,40,50,10}
```

→

```
>>> print(s1,type(s1),id(s1))  → {50, 20, 40, 10, 30} <class 'set'> 2485370854240 >>>
s1.clear()
>>> print(s1,type(s1),id(s1))          set() <class 'set'> 2485370854240

>>> print(s1,len(s1))                  → set() 0
>>> s1.clear()                         → No Output
>>> print(s1.clear())                  → None
```

2) add():

This Function is used for adding the values to set object.

Syntax: setobj.add(Value) **Examples:**

```
>>> s1={10,"Rossum",34.56,"HYD"}
>>> print(s1,type(s1),id(s1)) → {10, 'Rossum', 34.56, 'HYD'} <class 'set'>
2485370853568 >>> s1.add("NL")
>>> print(s1,type(s1),id(s1))→{34.56,'Rossum','NL',10,'HYD'}<class 'set'> 2485370853568
>>> s1.add(True)
>>> s1.add(2+3j)
>>> print(s1,type(s1),id(s1))→ {True, 34.56, 'Rossum', 'NL', 10, (2+3j), 'HYD'} <class 'set'>
                                2485370853568
>>> s1=set()
>>> print(s1,type(s1),id(s1))          →  set() <class 'set'> 2485370854240
>>> s1.add(100)
>>> s1.add("Naresh") >>>
s1.add(12345)
>>> s1.add("Python")
>>> print(s1,type(s1),id(s1)) →{'Naresh',12345,'Python',100}<class 'set'> 2485370854240
```

3) remove():

- This Function is used for removing the values of setobject.

Syntax: setobj.remove(Value)

- If the specified Values does not exist in set object, then we get KeyError.

Examples:

```
>>> s1={10,"Rossum",34.56,"HYD"}
>>> print(s1,id(s1))          → {10, 'Rossum', 34.56, 'HYD'} 2485370853568
>>> s1.remove(10)
>>> print(s1,id(s1))  >>>     → {'Rossum', 34.56, 'HYD'} 2485370853568
s1.remove("HYD")
>>> print(s1,id(s1))          → {'Rossum', 34.56} 2485370853568
>>> s1.remove("BANG")         → KeyError: 'BANG'
>>> set().remove(100)         → KeyError: 100
```

4) discard():

- This Function is used for Removing the Value from setobject.

→

Syntax: setobj.discard(Value)

- If the Specified Value does not exist, then we never get any Error.

Examples:

```
>>> s1={10,"Ankit",34.56,"Python","HYD"}
>>> print(s1,type(s1),id(s1))          {'Ankit', 34.56, 'HYD', 10, 'Python'} <class 'set'>
                                       2705097791904
>>> s1.discard(10)
>>> print(s1,type(s1),id(s1))          →{'Ankit', 34.56, 'HYD', 'Python'} <class 'set'>
                                       2705097791904
>>> s1.discard("HYD")
>>> print(s1,type(s1),id(s1)) → {'Ankit', 34.56, 'Python'} <class 'set'>
2705097791904 >>> s1.discard("Python")
>>> print(s1,type(s1),id(s1)) →{'Ankit', 34.56} <class 'set'> 2705097791904 >>>
s1.discard(1000)  # we won't get KeyError.
>>> print(s1,type(s1),id(s1)) →{'Ankit', 34.56} <class 'set'> 2705097791904 >>>
s1.remove(1000)KeyError: 1000
```

5) pop():

- This Function is used for Removing Any Arbitrary Element from non-empty set object.

Syntax: setobj.pop()

- When we call pop() upon empty set object then we get KeyError

Example 1: set elements are given and order of display not shown and pop() removes Arbitrary Element always.

```
>>> s1={10,"Ankit",34.56,"Python","HYD"}
>>> s1.pop()          → 'Ankit'
>>> s1.pop()          → 34.56
>>> s1.pop()          → 'HYD'
>>> s1.pop()          → 10
>>> s1.pop()          → 'Python'
>>> print(s1)         → set()
>>> s1.pop()          → KeyError: 'pop from an empty set'
>>> set().pop()       → KeyError: 'pop from an empty set'
```

Example 2: set elements are given and order of display shown and pop() removes First Element always.

```
>>> s1={100,200,300,150,450,-120}
>>> print(s1,id(s1)) →{450, 100, -120, 150, 200, 300} 2705097791232
>>> s1.pop()          → 450
>>> s1.pop()          → 100
>>> s1.pop()          → 120
>>> s1.pop()          → 150
>>> s1.pop()          → 200
>>> s1.pop()          → 300
```

→

>>> print(s1,id(s1)) →set() 2705097791232

>>> s1.pop() → KeyError: 'pop from an empty set' **6)**

isdisjoint():

- This Function Returns True Provided There is No Common Element in setobj1 and setobj2.

Syntax: setobj1.isdisjoint(setobj2)

- This Function Returns False Provided There is at least one Common Element in setobj1 and setobj2.

Examples:

```
>>> s1={10,20,30,40}
>>> s2={15,25,35}
>>> s3={10,100,200,300}
>>> s1.isdisjoint(s2)  → True
>>> s1.isdisjoint(s3)  → False
>>> s2.isdisjoint(s3)  → True
```

7)

issubset():

Syntax: setobj1.issubset(setobj2)

- This function returns True Provided All the elements of setobj1 present in setobj2 (OR) setobj2 contains all the elements of setobj1.
- This function returns False Provided at least one element of setobj1 not present in setobj2 (OR) not containing at least one element of setobj1. **Examples:**

```
>>> s1={0,1,2,3,4,5,6,7,8,9}
>>> s2={3,4,5}
>>> s3={10,20,30}
>>> s2.issubset(s1)          → True
>>> s3.issubset(s1)          → False
>>> s1.issubset(s3)          → False
>>> s4={1,15,25}
>>> s4.issubset(s1)          → False
>>> s4={1,2,3,10}
>>> s4.issubset(s1)          → False
```

8) issuperset():

Syntax: setobj1.issuperset(setobj2)

- This function returns True Provided setobj1 contains all the elements of setobj2 (OR) all elements of setobj2 present in setobj1.
- This function returns False Provided at least one element of setobj2 not present in setobj1 (OR) setobj1 not containing at least one element of setobj2. **Examples:**

```
>>> s1={0,1,2,3,4,5,6,7,8,9}
>>> s2={3,4,5}
>>> s3={10,20,30}
>>> s1.issuperset(s2)          → True
>>> s1.issuperset(s3)          → False
>>> s4={1,2,3,10}
>>> s1.issuperset(s4)          → False
```

9) union():

Syntax: setobj3=setobj1.union(setobj2)

(OR)

Syntax: setobj3=setobj2.union(setobj1)

- This Function is used for Taking all Unique Elements of setobj1 and setobj2 and place them in setobj3.

Examples:

```
>>> cp={"Sachin", "Rohit", "Kohli"}
>>> tp={"Kohli", "Rossum", "Travis"}
>>> print(cp, type(cp))          → {'Sachin', 'Kohli', 'Rohit'} <class 'set'>
>>> print(tp, type(tp))          → {'Rossum', 'Travis', 'Kohli'} <class 'set'>
>>> allcptp=cp.union(tp)
>>> print(allcptp, type(allcptp))  →{'Sachin', 'Kohli', 'Rohit', 'Rossum', 'Travis'} <class 'set'>
                (OR)
>>> allcptp=tp.union(cp)
>>> print(allcptp, type(allcptp))  →{'Sachin', 'Kohli', 'Rohit', 'Rossum', 'Travis'} <class 'set'>
```

10) intersection():

Syntax: setobj3=setobj1.intersection(setobj2)

(OR)

Syntax: setobj3=setobj2.intersection(setobj1)

- This Function is used for Taking Common Elements of setobj1 and setobj2 and place them in setobj3.

Examples:

```
>>> cp={"Sachin", "Rohit", "Kohli"}
>>> tp={"Kohli", "Rossum", "Travis"}
>>> print(cp, type(cp))          → {'Sachin', 'Kohli', 'Rohit'} <class 'set'>
>>> print(tp, type(tp))          → {'Rossum', 'Travis', 'Kohli'} <class 'set'>
>>> bothcptp=cp.intersection(tp)
>>> print(bothcptp, type(bothcptp)) →{'Kohli'} <class 'set'>
                OR
>>> bothcptp=tp.intersection(cp)
>>> print(bothcptp, type(bothcptp))→{'Kohli'} <class 'set'>

>>> a={10,20}
>>> b={30,40}
>>> a.intersection(b)            → set()
>>> {10,20,30}.intersection({'a', 'e', 'i', 'o', 'u'})          → set()
>>> {10,20,30}.intersection({'a', 'e', 'i', 'o', 'u', 10})      → {10}
```

11) difference():

Syntax1: setobj3=setobj1.difference(setobj2)

- This Function Removes the common Elements from setobj1 and setobj2 and It takes Remaining Elements from setobj1 and placed in setobj3.

Examples:

```
>>> cp={"Sachin", "Rohit", "Kohli"}
>>> tp={"Kohli", "Rossum", "Travis"}
>>> print(cp,type(cp))          → {'Sachin', 'Kohli', 'Rohit'} <class 'set'>
>>> print(tp, type(tp))        → {'Rossum', 'Travis', 'Kohli'} <class 'set'> >>>
onlycp=cp.difference(tp)
>>> print(onlycp, type(onlycp)) →{'Sachin', 'Rohit'} <class 'set'>
```

Syntax2: setobj3=setobj2.difference(setobj1)

- This Function Removes the common Elements from setobj2 and setobj1 and It takes Remaining Elements from setobj2 and placed in setobj3.

Examples:

```
>>> cp={"Sachin", "Rohit", "Kohli"}
>>> tp={"Kohli", "Rossum", "Travis"}
>>> print(cp,type(cp))          → {'Sachin', 'Kohli', 'Rohit'} <class 'set'>
>>> print(tp, type(tp))        → {'Rossum', 'Travis', 'Kohli'} <class 'set'> >>>
onlytp=tp.difference(cp)
>>> print(onlytp, type(onlytp))  → {'Rossum', 'Travis'} <class 'set'>
>>> a={10,20}
>>> b={10,30,40,20}
>>> a.difference(b)   → set() >>>
b.difference(a)   → {40, 30} 12)
```

symmetric_difference():

Syntax: setobj3=setobj1.symmetric_difference(setobj2)

(OR)

Syntax: setobj3=setobj2.symmetric_difference(setobj1)

- This Function Removes the common Elements from setobj1 and setobj2 and It takes Remaining Elements from both setobj1 and setobj2 and place them in setobj3.

Examples:

```
>>> cp={"Sachin", "Rohit", "Kohli"}
>>> tp={"Kohli", "Rossum", "Travis"}
>>> print(cp,type(cp))          →  {'Sachin', 'Kohli', 'Rohit'} <class 'set'>
>>> print(tp, type(tp))          → {'Rossum', 'Travis', 'Kohli'} <class 'set'>
>>> exclcptp=cp.symmetric_difference(tp)
>>> print(exclcptp, type(exclcptp)) →{'Rohit', 'Travis', 'Sachin', 'Rossum'} <class 'set'> >>>
exclcptp=tp.symmetric_difference(cp)
>>> print(exclcptp, type(exclcptp)) →{'Rohit', 'Travis', 'Sachin', 'Rossum'} <class 'set'>
```

OR

```
>>> exclcptp=tp.symmetric_difference(cp)
```

```
>>> print(exclcptp)                    → {'Rohit', 'Travis', 'Sachin', 'Rossum'}
                    (OR)
 >>> exclcptp=cp.union(tp).difference(cp.intersection(tp))
>>> print(exclcptp)                    → {'Rossum', 'Sachin', 'Travis', 'Rohit'}
```

Special Points:

```
 >>> s1={10,20,30,40}
 >>> s2={10,20,50,60}
 >>> s3=s1.difference(s2)
 >>> print(s3)  → {40, 30} >>>
 s4=s2.difference(s1)
 >>> print(s4)  → {50, 60}
 >>> s5=s1.symmetric_difference(s2)
 >>> print(s5)  → {40, 50, 60, 30}
                          (OR)
 >>> s6=s1.union(s2).difference(s1.intersection(s2))
 >>> print(s6)  → {40, 50, 60, 30}
                    (OR)
 >>> s7=s1.difference(s2).union(s2.difference(s1))
 >>> print(s7)  → {40, 50, 60, 30}
```

MOST IMP→ Perform the above Set Operations without set Functions by Using Bitwise Operators

```
 >>> cp={"Sachin", "Rohit", "Kohli"}
 >>> tp={"Kohli", "Rossum", "Travis"}
>>> print(cp, type(cp))                → {'Sachin', 'Kohli', 'Rohit'} <class 'set'>
>>> print(tp, type(tp))                → {'Rossum', 'Travis', 'Kohli'} <class 'set'>
 >>> allcptp=cp|tp  # Bitwise OR Operator OR union()
>>> print(allcptp)                     → {'Sachin', 'Kohli', 'Rohit', 'Rossum', 'Travis'}
 >>> botcptp=cp&tp # Bitwise AND Operator  OR intersection()
>>> print(bothcptp)                    → {'Kohli'}
 >>> onlycp=cp-tp  # - Operator  OR difference()
 >>> print(onlycp, type(onlycp))   → {'Sachin', 'Rohit'} <class 'set'> >>>
 onlytp=tp-cp
>>> print(onlytp, type(onlytp))              → {'Rossum', 'Travis'} <class 'set'>
 >>> exclcptp=cp^tp # Bitwise XOR Operator  OR symmetric_difference()
 >>> print(exclcptp, type(exclcptp))        → {'Rohit', 'Travis', 'Sachin', 'Rossum'} <class 'set'>
```

13) update():

Syntax: setobj1.update(setobj2)

- This Function is used Merging setobj2 values with setobj1. **Examples:**

```
>>> s1={10,20,30,40}
>>> s2={"Python", "Java"}
>>> print(s1,type(s1))          → {40, 10, 20, 30} <class 'set'>
>>> print(s2,type(s2))          → {'Python', 'Java'} <class 'set'>
>>> s1.update(s2)
>>> print(s1)                   → {20, 40, 10, 'Python', 'Java', 30}

>>> s1={10,20,30,40}
>>> s2={"Python","Java",10}
>>> s1.update(s2)
>>> print(s1)                   → {20, 40, 10, 'Python', 'Java', 30}
```

Nested or Inner Properties of Set:

- We can define.

a) Defining set inside of another set NOT POSSIBLE
b) Defining set inside of another Tuple POSSIBLE
c) Defining set inside of another List POSSIBLE
d) Defining tuple inside of Another Set POSSIBLE
e) Defining list inside of another set NOT POSSIBLE **Examples:**

```
>>> s1={10,"Rossum",{10,15,16},"OUCET"} →TypeError: unhashable type: 'set'
>>> s1={10,"Rossum",[10,15,16],"OUCET"} →TypeError: unhashable type: 'list'
>>> s1={10,"Rossum",(10,15,16),"OUCET"}
>>> print(s1,type(s1))          →{'Rossum', 10, 'OUCET', (10, 15, 16)} <class 'set'> >>>
t1=(10,"Rossum",{10,15,16},"OUCET")
>>> print(t1[2],type(t1[2]),type(t1))          →{16, 10, 15} <class 'set'> <class 'tuple'>
>>> l1=[10,"Rossum",{10,15,16},"OUCET"]
>>> print(l1[2],type(l1[2]),type(l1))          →{16, 10, 15} <class 'set'> <class 'list'>
```

Frozenset:

Properties:

- 'frozenset' is one of the pre-defined classes and treated as set data type.
- The purpose of frozenset data type is that " To store Multiple Values either of Same Type or Different Type or Both the Types in Single Object with Unique Values (No Duplicates are allowed).
- The Elements of frozenset set must be obtained from list, tuple and set by using a Type casting Function fronset().
- An object of frozenset does not maintain Insertion Order because PVM displays any possibility of frozenset elements.
- On the object frozenset, we can't perform Both Indexing and Slicing Operations because frozenset never maintains Insertion order.

- An object of frozenset belongs to immutable in the case of Item Assignment. and no functions are allowed to modify/ update the values of frozenset object.
- With respect to frozenset , we can create Two Types of frozenset objects. They are.
    a) Empty frozenset
    b) Non-Empty frozenset

a) Empty frozenset:

• An Empty frozenset is One, which does not contain any Elements and whose length is 0.

Syntax: varname=frozenset()

b) Non-Empty frozenSet:

• An Nn-Empty Set is One, which contains Elements and whose length is > 0.

Syntax1: varname=frozenset([val_1,val_2...val_n])

(OR)

Syntax2: varname=frozenset((val_1,val_2...val_n))

(OR)

Syntax3: varname=frozenset({val_1,val_2...val_n})

NOTE:- The functionality of frozenset is exactly similar to set type but an object of set belongs to Both Mutable(in the case add() and other Functions) and Immutable (in the case of Item Assignment), where as an object of frozenset belongs to Immutable because Item Assignment does not support and No Functions are allowed which are modifying the values of frozenset.

Examples:

```
>>> fs=frozenset()
>>> print(fs, type(fs),len(fs))          → frozenset() <class 'frozenset'> 0
>>> fs=frozenset([10,20,10,10,10,10])
>>> print(fs, type(fs),len(fs))  → frozenset({10, 20}) <class 'frozenset'> 2 >>>
fs=frozenset((10,20,10,10,10,10))
>>> print(fs, type(fs),len(fs))  → frozenset({10, 20}) <class 'frozenset'> 2 >>>
fs=frozenset({10,20,10,10,10,10})
>>> print(fs, type(fs),len(fs))          → frozenset({10, 20}) <class 'frozenset'> 2
>>> fs=frozenset([10,"Travis",45.67,True])
>>> print(fs, type(fs)) → frozenset({True, 10, 45.67, 'Travis'}) <class 'frozenset'> >>> fs[0]
                  → TypeError: 'frozenset' object is not subscriptable.
>>> fs[0:2]              → TypeError: 'frozenset' object is not subscriptable. >>>
fs[0]=100 → TypeError: 'frozenset' object does not support item assignment >>>
fs.add(100)          → AttributeError: 'frozenset' object has no attribute 'add'.
```

Pre-Defined Functions in frozenset:

- frozenset contains the following Functions copy(), isdisjoint(), issuperset(), issubset(), union(), intersection(), difference(), symmertic_difference() **NOTE:**

```
>>> fs1=frozenset({10,20,30,409})
>>> print(fs1,type(fs1),id(fs1)) → frozenset({409, 10, 20, 30}) <class 'frozenset'>
                                        2068835340960
>>> fs2=fs1.copy()
>>> print(fs2,type(fs2),id(fs2)) →frozenset({409, 10, 20, 30}) <class 'frozenset'>
                                        2068835340960
```

- In General, Immutable Object content is Not Possible to copy(in the case of tuple). Whereas in the case of frozenset, we are able to copy its content to another frozenset object. Here Original frozenset object and copied frozenset object contains Same Address and Not at all possible to Modify / Change their content.

Examples:

```
>>> s1={10,"RS",3.4,"PYTHON"}
>>> fs1=frozenset(s1)
>>> print(fs1,type(fs1),id(fs1→ frozenset({10, 3.4, 'RS', 'PYTHON'}) <class 'frozenset'>
                                        1774317640096
>>> fs2=fs1.copy()
>>> print(fs2,type(fs2),id(fs2))→frozenset({10, 3.4, 'RS', 'PYTHON'}) <class 'frozenset'>
                                        1774317640096
```

- **frozenset** does not contain the following Functions clear(), add(), remove(), discard(), pop(), update()

Dict Category Data Types (Collection Data Types or Data Structures):

Properties:

- 'dict' is one of the pre-defined classes and treated as Dict Data Type.
- The purpose of dict data type is that "To store (Key, value) in single variable".
- In (Key, Value), the values of Key are Unique and Values of Value may or may not be unique.
- The (Key, value) must be organized or stored in the object of dict within Curly Braces {} and they separated by comma.
- An object of dict does not support Indexing and Slicing because Values of Key itself considered as Indices.
- In the object of dict, Values of Key are treated as Immutable and Values of Value are treated as mutable.
- Originally an object of dict is mutable because we can add (Key, Value) externally.
- We have two types of dict objects. They are.

a) Empty dict
b) Non-empty dict

a) Empty dict:

- Empty dict is one, which does not contain any (Key, Value) and whose length is 0.

Syntax:- dictobj1= { }

or

dictobj=dict()

Syntax for adding (Key, Value) to empty dict:

```
dictobj[Key1]=Val1
dictobj[Key2]=Val2
……
dictobj[Key-n]=Val-n
```

- Here Key1,Key2...Key-n are called Values of Key and They must be Unique.
- Here Val1, Val2...Val-n are called Values of Value and They may or may not be unique.

b) Non-Empty dict:

- Non-Empty dict is one, which contains (Key, Value) and whose length is >0.

Syntax:- dictobj1= { Key1:Val1,Key2:Val2.....Keyn:Valn}

- Here Key1,Key2...Key-n are called Values of Key and They must be Unique.
- Here Val1, Val2...Val-n are called Values of Value and They may or may not be unique.

Examples:

```
>>> d1={10:"Python",20:"Data Sci",30:"Django"}
>>> print(d1,type(d1))          → {10: 'Python', 20: 'Data Sci', 30: 'Django'} <class 'dict'>
>>> d2={10:3.4,20:4.5,30:5.6,40:3.4}
>>> print(d2,type(d2))          → {10: 3.4, 20: 4.5, 30: 5.6, 40: 3.4} <class 'dict'>
>>> len(d1)                     → 3
>>> len(d2)                     → 4
>>> d3={}
>>> print(d3,type(d3))          → {} <class 'dict'>
>>> len(d3)                     → 0
>>> d4=dict()
>>> print(d4,type(d4))          → {} <class 'dict'>
>>> len(d4)                     → 0
>>> d2={10:3.4,20:4.5,30:5.6,40:3.4}
>>> print(d2)                   → {10: 3.4, 20: 4.5, 30: 5.6, 40: 3.4}
>>> d2[0]                       → KeyError: 0
>>> d2[10]                      → 3.4
>>> d2[10]=10.44
>>> print(d2)                   → {10: 10.44, 20: 4.5, 30: 5.6, 40: 3.4}
>>> d2={10:3.4,20:4.5,30:5.6,40:3.4}
```

```
>>> print(d2,type(d2),id(d2))        → {10: 3.4, 20: 4.5, 30: 5.6, 40: 3.4} <class 'dict'>
2090750380736
>>> d2[50]=5.5
>>> print(d2,type(d2),id(d2))        →{10: 3.4, 20: 4.5, 30: 5.6, 40: 3.4, 50: 5.5} <class 'dict'>

        2090750380736
>>> d3={}
>>> print(d3,type(d3),id(d3))        →{} <class 'dict'> 2090750332992
>>> d3["Python"]=1
>>> d3["Java"]=3
>>> d3["C"]=2
>>> d3["GO"]=1
>>> print(d3,type(d3),id(d3))        →{'Python': 1, 'Java': 3, 'C': 2, 'GO': 1} <class 'dict'>
                                       2090750332992
>>> d4=dict()
>>> print(d4,type(d4),id(d4))        →{} <class 'dict'> 2090754532032
>>> d4[10]="Apple"
>>> d4[20]="Mango" >>>
d4[30]="Kiwi"
>>> d4[40]="Sberry"
>>> d4[50]="Orange"
>>> print(d4,type(d4),id(d4))        →{10: 'Apple', 20: 'Mango', 30: 'Kiwi', 40: 'Sberry', 50:
                                       'Orange'} <class 'dict'> 2090754532032
>>> d4[10]="Guava"
>>> print(d4,type(d4),id(d4))        →{10: 'Guava', 20: 'Mango', 30: 'Kiwi', 40: 'Sberry', 50:
                                       'Orange'} <class 'dict'> 2090754532032
>>> d2={10:3.4,20:4.5,30:5.6,40:3.4}
>>> print(d2,type(d2),id(d2)) →{10:3.4,20:4.5,30:5.6,40:3.4}<class 'dict'>
2090754531520 >>> d2[50]=1.2
>>> print(d2,type(d2),id(d2)) →{10: 3.4, 20: 4.5, 30: 5.6, 40: 3.4, 50: 1.2} <class 'dict'>
                                2090754531520
```

Pre-Defined functions in dict:

• In dict object, we have the following function for performing Various Operations.

1. clear():

Syntax: dictobj.clear()

• This Function is used for Removing all the elements of non-empty dict object. • If we call this function upon empty dict object, then we get None / No Output.

Examples:

```
>>> d1={10:"Python",20:"Java",30:"C++",40:"C"}
>>> print(d1,type(d1),id(d1)) →{10: 'Python', 20: 'Java', 30: 'C++', 40: 'C'} <class 'dict'>
```

```
>>> d1.clear()
>>> print(d1,type(d1),id(d1))  →{} <class 'dict'> 2364664861568
>>> print(d1.clear())          →None >>>
print(dict().clear())          →None >>>
print({}.clear())          →None
```

2. pop():

Syntax: dictobj.pop(Key)

- This Function is used for Removing (Key, value) from non-empty dictobject by passing Value of Key.
- If the Value of Key does not exist, then we get KeyError.
- If we call this pop(), upon empty dict object then we get KeyError.

Examples:

```
>>> d1={10:"Python",20:"Java",30:"C++",40:"C"}
>>> print(d1,id(d1))  → {10: 'Python', 20: 'Java', 30: 'C++', 40: 'C'} 2364669155776
>>> d1.pop(20)              → 'Java'
>>> print(d1,id(d1))        →  {10: 'Python', 30: 'C++', 40: 'C'} 2364669155776
>>> d1.pop(30)              → 'C++'
>>> d1.pop(40)              → 'C'
>>> print(d1,id(d1))        → {10: 'Python'} 2364669155776
>>> d1.pop(50)              → KeyError: 50
>>> print(d1,id(d1))        → {10: 'Python'} 2364669155776
>>> d1.pop(10)              → 'Python'
>>> print(d1,id(d1))        → {} 2364669155776
>>> d1.pop(10)              → KeyError: 10
>>> dict().pop(10)          → KeyError: 10
>>> {}.pop("Python")        → KeyError: 'Python'
```

3. popitem():

Syntax: dictobj.popitem()

- This Function is used for Removing Last (Key, value) entry from non-empty dictobject.
- If we call this popitem(), upon empty dict object then we get KeyError.

Examples:

```
>>> d1={10:"Python",20:"Java",30:"C++",40:"C"}
>>> print(d1,id(d1))              → {10: 'Python', 20: 'Java', 30: 'C++', 40: 'C'}
 2364669156096
>>> d1.popitem()                  → (40, 'C')
>>> print(d1,id(d1))              → {10: 'Python', 20: 'Java', 30: 'C++'} 2364669156096
>>> d1.popitem()                  → (30, 'C++')
>>> print(d1,id(d1))              → {10: 'Python', 20: 'Java'} 2364669156096
>>> d1.popitem()                  → (20, 'Java')
```

>>> print(d1,id(d1)) → {10: 'Python'} 2364669156096
>>> d1.popitem() → (10, 'Python')
>>> print(d1,id(d1)) → {} 2364669156096
>>> d1.popitem() → KeyError: 'popitem(): dictionary is empty'. >>>
dict().popitem() → KeyError: 'popitem(): dictionary is empty'.
>>> {}.popitem() → KeyError: 'popitem(): dictionary is empty'.

4. copy():

Syntax: dictobj2=dictobj1.copy()

This function is used for Copying the content of One dict object into another dict object.

Examples:

>>> d1={10:"Python",20:"Java",30:"C++",40:"C"}
>>> print(d1,id(d1)) → {10: 'Python', 20: 'Java', 30: 'C++', 40: 'C'} 2364669156416 >>>
d2=d1.copy() # Shallow Copy
>>> print(d2,id(d2)) → {10: 'Python', 20: 'Java', 30: 'C++', 40: 'C'} 2364669156096
>>> d1[50]="Dsc"
>>> d2[60]="Django"
>>> print(d1,id(d1)) →{10:'Python', 20: 'Java', 30: 'C++', 40: 'C', 50: 'Dsc'}
2364669156416 >>> print(d2,id(d2)) → {10:'Python',20:'Java',30:'C++',40:'C',60:'Django'}
2364669156096 Deep Copy Process of dict objects:
>>> d1={10:"Python",20:"Java",30:"C++",40:"C"}
>>> print(d1,id(d1)) →{10: 'Python', 20: 'Java', 30: 'C++', 40: 'C'}
2364669155776 >>> d2=d1 # Deep Coy
>>> print(d2,id(d2)) → {10: 'Python', 20: 'Java', 30: 'C++', 40: 'C'} 2364669155776 >>>
d1[50]="Dsc"
>>> print(d1,id(d1)) → {10: 'Python', 20: 'Java',30: 'C++', 40: 'C', 50: 'Dsc'}
2364669155776 >>> print(d2,id(d2)) →{10: 'Python', 20: 'Java', 30: 'C++', 40: 'C', 50: 'Dsc'}
2364669155776

5. get():

- This Function is used for obtaining Value of Value by Passing Value of Key.
- If the value of Key does not exist, then we get None as a result.

(OR)

Syntax: dictobj[Key] →Will give Value of Value if Key Present otherwise get KeyError

Examples:

>>> d1={10:"Python",20:"Java",30:"C++",40:"C"}
>>> print(d1,id(d1)) → {10: 'Python', 20: 'Java', 30: 'C++', 40: 'C'} 2364669156096
>>> v=d1.get(10)
>>> print(v) → Python
>>> v=d1.get(20)
>>> print(v) → Java
#OR

```
>>> d1.get(10)                → 'Python'
>>> d1.get(20)                → 'Java'
>>> print(d1.get(200))        → None
>>> v=d1.get(200)
>>> print(v)                  → None
>>> print(dict().get(10))     →  None
>>> print({}.get(10))         →None.
                (OR)
>>> d1={10:"Python",20:"Java",30:"C++",40:"C"}
>>> print(d1,id(d1))          → {10: 'Python', 20: 'Java', 30: 'C++', 40: 'C'} 2364669156288
>>> d1.get(10)                → 'Python'

>>> d1[10]                    → 'Python'
>>> d1[20]                    → 'Java'
>>> d1[40]                    → 'C'
>>> d1[400]                   → KeyError: 400
>>> print(d1.get(400))        →  None
```

6. keys():

Syntax: varname=dictobj.keys()

(OR)

dictobj.keys()

• This Function is used for obtaining set of Keys in the form of dict_keys object.

Examples:

```
>>> d1={10:"Python",20:"Java",30:"C++",40:"C"}
>>> print(d1,id(d1))          → {10: 'Python', 20: 'Java', 30: 'C++', 40: 'C'} 2364669156096
>>> ks=d1.keys()
>>> print(ks, type(ks))  >>> → dict_keys([10, 20, 30, 40]) <class 'dict_keys'>
for k in ks:
...         print(k)
...
            10
            20
            30
            40
>>> for k in d1.keys():
...         print(k)
...
                10
                20
```

30
40

```
>>> for k in d1.keys(): ...
print(k,"-->",d1.get(k))
...
        10 --> Python
        20 --> Java
        30 --> C++
        40 --> C
>>> for k in d1.keys(): ...
print(k,"-->",d1[k])
...
        10 --> Python
        20 --> Java
        30 --> C++
        40 --> C
```

7. values():

Syntax: varname=dictobj.values()

(OR)

dictobj.values()

This Function is used for obtaining set of Values in the form of dict_values object.

Examples:

```
>>> d1={10:"Python",20:"Java",30:"C++",40:"C"}
>>> print(d1,id(d1)) → {10: 'Python', 20: 'Java', 30: 'C++', 40: 'C'} 2364669156288 >>>
vs=d1.values()
>>> print(vs)          → dict_values(['Python', 'Java', 'C++', 'C'])
```

```
>>> for v in vs:
...    print(v)
...
        Python
        Java
        C++
        C
>>> for v in d1.values():
...    print(v)
...
```

```
                Python
                Java
                C++
                C
```

MOST IMP:

```
>>> for x in d1:
...             print(x)
...
                        10
                        20
                        30
                        40

>>> for x in d1:
...             print(x,"-->",d1[x])
...
                        10 --> Python
                        20 --> Java
                        30 --> C++
                        40 --> C
```

8. items():

Syntax: varname=d1.items()

- This Function is used for obtaining (Key, value) in the form of tuples of list in the object of dict_items.

Examples:

```
>>> d1={10:"Python",20:"Java",30:"C++",40:"C"}
>>> kvs=d1.items()
>>> print(kvs)          → dict_items([(10, 'Python'), (20, 'Java'), (30, 'C++'), (40, 'C')])
```

```
>>> for kv in kvs:
...     print(kv)
...
                (10, 'Python')
                (20, 'Java')
                (30, 'C++')
                (40, 'C')
>>> for kv in d1.items():
```

```
...     print(kv)
...
                (10, 'Python')
                (20, 'Java')
                (30, 'C++')
                (40, 'C')
>>> for k, v in d1.items():
...     print(k,"--->",v)
...
                10 ---> Python
                20 ---> Java
                30 ---> C++
                40 ---> C
```

9. update():

Syntax: dictobj1.update(dictobj2)

- This Function is use for Joining/Updating/Modifying the dictobj1 content with dictObj2 content.

Examples:

```
>>> d1={10:1.2,20:3.4,30:4.5}
>>> d2={100:"Apple",200:"Kiwi"}
>>> print(d1,type(d1))  → {10: 1.2, 20: 3.4, 30: 4.5} <class 'dict'>
>>> print(d2,type(d2))  → {100: 'Apple', 200: 'Kiwi'} <class 'dict'> >>>
d1.update(d2)
>>> print(d1,type(d1)) → {10: 1.2, 20: 3.4, 30: 4.5, 100:'Apple',200:'Kiwi'}<class 'dict'>
>>> print(d2,type(d2))  → {100: 'Apple', 200: 'Kiwi'} <class 'dict'>
>>> d1={10:1.2,20:3.4,30:4.5}
>>> d2={100:"Apple",20:13.4}
>>> print(d1,type(d1))  →  {10: 1.2, 20: 3.4, 30: 4.5} <class 'dict'>
>>> print(d2,type(d2)) →{100: 'Apple', 20: 13.4} <class 'dict'>
>>> d1.update(d2)
>>> print(d1,type(d1))        →{10: 1.2, 20: 13.4, 30: 4.5, 100: 'Apple'} <class 'dict'>
>>> print(d2,type(d2))        →{100: 'Apple', 20: 13.4} <class 'dict'>

>>> d1={10:1.2,20:3.4,30:4.5}
>>> d2={10:11.2,20:13.4,30:14.5}
>>> print(d1,type(d1))              → {10: 1.2, 20: 3.4, 30: 4.5} <class 'dict'>
>>> print(d2,type(d2))       → {10: 11.2, 20: 13.4, 30: 14.5} <class 'dict'> >>>
d1.update(d2)
>>> print(d1,type(d1))              → {10: 11.2, 20: 13.4, 30: 14.5} <class 'dict'>
```

>>> print(d2,type(d2)) → {10: 11.2, 20: 13.4, 30: 14.5} <class 'dict'> **Most Imp Points:**

Dict in Dict →Possible to write.

>>> d1={"sno": 10, "sname" "Rossum", imarks": "cm":17,"cppm":16,"pym":18}, emarks": "cm": 7, "cppm": 78, "pym": 70}, "cname": "OUCET"}

>>> print(d1,type(d1)) → {"sno": 10, "sname" "Rossum", imarks": "cm": 17, "cppm" : 16, "pym": 18}, emarks": "cm": 7, "cppm": 78, "pym": 70}, "cname": "OUCET"} >>> for k in d1.keys():

... print(k)

...

sno

sname

imarks

emarks

cname

>>> for v in d1.values():

... print(v)

...

10

Rossum

{'cm': 17, 'cppm': 16, 'pym': 18}

{'cm': 67, 'cppm': 78, 'pym': 70}

OUCET

>>> for v in d1.values():

... print(v, type(v))

...

10 <class 'int'>

Rossum <class 'str'>

{'cm': 17, 'cppm': 16, 'pym': 18} <class 'dict'>

{'cm': 67, 'cppm': 78, 'pym': 70} <class 'dict'>

OUCET <class 'str'>

>>> for kv in d1.items():

... print(kv)

...

('sno', 10)

('sname', 'Rossum')

('imarks', {'cm': 17, 'cppm': 16, 'pym': 18})

('emarks', {'cm': 67, 'cppm': 78, 'pym': 70})

('cname', 'OUCET')

>>> for k in d1.keys():

... print(k)

```
...
                    sno
            sname
            imarks
            emarks
            cname
>>> for k in d1.keys(): ...
print(k,"--->",d1.get(k))
...
            sno ---> 10
        sname ---> Rossum
            imarks ---> {'cm': 17, 'cppm': 16, 'pym': 18}
        emarks ---> {'cm': 67, 'cppm': 78, 'pym': 70}
              cname ---> OUCET
>>> for v in d1.get("imarks"):
...     print(v)
...
                    cm
            cppm
        pym
>>> for v in d1.get("imarks"):
...     print(v,"-->",d1.get("imarks").get(v))
...
                        cm --> 17
                    cppm --> 16
            pym --> 18
>>> for v in d1.get("imarks"): ...
print(v,"-->",d1["imarks"][v])
...
                    cm --> 17
            cppm --> 16
        pym --> 18
```

- Inside of dict, we define list, tuple and set also.

Examples:

```
>>> d1= {"sno": 10, "sname": "Rossum", "imarks": [16,18,17], "emarks": (60,77,67), "crs": {"B.Tech(cse)", "M.Tech(AI)"}, "cname": "OUECT"} >>> for k, v in d1.items(): ...     print(k,"-->",v,"--->",type(v))
...
                    sno --> 10 ---> <class 'int'>                    sname --> Rossum ---> <class 'str'>                    imarks --> [16, 18, 17] ---> <class 'list'>                    emarks --> (60, 77, 67) ---> <class
```

'tuple'> crs --> {'B.Tech(cse)', 'M.Tech(AI)'} ---> <class
'set'> cname --> OUECT ---> <class 'str'> **dict in list**
is Possible:
Examples:

```
>>> l1=[10,"Rossum",{"cm":16,"cppm":17,"pym":18},"OUCET"]
>>> print(l1,type(l1))---[10, 'Rossum', {'cm': 16, 'cppm': 17, 'pym': 18}, 'OUCET'] <class
'list'>
>>> for val in l1: ...
print(val, type(val))
...
        10 <class 'int'>
        Rossum <class 'str'>
        {'cm': 16, 'cppm': 17, 'pym': 18} <class 'dict'>
        OUCET <class 'str'>
>>> for k, v in l1[2].items():
...    print(k,"--->",v)
...
      cm ---> 16
    cppm ---> 17
    pym ---> 18
```

- dict in tuple is Possible.

Examples:

```
>>> t1=(10,"Rossum",{"cm":16,"cppm":17,"pym":18},"OUCET")
>>> print(t1,type(t1))→(10,'Rossum',{'cm': 16,'cppm':17,'pym':18},'OUCET')<class
'tuple'> >>> for val in t1: ...    print(val, type(val))
...
        10 <class 'int'>
        Rossum <class 'str'>
        {'cm': 16, 'cppm': 17, 'pym': 18} <class 'dict'>
        OUCET <class 'str'>
>>> for k, v in t1[-2].items():
...    print(k,"-->",v)
...
        cm --> 16
      cppm --> 17
    pym --> 18
```

- dict in set not possible

```
>>> s1={10,"Rossum",{"cm":16,"cppm":17,"pym":18},"OUCET"} →
                                        TypeError: unhashable type: 'dict'
```

NoneType data type:

- 'NoneType' is one the pre-defined class and treated as None type Data type. • "None" is keyword acts as value for <class,'NoneType'>
- The value of 'None' is not False, Space , empty , 0.
- An object of NoneType class can't be created explicitly. **Examples:**

```
>>> a=None
>>> print(a, type(a))        → None <class 'NoneType'>
>>> a=NoneType()             → NameError: name 'NoneType' is not defined
>>> l1=[]
>>> print(l1.clear())        → None
>>> s1=set()
>>> print(s1.clear())        → None
>>> d1=dict()
>>> print(d1.clear())        → None
>>> d1={10:1.2,20:3.4}
>>> print(d1.get(100))       → None
```

1.4 Type Casting and Data Conversion

Type casting is an essential operation in Python, allowing developers to transform data from one type into another. This is particularly critical in data processing, input handling, mathematical models, file parsing, networking applications, and large-scale system integration.

Python makes type conversion simple by providing **five core built-in conversion functions**:

- int()
- float()
- bool()
- complex()
- str()

But understanding the full behaviour of these functions — especially the edge cases — requires deeper insight into Python's internal mechanics.

Purpose of Type Casting

Why type conversion is needed: 1.

User Input Handling

All user input arrives as **strings**.

```
age = input("Enter age: ")   # "25"    → This is a string
age = int(age)               # 25      → We converted into integer
```

2. **Numeric Computations**
   Many operations require numbers, not strings or Booleans.
3. **File & Database Processing**
   CSV files store values as text; type conversion is required for analytics.
4. **API & Network Data**
   JSON data often stores numbers as strings.

5. **Data Normalization**
   Converting mixed data types into consistent formats for machine learning.

Internal Mechanics of Type Conversion

Python follows these principles when converting types:

A. Every object has an internal type

Represented by:

***type*(*value*)**

B. Conversions produce a *new object*

Original data is not modified.

Example: `x = "10" y = int(x)` Memory diagram: `x ---> ["10"] y ---> [10]`

C. Python tries to interpret the input string numerically

If interpretation fails → `ValueError`.

D. Booleans follow integer rules (True = 1, False = 0)

Python internally maps:

```
True  → 1
False → 0
```

E. Floating-point conversions follow IEEE 754

Includes approximation and rounding behaviours.

The Five Fundamental Conversion Functions

We now explore each function in depth.

int() — Integer Conversion

Valid input in integer datatype conversion:

Input Type	Example	Result
Integer	int(10)	10
Float	int(4.9)	4 (floor toward zero)
Boolean	int(True)	1
Numeric string	int("25")	25

Invalid Inputs:

int("2.5") # *ValueError*
int("*abc*") # *ValueError*
int("1+2") # *ValueError*
int("5+2j") #*ValueError*
int(" 2 0") # ERROR: internal space

Converting Floats Python truncates decimals:

int(7.9) # 7
int(−3.8) # − 3

Python does **not** round — it **drops** the decimal part. `float()`

— Floating-Point Conversion

Valid input in float datatype conversion

Input Type	Example	Result
Integer	float(10)	10.0
Boolean	Float(True)	1.0
Floating string	Float("25.12")	25.12

Edge Cases:
$float("nan")$ $\#NaN$
$float(\text{inf})$ $\#Infinity$
$float("-inf")$ $\#Infinity$
Invalid Inputs
$float("abc")$ $\#\ ValueError$
$float("10.5.6")\#\ ValueError$

bool() — Boolean Conversion

Rule: Boolean conversion checks if the value is considered "truthy" or "falsy". **Falsy Values:**

Value	bool(value)
0	False
0.0	False
""	False
[]	False
{}	False
None	False

Truthy: Everything else (non-zero numbers, non-empty strings, collections).
Example:
$bool("hello")$ $\#\ True$
$bool(" ")$ $\#\ True$
$bool([])$ $\#\ False$

complex() — Complex Number Conversion

Basic usage:
$complex(5)$ $\#\ (5+0j)$ $complex(5, 2)$ $\#\ (5+2j)$
Invalid inputs:
$complex("7+2")$ $\#\ ValueError\ (missing\ 'j')$
Valid:
$complex("7+2j")$ $\#\ (7+2j)$

`str()` — String Conversion

It's always Valid. And converts any object to its string representation.

Examples: `str(123)` # "123" `str(True)` # "True" `str([1, 2, 3])` # "[1, 2, 3]"

`str(None)` # "None"

Conversion Compatibility Tables

From	To int	To float	To bool	To complex	To str
int	✓	✓	✓	✓	✓
float	✓	✓	✓	✓	✓
bool	✓	✓	✓	✓	✓
str (numeric)	✓	✓	✓	✓	✓
str (non-numeric)	✗	✗	✓	✗	✓
list	✗	✗	✓	✗	✓
None	✗	✗	False → ✓	✗	✓

CHAPTER 2 — OPERATORS AND EXPRESSIONS (FULL STRUCTURAL EXPANSION)

2. OPERATORS AND EXPRESSIONS (FULL STRUCTURAL EXPANSION)

2.1 INTRODUCTION — OPERATORS AND EXPRESSIONS

Operators and expressions form the computational engine of every Python program. Whether performing arithmetic calculations, evaluating conditions, manipulating bits, or chaining logical decisions, operators determine how data is processed and how results emerge during runtime. Understanding operators deeply is essential because:

- They drive **control flow**, **data transformations**, and **algorithmic logic**.
- Their precedence and evaluation rules can change the outcome of expressions.
- Misuse of operators is among the **top causes of logical bugs** in beginner and intermediate codebases.
- They link to Python's **object model**, since operators internally call methods such as __add__, __eq__, etc.

- They interact with **type casting**, **memory behaviour**, and **data structure semantics**. This section therefore introduces the foundational concepts behind Python's computational grammar.

What Are Operators?

Operators are **symbols** that instruct Python to perform specific operations on values or objects.

Formal Definition:

An operator is a special symbol used to perform an action on one or more operands, resulting in an expression that evaluates to a value.

Example:

$3 + 5 \quad \#+ is\ an\ operator$

$a * b \quad \# * is\ an\ operator$

$x\ and\ y \quad \#\ logical\ operator$

$p << 2 \quad \#\ bitwise\ shift\ operator$

Python supports many operator categories:

- Arithmetic
- Assignment
- Relational
- Logical
- Bitwise
- Identity
- Membership (covered later in Chapter 3/4)

Every operator maps internally to a method:

$+ \rightarrow \_add\_()$

$- \rightarrow \_sub\_() ==$

$\rightarrow \_eq\_()$

$>> \rightarrow \_rshift\_()$

This reflects Python's object-oriented design.

What Are Expressions?

Formal Definition:

An expression is a combination of operands and operators that Python evaluates to produce a result.

Examples:

$3 + 4 * 5$

$(10 - 3) / 7$

$a < b\ and\ b < c$

Python evaluates expressions using:

- Operator precedence
- Associativity rules
- Type-specific behaviour
- Short-circuit logic

In essence, **expressions ≡ executable mathematical/ logical statements**.

Role of Operators in Program Execution

Operators fundamentally drive the behaviour of programs in the following domains:

A. Arithmetic Computation Used in:

- Financial calculations
- Scientific computing
- Simulation models
- Game development

B. Logical Decision Making

Logical operators (and, or, not) control:

- if…else branching
- loop guard conditions
- data validation

C. Data Transformation

Bitwise operations manipulate:

- Encryption systems
- Frequency flags
- Low-level network protocols
- Permission bits

D. Object-Based Operations

Operators provide syntactic shortcuts for calling methods:

$a + b == a.\_\_add\_\_(b)$ This enables:

- Operator overloading
- Rich object interactions

E. Flow of Control Expressions determine:

- Entry vs exit conditions
- Loop continuation or termination
- Conditional branching logic

Operator Precedence and Associativity

Operator precedence defines **which operator executes first** in an expression.

Associativity defines **execution direction** when operators share the same precedence.

Precedence Example:

$3 + 4 * 5$ $\#\ evaluates\ as\ 3 + (4 * 5)$ **Associativity**

Example:

$2 ** 3 ** 2\ \#\ right-associative \rightarrow 2 ** (3 ** 2)$ Without parentheses:

- Code can become ambiguous
- Results may become unintuitive
- Bugs become harder to detect

Therefore, precedence and associativity must be mastered before writing complex expressions.

NOTE: Python Programming does not Support Unary Operators (++ , - -)

Python Programming does not Support Ternary Operator of C,C++,C#.net, Java (? :)

Python Programming Contains Shorthand Operators

Python Programming Contains Its Own Ternary Operator-- if...else operator

2.2 ARITHMETIC AND ASSIGNMENT OPERATORS

Arithmetic and assignment operators form the computational backbone of Python. They define how numerical data is processed, manipulated, and stored. This section examines each operator rigorously—its logical purpose, syntax rules, evaluation behavior, and edge cases.

Arithmetic Operators Overview

- The purpose of Arithmetic Operators is that "To perform Arithmetic Operations such as, Subtraction, Multiplication...etc".
- If two or more Variables (Objects) connected with Arithmetic Operators, then we call it as Arithmetic Expression.
- In Python Programming, we have 7 Arithmetic Operators. They are given in the following table.

Operator	Meaning	Example
+	Addition	a + b
-	Subtraction / Unary Negation	a - b, -a
*	Multiplication	a * b
/	True Division	a / b → float
//	Floor Division	a // b → int or float
%	Modulus (Remainder)	a % b
**	Exponentiation	a ** b

Addition (+)

Purpose:

- Adds two operands
- Concatenates sequences (strings, lists, tuples) **Examples:**

```
3 + 5            # 8
"Py" + "thon"    # "Python"
[1,2] + [3]      # [1,2,3]
3 + "3"          # TypeError
```

Subtraction (-)

Supports **binary subtraction** and **unary negation**. **Examples:**

```
7 - 3    # 4
-x       # negates the value
```

Multiplication (*)

Supports:

- Numeric multiplication
- Sequence repetition **Examples:**

```
4 * 3      # 12
"Hi" * 3   # "HiHiHi"
[1] * 4    # [1,1,1,1]
```

True Division (/)

Always returns a **float**. → `7 / 2  # 3.5`

Even for integers: → 10 / 5 # 2.0

ZeroDivisionError → 10 / 0

Floor Division (//)

Returns:

- Integer (if both operands int)
- Float (if operand float) **Examples:**

7 // 2 # 3

7.5 // 2 # 3.0

Floor division *rounds down* toward negative infinity: → −7 // 2 # − 4

Modulus (%)

Formula:

$a \% b = remainder\ of\ a \div b$

Examples:

10 % 3 # **1**

13 % 5 # **3**

Exponentiation (**)

Examples:

2 ** 3 # 8

10 ** 0.5 # 3.162277...

Right associative: 2 ** 3 ** 2 # 512 → Because → 2 ** (3**2)

Assignment Operator (=)

- The purpose of assignment operator is that "To assign or transfer Right Hand Side (RHS) value/ expression value to the Left-Hand Side (LHS) Variable"
- The Symbol for Assignment Operator is single equal to (=).
- In Python Programming, we can use Assignment Operator in three ways.

1. Single Line Assignment
2. Multi Line Assignment
3. Shallow vs Deep Assignment

1. Single Line Assignment:

Syntax: LHS Varname= RHS Value

LHS Varname= RHS Expression

- With Single Line Assignment at a time, we can assign one RHS Value/Expression to the single LHS Variable Name. Examples: >>> a=10

>>> b=20

>>> c=a + b

>>> print(a, b, c) → 10 20 30

2. Multi Line Assignment:

Syntax: Var1, Var2.... Var-n = Val1, Val2.... Val-n

Var1, Var2.... Var-n= Expr1, Expr2... Expr-n

Here The values of Val1, Val2...Val-n are assigned to Var1,Var2...Var-n Respectively.

Here The values of Expr1, Expr2...Expr-n are assigned to Var1,Var2...Var-n Respectively.

Examples:

```
>>> a, b=10,20
>>> print(a, b)                →10 20
>>> c, d, e=a + b, a - b, a * b
>>> print(c, d, e) →30  -10  200
>>> sno, sname, marks=10, "Rossum", 34.56
>>> print(sno, sname, marks)                →10 Rossum 34.56
```

3. Shallow vs Deep Assignment:

Shallow:

Two variables refer to same object.

$a = [1,2,3]$

$b = a$ **Deep**

copy:

import copy b = copy.
deepcopy(a)

Memory Binding Rules Assignment:

- **Does not copy data**
- **Creates references** Example:

$x = 10$

$y = x$ # Both reference same object → does not copy

Compound Assignment Operators

Compound assigners apply an operation and assignment in one step.

+= Addition Assignment

x += 5 # x = x + 5

-= Subtraction x

-= 2

***= Multiplication**

x *= 3

/= True Division x

/= 4

//= Floor Division x

//= 2

%= Modulus

x %= 7

****= Exponentiation** x

**= 2

Bitwise Assignment Forms

- &=
- |=
- ^=
- <<=
- >>=

In-Place vs Rebinding Behavior For
Immutable Types:

$x = 5$
$x \mathrel{+}= 1$ `# new object created` For

Mutable Types:

$lst = [1,2]$
$lst \mathrel{+}= [3]$ `# modifies existing list in place`

2.3 RELATIONAL AND LOGICAL OPERATORS

Relational and logical operators are central to decision-making in Python. They form the core of conditional execution, filtering, comparisons, authentication logic, data validation, and more. Understanding their evaluation model is fundamental for mastering control flow and algorithmic reasoning.

RELATIONAL (COMPARISON) OPERATORS

Relational operators compare values and return **Boolean results** (True or False). Python supports six relational operators:

Operator	Meaning	Example	Output
>	Greater than	5 > 3	True
Operator	**Meaning**	**Example**	**Output**
<	Less than	2 < 7	True
==	Equal to	4 == 4	True
!=	Not equal to	4 != 5	True
>=	Greater than or equal	5 >= 5	True
<=	Less than or equal	3 <= 4	True

Syntax and Purpose:
Relational operators mathematically compare two operands:
$operand1 < \boldsymbol{operator} > operand2$
The result is always a Boolean.
Examples:
$age = 20$
$age >= 18$ $\#\ True$
Comparison of Mixed Types:
Python does not allow arbitrary comparisons between incompatible types:
$\mathbf{3} < \text{"}\mathbf{3}\text{"}$ $\#\ TypeError$
But some combinations are allowed:
$\mathbf{3} == \mathbf{3.0}$ $\#\ True\ (numeric\ equivalence)$
Chained Comparisons:
Chained comparisons evaluate like mathematical expressions. **Example:**
1 < 2 < 3

Evaluated internally as:
1 < 2 and 2 < 3 But:
1 < 2 < 3 < 10 < 12 is
also perfectly valid.
Python does not evaluate all operands first — it evaluates progressively and short-circuits when possible.

Special Case: String Comparisons
Python compares strings **lexicographically** using Unicode code points.
"***apple***" < "***banana***" # *True*
ASCII/Unicode order makes:
"***Z***" < "***a***" # *True*
Examples:
10 > 3 # True
10 >= 10 # True
"dog" == "Dog" # False (case-sensitive)

LOGICAL OPERATORS

Python supports three logical operators:

Operator	Meaning	Type
and	Logical conjunction	binary
Operator	**Meaning**	**Type**
or	Logical disjunction	binary
not	Logical negation	unary

Logical operators return:

- **Boolean values**, or
- **Short-circuit values** (non-Boolean)

a) Logical AND (and)
Definition:
Returns **True** only if *both* operands are True.
Truth Table:

A	B	A and B
T	T	T
T	F	F
F	T	F
F	F	F

Short-Circuit Evaluation:

If A is False → Return A (stop)

Else → Return B

Examples:

True and True # *True*

0 ***and*** 10 # 0 (*short − circuits on first falsy*)

"" ***and*** "*Python*" # "" (*empty string is falsy*)

5 ***and*** 4 # 4 (*first truthy, returns second*)

b) Logical OR (or)

Definition:

Returns **True** if *at least one* operand is True.

Truth Table:

A	B	A or B
T	T	T
T	F	T
F	T	T
F	F	F

Short-Circuit Evaluation:

```
If A is True → Return A (stop)
Else → Return B
```

Examples:

```
0 or 100          # 100 (0 is falsy)
"" or "Hi"        # "Hi"
[] or [1,2]       # [1,2]
10 or 20          # 10 (first truthy returned)
```

c) Logical NOT (not) Definition:

Unary operator that reverses truth value. **Truth Table:**

A	not A
T	F
F	T

Examples:

```
not True          # False not 0
# True (0 is falsy) not "Hello"
# False
```

Truth Tables and Boolean Algebra:

Boolean algebra is the formal foundation for logical reasoning.

Python Boolean domain:

$$\boldsymbol{B} = \{\boldsymbol{True}, \boldsymbol{False}\}$$

AND Operator Truth Table **OR Operator Truth Table** **NOT Operator Truth Table**

A	B	A and B
T	T	T
T	F	F
F	T	F
F	F	F

A	B	A and B
T	T	T
T	F	F
F	T	F
F	F	F

A	Not A
T	T
T	F

Python evaluates based on:

a) **Precedence** Higher than:
- Logical AND
- Logical OR Lower than:
- metic operators

Example:

$$\mathbf{3 + 2 > 4 \ and \ 8 \ ! = 9}$$

b) **Associativity**

Highest → Lowest:

1. not
2. and
3. or

c) **Short-circuit behaviour** Example:

$$True\ or\ False\ or\ False \ \ \#\ Evaluated\ left\ to\ right$$

d) **Nested Expressions**

Use parentheses to override precedence: $(\mathbf{3 + 5}) < (\mathbf{4 * 2})$

e) **Common Mistakes Example:**

$$if\ a\ and\ b == 10:$$
$$pass$$

WRONG interpretation: $(a\ and\ b) == 10$

CORRECT: $a\ and\ (b == 10)$

Mistake-1: if x = 10: # SyntaxError Using = instead of ==

Mistake-2: name and name.lower() Ignoring Short-Circuit Behavior

Mistake-3: 10 > "5" # TypeError Comparing Incompatible Types Mistake-4:

```
x = 1000
y = 1000
  Misusing is instead of == x is y     #
  False (different objects)
```

Example1:

$$if\ username == db\_user\ and\ password == db\_pass:$$
$$print(\text{“}Login\ successful\text{”})$$ Example2:

$if\ temp > 40\ or\ humidity < 20$:
$activate\_alaram()$ Example3:
$result = [x\ for\ x\ in\ data\ if\ x > 10\ and\ x\ \%\ 2 == 0]$

2.4 BITWISE AND IDENTITY OPERATORS

Bitwise operators operate **only on integers**, treating them as sequences of bits rather than numeric magnitudes.
Python internally represents integers using **two's complement binary format**, enabling unlimited precision arithmetic.

BITWISE OPERATORS

Bitwise operators are designed to operate exclusively on integer data types. Instead of working with numeric magnitudes, these operators manipulate data at the binary level by treating integers as sequences of bits.
In Python, integers are internally represented using a two's complement–style binary representation, which enables efficient and conceptually unbounded precision arithmetic. This internal representation allows Python to seamlessly handle integers of arbitrary size while still supporting lowlevel bit manipulation.
The primary purpose of bitwise operators is to perform operations on binary data on a bit-by-bit basis. Each bit within the binary representation of an integer is individually evaluated and processed according to the specific bitwise operation being applied.
Bitwise operators are applicable only to integer values and not to floating-point numbers. This is because integers have an exact and deterministic binary representation, whereas floating-point values are stored using an approximate format that lacks absolute certainty at the bit level. As a result, bitwise manipulation on floating-point data is neither reliable nor meaningful.
The execution process of bitwise operators follows a well-defined workflow:

1. The given integer values are first converted into their binary representation.
2. The specified bitwise operation is applied to corresponding bits of the operands.
3. The resulting binary value is then converted back into an integer form.

Due to this direct manipulation of individual bits, these operators are collectively referred to as *bitwise operators*. In Python programming, there are **six types of bitwise operators**:

1. Bitwise Left Shift Operator (<<)
2. Bitwise Right Shift Operator (>>)
3. Bitwise OR Operator (|)
4. Bitwise AND Operator (&)
5. Bitwise Complement Operator (~)
6. Bitwise XOR Operator (^)

1. Bitwise Left Shift Operator (<<):

Definition: Shifts bits to the right, discarding bits on the right.
Syntax: $varname = GivenNumber << No.\ of\ Bits$
Concept: The Bitwise Left Shift Operator (<<) shifts No. of Bits Towards Left Side By adding No. of Zero at Right Side (depends on No. of Bits). So that result value will display in the form of decimal number system. **Examples:**

```
>>> a=10
>>> b=3
>>> c=a<<b
```

>>> print(c)----------80
>>> print(10<<3)--------80
>>> print(3<<2)---------12
>>>print(20<<2)-----80
>>> print(25<<3)-----200
>>> **# Left shift moves the binary bits of 25 (11001) three positions to the left, appending zeros on the right, which is equivalent to multiplying by 2^3 and results in 200 (11001000)**

2. Bitwise Right Shift Operator (>>)

Definition: Shifts bits to the left by *n* positions, inserting zeros on the right.

Syntax: $varname = GivenNumber >> No.\ of\ Bits$

Concept: The Bitwise Right Shift Operator (>>) shifts No. of Bits Towards Right Side By Adding No. of Zero at Left Side (depends on No. of Bits). So that value will display in the form of decimal number system . Examples: >>> a=10
>>> b=3
>>> c=a>>b
>>> print(c)----------1
>>> print(10>>2)----------2
>>> print(25>>4)----------1
>>> print(24>>3)---------3
>>> print(50>>4)--------3
>>> print(45>>2)--------11
>>> # **Right shift moves the binary bits of 45 (101101) two positions to the right, discarding the last two bits and yielding 11 (001011)**

3. Bitwise OR Operator (|)

Definition: Sets each bit to 1 if at least one corresponding bit is 1.
The Functionality of Bitwise OR Operator (|) is shown in the following Truth table. Truth Table:

A	B	A | B
0	0	0
0	1	1
1	0	1
1	1	1

Examples
>>> 0|0----------0
>>> 0|1----------1
>>> 1|0----------1
>>> 1|1----------1
>>> a=10
>>> b=12
>>> c=a | b
>>> print(c)--------14

>>># **Bitwise OR compares each corresponding bit of 10 (1010) and 12 (1100); if a bit is 1 in either operand, it is set to 1, resulting in 14 (1110)** >>> print(5|4)----5 **Most Imp Points:** >>>s1={10,20,30}
>>>s2={10,15,25}
>>> s3=s1|s2 # Bitwise OR Operator for UNION Operation
>>> print(s3,type(s3))--------{20, 25, 10, 30, 15} <class 'set'>
>>> s1={"Python", "Data Sci"}
>>> s2={"C","C++","DSA"}
>>> print(s1,type(s1))----------{'Python', 'Data Sci'} <class 'set'>
>>> print(s2,type(s2))----------{'C', 'DSA', 'C++'} <class 'set'>
>>> s3=s1|s2
>>> print(s3,type(s3))---------{'C', 'DSA', 'C++', 'Python', 'Data Sci'} <class 'set'>

4. Bitwise AND Operator (&)

Definition: Performs a logical AND operation on each corresponding pair of bits.
The Functionality of Bitwise AND Operator (&) is shown in the following Truth table. Truth Table:

A	B	A & B
0	0	0
0	1	0
1	0	0
1	1	1

Example:
>>> 0&1---------------0
>>> 1&0---------------0
>>> 0&0---------------0
>>> 1&1---------------1
>>> a=8
>>> b=6
>>> c=a & b
>>> print(c)-------0
>>> a=10
>>> b=12
>>> c=a & b
>>> print(c)--------8
>>> **# Bitwise AND compares each corresponding bit of 10 (1010) and 12 (1100); only bits that are 1 in both positions remain 1, producing the result 8 (1000) Note:**

- flags = 4 and 2   # Wrong
- flags = 4 & 2   # Correct

5. Bitwise Complement Operator (~)

Definition: Inverts all bits (one's complement), then apply two's complement.
Syntax: varname = ~Value
The Bitwise Complement Operator (~) will Invert the Binary Bits.
Inverting the Bits are nothing but 1 becomes 0 and 0 becomes 1

Formula:- Formula for ~Value = - (Value+1)

Example: >>>a=10

>>>b=~a

>>>print("Val of b=",b)---------- -11

>>> a=35

>>> b=~a

>>> print("Val of b=",b)----------Val of b= -36

>>> a=-101

>>> b=~a

>>> print("Val of b=",b)----------Val of b= 100

>>> **# Bitwise NOT inverts all bits of −101; in two's complement representation this yields the value 100, since ~a is mathematically equal to −(a + 1)**

6. Bitwise XOR Operator (^) Definition:

Returns 1 if bits differ; 0 if they are the same.

The Functionality of Bitwise XOR Operator (^) is shown in the following Truth table. Truth Table:

A	B	A ^ B
0	0	0
0	1	1
1	0	1
1	1	0

Example:

>>> 1^0----------1

>>> 0^1----------1

>>> 0^0----------0

>>> 1^1----------0

>>> a=10

>>> b=5

>>> c=a ^ b

>>> print(c)----------15

>>> a=10

>>> b=4

>>> c=a ^ b

>>> print(c)----------14

>>> a=9

>>> b=8

>>> c=a ^ b

>>> print(c)----------1

>>> **# Bitwise XOR compares each bit of 9 (1001) and 8 (1000); bits that differ are set to 1, producing the result 1 (0001)** >>> print(4^4)----------0 **Most Imp Points:**

>>> a={10,20,30}

>>> b={-10,15,25}

>>> c=a.symmetric_difference(b)

>>> print(c,type(c))---{15, 20, 25, 30} <class 'set'> **# Return the element which are not common** >>> d=a^b
>>> print(d,type(d))--------{15, 20, 25, 30} <class 'set'>

IDENTITY OPERATORS

Identity operators test **object identity**, not value equality. The purpose of Identity Operators is that "To Compare Memory Address of Two Objects".
In Python Programming, we have Two Types of Identity Operators. They are
1. == → compares values
2. is → compares memory addresses

1. ==

It compares the two value and returns a Boolean value.
Example:

```
>>> a = [1,2]
>>> b = [1,2]
>>> a == b              # True
>>> 10==21              #False
```

2. Is

Definition: Returns True if both variables reference the same object.
Syntax: Object1 is Object2
"is" Operator returns True Provided Both the objects Contains SAME Address.
"is" Operator returns False Provided Both the objects Contains Different Address.

Is not

Definition: Negation of identity comparison.
Syntax: Object1 is not Object2
"is not " Operator returns True Provided Both the objects Contains Different Address.
"is not " Operator returns False Provided Both the objects Contains Same Address. **Example:**

```
>>> a=None
>>> b=None
>>> print(a,type(a),id(a))      →None <class 'NoneType'> 140704386324472
>>> print(b,type(b),id(b))      →None <class 'NoneType'> 140704386324472
>>> a is b                      →True
>>> a is not b                  →False
>>> d1={10:"Apple",20:"Mango",30:"Kiwi"}
>>> d2={10:"Apple",20:"Mango",30:"Kiwi"}
>>> print(d1,type(d1),id(d1)) →{10: 'Apple', 20: 'Mango', 30: 'Kiwi'} <class 'dict'>
2864804287360 >>> print(d2,type(d2),id(d2)) →{10: 'Apple', 20: 'Mango', 30: 'Kiwi'} <class 'dict'>
2864804287488
>>> d1 is d2                    →False
>>> d1 is not d2                →True
>>> s1={10,20,30,40}
>>> s2={10,20,30,40}
>>> print(s1,type(s1),id(s1)) →{40, 10, 20, 30} <class 'set'> 2864805025088
>>> print(s2,type(s2),id(s2)) → {40, 10, 20, 30} <class 'set'> 2864805024640
```

>>> s1 is s2 →False

>>> s1 is not s2 →True

>>> fs1=frozenset(s1)

>>> fs2=frozenset(s1)

>>> print(fs1,type(fs1),id(fs1)) → frozenset({40, 10, 20, 30}) <class 'frozenset'> 2864805026208

>>> print(fs2,type(fs2),id(fs2)) → frozenset({40, 10, 20, 30}) <class 'frozenset'> 2864805026656

>>> fs1 is fs2 →False

>>> fs1 is not fs2 →True

Most Important:

>>> a, b=400,400 # Multi Line Assignment

>>> print(a, type(a),id(a)) →400 <class 'int'> 2864803995728

>>> print(b, type(b),id(b)) →400 <class 'int'> 2864803995728

>>> a is b →True

>>> a is not b →False

Note:

- What are all the objects Participates in DEEP Copy then Those Objects Contains Same Address and "is" operator Returns True and "is not" operator Returns False.
- What are all the objects Participates in SHALLOW DEEP Copy then Those Objects Contains Different Address and "is" operator Returns False and "is not" operator Returns True.
- **Why not == None?**
    - ○ is avoids overridden equality behaviour ○
    Faster and semantically correct
- x is 10 # Incorrect usage
- x == 10 # Correct

Membership OPERATORS

- The purpose of membership operators is that "To Check whether the Specific Value present in Iterable object Or Not".
- An Inerrable object is one, which contains a greater Number of Values (Examples: Sequence Type, List Type, Set type, Duct type)
- In Python Programming, we have 2 types of Membership Operators. They are

1. in
2. not in

1. In

Syntax: Value in IterableObject

"in" operator returns True Provided Value present in Iterable object

"in" operator returns False Provided Value does not present in Iterable object

2. Not in

Syntax: Value not in IterableObject

"not in" operator returns True Provided Value does not present in Iterable object

"not in" operator returns False Provided Value present in Iterable object **Example:**

```
>>> s="PYTHON"
>>> print(s, type(s))          →PYTHON <class 'str'>
>>> "P" in s                   →True
>>> "O" in s                   →True
>>> "O" not in s               →False
>>> "K" in s                   →False
>>> "K" not in s               →True
>>> "THON" in s                →True
>>> "THONS" in s               →False
>>> "THONS"  not in s          →True
>>> lst=[10,"Rossum",True,2+3j]
>>> print(lst, type(lst))      →[10, 'Rossum', True, (2+3j)] <class 'list'>
>>> 10 in lst                  →True
>>> True not in lst            →False
>>> 2 in lst[-1]               →TypeError: argument of type 'complex' is not iterable
>>> 3 in lst[-1].imag          →TypeError: argument of type 'float' is not iterable
>>> d1={10:"Python",20:"Java",30:"C++"}
>>> print(d1,type(d1))          →{10: 'Python', 20: 'Java', 30: 'C++'} <class 'dict'>
>>> 10 in d1                   →True
>>> 30 not in d1               →False >>>
40 in d1                       →False >>>
"Python" in d1          →False
>>> "Python" in d1[10]         →True
>>> "C++" in d1.get(30) →True
>>> "C++"[::-1] not in d1.get(30)[::-1]          →False
>>> "C++"[::-1] in d1.get(30)[::-1]              →True
```

2.5 OPERATOR PRECEDENCE

Definition: Operator precedence is the priority assigned to operators that determines the order in which operations are evaluated in an expression **without parentheses**.
Higher-precedence operators are evaluated **before** lower-precedence operators.

Why Precedence Exists?
Without precedence rules, expressions like **3 + 4 * 5** would be ambiguous.

Python applies precedence to ensure:

- Mathematical correctness
- Consistent evaluation
- Predictable behaviour across platforms

Python Operator Precedence Hierarchy

Below is a **simplified but complete precedence hierarchy**, from **highest to lowest**:

Priority	Operators	Priority	Operators
1	**() Parentheses**	8	**^**
2	**** Exponentiation**	9	**`**
3	**+ - ~ Unary operators**	10	**== != > < >= <=**
4	*** / // %**	11	**not**
5	**+ -**	12	**and**
6	**<< >>**	13	**or**
7	**&**	14	**= Assignment**

Definition of Associativity

Associativity determines the direction in which operators of the **same precedence** are evaluated.
Possible directions:

- **Left-to-right**
- **Right-to-left**

Left-Associative Operators:

Most operators in Python are **left-associative**. **Example:**
10 - 5 - 2
Evaluation: (10 - 5) - 2 = 3 Other
left-associative operators:

- +, -, *, /, //, %
- Relational operators
- Logical operators

Right-Associative Operators:

Only a few operators are right-associative.
Exponentiation (**): 2 ** 3 ** 2

Evaluation: 2 ** (3 ** 2)
= 2 ** 9
= 512

Assignment (=): a = b = c = 10
Evaluates right-to-left: b = c = 10
a = 10

PARENTHESES AND EXPLICIT CONTROL

Role of Parentheses: Parentheses () **override precedence and associativity**.
They:

- Increase readability
- Remove ambiguity
- Are recommended in production code **Parentheses Example:**

(10 + 5) * 2

Without parentheses:

10 + (5 * 2) = 20 With parentheses:

(10 + 5) * 2 = 30

Nested Parentheses

Python evaluates **innermost parentheses first**.

((2 + 3) * (4 - 1)) ** 2

Example of a step-by-step evaluation of the Operator precedence:

$result = not\ 3 + 2 > 4\ and\ 10\ \%\ 3 == 1$ Step-by-step:

1. 3 + 2 → 5
2. 5 > 4 → True
3. not True → False
4. 10 % 3 → 1
5. 1 == 1 → True
6. False and True → False

Final result: **False**

2.6 ERRORS, EDGE CASES, AND COMMON PITFALLS

Despite Python's readability and expressive syntax, operator misuse remains one of the most frequent sources of **logical defects, runtime failures, and silent bugs**. This section systematically analyzes common error patterns associated with operators and expressions, explains *why* they occur, and establishes best practices to mitigate them.

DIVISION BY ZERO

ZeroDivisionError

Division by zero is undefined in mathematics and explicitly prohibited in Python. **Examples:**

10 / 0

10 // 0

10 % 0

Error Raised: ZeroDivisionError: division by zero **Explanation:**

- /, //, and % require a non-zero divisor
- Python detects this at runtime and halts execution

Preventive Strategies: A.

Conditional Check:

```
if divisor != 0:
    result = dividend / divisor
```

B. Exception Handling (preview of Chapter 6):

```
try:
    result = dividend / divisor
except ZeroDivisionError:
print("Division not allowed")
```

COMPARISON OF INCOMPATIBLE TYPES

TypeError in Comparisons

Python does not allow arbitrary comparisons between unrelated types. **Invalid:**

```
10 > "5"
```

Error: TypeError: '>' not supported between instances of 'int' and 'str'

Why This Happens:

- Python 3 enforces **type safety**
- Prevents ambiguous or misleading comparisons

Safe Comparison Techniques

A. Explicit Type Conversion: int("5") > 3

B. Type Checking:

```
if isinstance(x, int):
    ...
```

LOGICAL OPERATOR SHORT-CIRCUIT MISUSE

Short-circuit evaluation is powerful but often misunderstood.

AND Short-Circuit Pitfall

$$result = x\ != 0\ and\ 10\ /\ x$$

✓ Safe when x == 0

✗ Dangerous if developer expects division to always occur

OR Short-Circuit Trap

$$username = user\_input\ or\ "guest"$$

If user_input = "", "guest" is used — possibly unintended.

Best Practice

Be explicit:

$$if\ user\_input == "":$$
$$\quad username = "guest"$$

CONFUSION BETWEEN LOGICAL AND BITWISE OPERATORS

One of the most frequent beginner and intermediate mistakes.

Logical vs Bitwise

Purpose	Logical	Bitwise
Operates on	truth values	binary bits
Operators	and, or, not	&, `

Incorrect Usage

$$if\ flags\ and\ 4:$$
$$\quad \ldots$$

This checks truthiness, **not bit presence**.

Correct Usage

$$if\ flags\ \&\ 4:$$

...

FLOATING-POINT PRECISION ERRORS

Floating-Point Representation Issue

$0.1 + 0.2 == 0.3$

Result: **False Why?**

Decimal fractions are approximated in binary (IEEE-754)

Safe Comparison

$abs((0.1 + 0.2) - 0.3) < 1e - 9$ Or:

$from\ decimal\ import\ Decimal$

UNARY OPERATOR CONFUSION

Negative Exponent Trap: $-2 ** 2$

Evaluates as: $-(2 ** 2) = -4$ Correct Intent:

$(-2) ** 2 = 4$

CHAINED COMPARISON MISINTERPRETATION

Correct Usage: 1 < x < 10

Incorrect Translation: (1 < x) < 10 # WRONG mental model

Python evaluates chained comparisons intelligently.

CHAPTER 3 — CONTROL FLOW STATEMENTS (LOGIC AND ITERATION)

3. CONTROL FLOW STATEMENTS (LOGIC AND ITERATION)

Control flow statements determine **how and when** different parts of a Python program execute. While operators compute values, **control flow decides behavior**. This chapter transforms static expressions into **dynamic, decision-driven programs**.

3.1 INTRODUCTION TO CONTROL FLOW

A program without control flow executes line-by-line from top to bottom and stops. Such programs are limited, rigid, and incapable of responding to conditions or repeating tasks.

Control flow statements enable:

- Conditional execution (decision-making)
- Repetitive execution (loops)
- Controlled interruption and redirection of execution
- Modular and readable program logic

In real-world systems—banking software, operating systems, data pipelines, APIs—**control flow is the backbone of correctness and reliability**.

What Is Control Flow?

Definition: Control flow refers to the order in which individual statements, instructions, or function calls of a program are executed.

Python modifies default sequential execution using:

- Selection statements
- Iterative statements
- Transfer (jump) statements

Why Control Flow Is Necessary?

Control flow allows programs to:

1. Make Decisions `if`

```
age >= 18:
        print("Eligible")
```

2. Repeat Operations `while`

```
balance > 0:
        process()
```

3. Handle Conditions Dynamically

```
    if error_detected:
        recover() else:
    continue_execution()
```

Categories of Control Flow Statements in Python

Python provides three major categories:

A. Conditional (Selection) Statements:

Used to execute blocks **based on conditions**.

- if
- if…else
- if…elif…else
- match…case (Python 3.10+)

B. Looping (Iterative) Statements Used to execute blocks **repeatedly**.

- While loop or while…else loop
- for loop or for...else loop

C. Transfer (Jump) Statements:

Used to alter the normal flow of loops and functions.

- break
- continue
- pass
- return

Nested OR Inner Loops

- while loop in while loop
- for loop in for loop
- while loop in for loop
- for loop in while loop

Relationship Between Control Flow and Operators

Control flow relies directly on:

- Relational operators (>, <, ==, etc.)
- Logical operators (and, or, not)
- Boolean expressions Example:

if x > 10 and x < 20:
print("Valid range")

Without a solid understanding of **Chapter 2**, control flow logic becomes error-prone and unpredictable.

3.2 CONDITIONAL SELECTION STATEMENTS

Conditional and selection statements enable a Python program to **make decisions at runtime**. They allow the program to choose between alternative execution paths based on the evaluation of Boolean expressions. Without conditional logic, programs remain static and incapable of responding to real-world variability.

This section explores Python's decision-making constructs in a structured and execution-oriented manner.

Purpose of Conditional Statements

Definition: A conditional statement is a control structure that executes a specific block of code only when a given condition evaluates to True.

Conditional statements are used to:

- Validate user input
- Enforce business rules
- Handle errors gracefully
- Implement decision trees
- Control access and permissions

In enterprise systems, conditional logic governs everything from authentication to transaction approval.

The if Statement:

if is a keyword in Python.

The test condition can evaluate to either True or False.

If the condition is True, the indented block of statements is executed, followed by other statements in the program.

If the condition is False, the indented block is skipped, and the program proceeds directly to execute other statements.

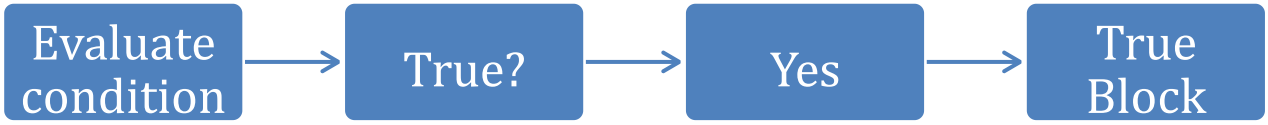
Syntax Structure:

```
if condition:
    # statement_block
```

Key Rules:

- The condition must evaluate to a Boolean value
- The colon (:) marks the start of the block
- Indentation defines the scope of execution

Flow Chat:



Example:

```
num = 5
if num % 2 == 0:
    print("Even")
```

The if…elif…else Statement:

Execute one block when the condition is True and another when it is False.

If the first condition is true, the corresponding block of statements is executed.

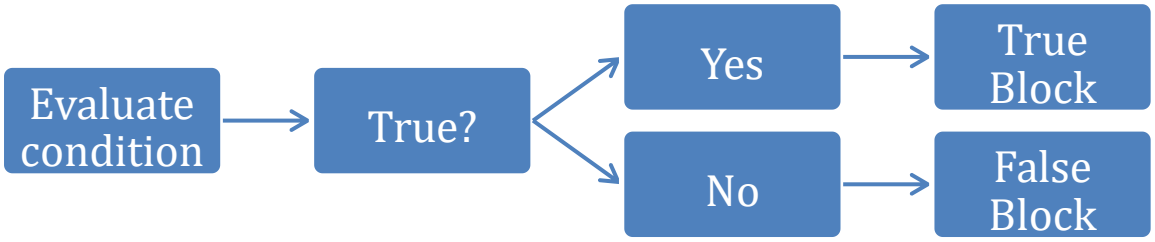
If the first condition is false, subsequent conditions are evaluated sequentially until a true one is found.

If all conditions are false, the optional else block is executed.

The else block is not mandatory for the program to run.

Syntax Structure:

```
if condition:
    # statement_block else:
    # statement_block
```



Example:

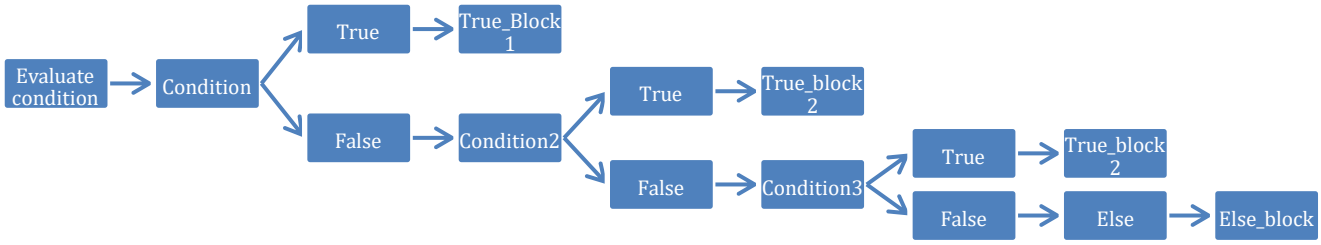
```
num = 5
if num % 2 == 0:
    print("Even") else:
    print("Odd")
```

The if…elif…else Statement

Used when **multiple mutually exclusive conditions** must be evaluated.

Syntax Structure:

```
if condition:
    # statement_block elif:
    # statement_block elif:
    # statement_block elif:
    # statement_block else:
    # statement_block
```



Example:

```
marks = 72
if marks >= 90:
    grade = "A"
elif marks >= 75:
grade = "B" elif
marks >= 60:
    grade = "C"
else:
    grade = "D"
```

The match…case Statement (Python 3.10+):

Here "**match case**" is one the new feature in Python 3.10 Version onwards.

Match case statement is recommended to take in deciding Pre-designed Conditions in Menu Driven Applications.

It provides a **structured alternative to long if–elif chains**, ideal for menu-driven and patternbased logic.

Syntax:

```
match(Choice Expr):
case Choice Label1:
        Block of Stements-1
case Choice Label2:
        Block of Stements-2
case Choice Label3:
        Block of Stements-3
    ---------------------------
case Choice Label-n:
        Block of Statements-n
    case _:    # Default Case Block
        default Block of Statements
```

Explanation:

- Here "match" and "case" are the keywords and "Choice Expr" represents either int, str or bool
- If "Choice Expr" is matching with "case label1" then PVM executes Block of Statements-1 and later executes Other statements in program.
- If "Choice Expr" is matching with "case label2" then PVM executes Block of Statements-2 and later executes Other statements in program.
- In General, "Choice Expr" is trying match with case label-1, case label-2,....case label-n then PVM executes corresponding block of statements and later executes Other statements in program.
- If "Choice Expr" is not matching with any of the specified case labels then PVM executes Default Block of Statements which are written under default case block(case _) and later executes Other statements in program.
- Writing default case block is optional and If we write then it must be written at last (Otherwise we get SyntaxError).
- When we represent multiple case labels under one case then those case labels must be combined with Bitwise OR Operator (|).

Example:

```
choice = 2
match choice:
    case 1:
        print("Add")
    case 2:
        print("Subtract")
    case 3:
        print("Multiply")
    case _:
        print("Invalid choice")
```

3.3 LOOPING AND ITERATIVE STATEMENTS

Looping statements allow a program to **execute a block of code repeatedly** until a specified condition becomes False or an Iterable is exhausted. Iteration is essential for handling collections, performing repeated computations, and building scalable logic.
Without loops, programs would require excessive code duplication, making them inefficient, errorprone, and difficult to maintain.

Purpose of Looping:

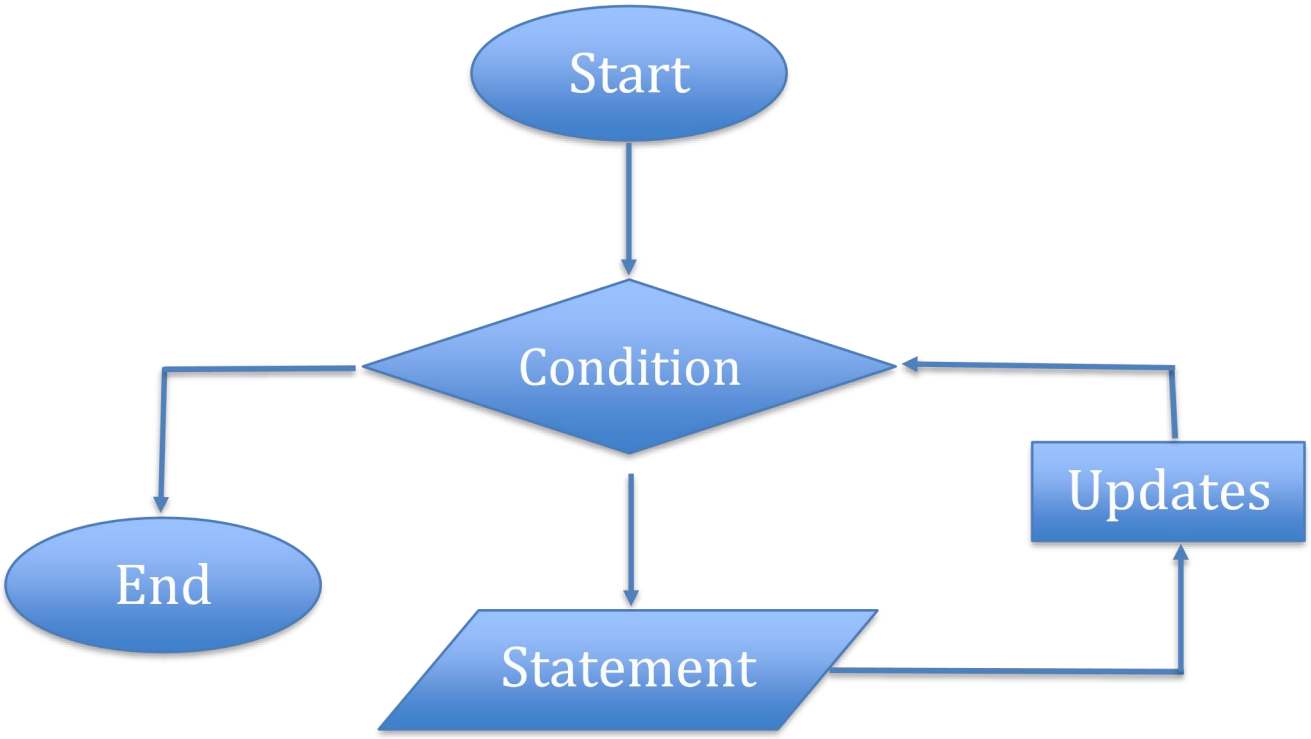
Definition: A loop is a control flow construct that repeatedly executes a block of code as long as a specified condition remains True or until all elements of an Iterable are processed.
Loops are used to:

- Process lists, files, and database records
- Automate repetitive tasks
- Implement counters and accumulators
- Traverse data structures
- Perform simulations and iterative algorithms

Fundamental Components of a Loop

Every loop logically consists of three components:
1. Initialization: Sets the starting point of iteration.
2. Condition: Determines whether the loop should continue.
3. Updation: Changes the loop variable to avoid infinite execution.

Control Flow:



The while Loop:

The while loop executes a block **as long as the condition evaluates to True**. **Syntax:**

```
while condition:
    #loop_body
      or
while condition:
    #loop_body else:
    #else_block
```

Explanation:

- Here "while" and "else" are called keywords. Here condition can be either True or False.
- If the condition is True, then PVM execute indentation block of statements and once again PVM control goes to condition. If once again conditions is True, then PVM executes indentation block of statements. This process repeated for finite number of times until conditions become false.
- Once condition becomes false, PVM executes else block of statements which are written else block(if present) and later executes other statements program.

Example 1:

```
i = 1
while i <= 5:
  print(i)
  i += 1
```

Output:

```
1
2
3
4
5
```

Example 2:

```
i = 10
while i > 5:
  print(i)
  i -= 1
else:
   print("i is less than 5")
```

Output:

```
10
9
8
7 6 i is less
than 5
```

Infinite Loop

Occurs when the condition never becomes False.

```
while True:
  print("Running forever")
```

Prevention:

- Ensure proper updation.
- Use break when necessary

The for Loop

Definition: The for loop iterates over elements of an **Iterable object**.

Syntax:

```
for variable in iterable:
    loop_body
        OR
for variable in iterable:
# for block  else:
    # else block
```

- Here 'for', 'in' and 'else' are keywords.
- Here iterable can be Sequence(bytes, bytearray, range, str), list(list, tuple), set(set, frozenset) and dict.
- The execution process for loop is "Each of Element of iterable selected, placed in variable and executes Indentation block of statements. This Process will be repeated until all elements of iterable completed.
- When all the elements of Iterable complete then PVM comes out of for loop and executes else block of statements which are written under else block.
- Writing else block is optional.

Example 1:

```
data = [1,"Hari", ["a", "b"], range(10), {3,2,4,2,5,1,3}]
for i in data:
print(i)
```

Output:

```
1
Hari
['a', 'b']
range(0, 10)
{1, 2, 3, 4, 5}
```

Example 2:

```
numbers = [2,4, 6, 8, 10]
target = 7
for num in numbers:
    if num == target:
        print("Target found: ",
num)       break else:
    print("Target not found in the list")
```

Output:

```
Target not found in the list
```

Example 3:

```
data = {"a": 1, "b": 2}
for key, value in data. items():
print(key, value)
```

Output:

```
a 1
b 2
```

Difference Between while and for

Feature	while	for
Condition-based	Yes	No
Iterable-based	No	Yes
Risk of infinite loop	High	Low
Use case	Unknown iterations	Known collections

Inner OR Nested Loops

- The Process of Defining One Loop inside of another Loop is called Nested OR Inner Loop.
- The Execution Process of Nested Loops is that "For Every Value of Outer Loop, Inner Loop executes repeatedly for Finite Number of Times".
- Inner or Nested Loops Can be Used with 4 Syntaxes.

Syntax-1 : for loop in for loop:

```
for Varname1 in Iterbale_object1: # Outer For Loop
    # outer block statement
    for Varname2 in Iterbale_object2: # Inner For Loop
# inner block statement    else:
        # inner block else statement else:
    # outer block else statement
```

Syntax-2: while loop in while loop:

```
while (condition1): # Outer while Loop
    # outer block statement
    while (condition2): # Inner while Loop
# inner block statement    else:
        # inner block else statement else:


    # outer block else statement
```

Syntax-3 : while loop in for loop:

```
for Varname1 in Iterbale_object1: # Outer For Loop
    # outer block statement
    while (condition2): # Inner while Loop
# inner block statement    else:
        # inner block else statement else:
    # outer block else statement
```

Syntax-4: for loop in while loop:

```
while (condition1): # Outer while Loop
    # outer block statement
    for Varname1 in Iterbale_object1: # Inner for Loop
# inner block statement    else:
        # inner block else statement else:
    # outer block else statement
```

3.4 TRANSFER CONTROL STATEMENTS

Transfer control statements modify the default flow of execution within loops and functions. While conditional and iterative statements determine *whether* code executes, transfer statements determine *when execution should change direction*.

They are essential for:

- Terminating loops early
- Skipping specific iterations
- Creating placeholder blocks
- Returning values from functions

The purpose of Transfer Flow Control Statements in Python is that "To change the control of PVM from One Part of the Program to another part of Program".

Python provides four primary transfer control statements:

1. break
2. continue
3. pass
4. return

Formal Definition: A transfer control statement changes the normal sequential execution of a program by redirecting control to another part of the code. These statements are used to:

- Exit loops prematurely
- Skip unnecessary computations
- Handle exceptional logic paths
- Control function termination

In enterprise applications, they help optimize performance and maintain clean execution logic.

The break Statement

Definition: The break statement **immediately terminates the nearest enclosing loop** and transfers control to the statement following the loop.

- The purpose of break statement is that "To terminate the execution of loop logically when certain condition is satisfied and PVM control comes of corresponding loop and executes other statements in the program".
- When break statement takes place inside for loop or while loop then PVM will not execute corresponding else block(Because loop is not becoming False) but it executes other statements in the program

Syntax1:

```
for Varname in Iterbale_object:
    if (condition):
        break
```

Syntax2: :

```
while (condition1):
    if (condition2):
        break
```

Common Mistake

Using break outside a loop:

```
while (condition):
    break   # SyntaxError
```

Example1:

```
for user in users:
    if user.id == uname:
        print("User found")
        break
```

Example2:

```
while slot_number <= total_slots:
    if status == "yes":
        print("Available parking slot found.")
        break
```

The Continue Statement

Definition: The continue statement **skips the remaining code in the current iteration** and moves to the next iteration of the loop.

- Continue statement is used for making the PVM to go to the top of the loop without executing the following statements which are written after continue statement for that current Iteration only.
- Continue statement to be used always inside of loops.
- When we use continue statement inside of loop then else part of corresponding loop also executes provided loop condition becomes false.

Syntax1:

```
for Varname in Iterbale_object:
    if (condition):
        continue
```

Syntax2: :

```
while (condition1):
    if (condition2):
        continue
```

Common Mistake

Using break outside a loop:

```
while (condition):
    continue   # SyntaxError
```

Example1:

```
for emp in employees:
    if emp.status == "inactive":
        continue
    print(Active employee:,emp.name)
```

Example2:

```
while time < 10:
    if store_status == "closed":
        continue
    print("Store is open")
```

The Pass Statement

Definition: The pass statement is a **null operation**. It does nothing but serves as a syntactic placeholder.

Purpose: Used when:

- A block is required syntactically
- Logic will be implemented later • Avoiding syntax errors in empty blocks

Syntax1:

```
for Varname in Iterbale_object:
    if (condition):
pass
```

Syntax2: :

```
if (condition2):
    pass
```

Example1:

```
feature_enabled = False
if feature_enabled:
pass else:
    print("Using stable version")
```

Example2:

```
for user in ["Admin", "Guest", "Manager"]:
  if user == "Guest":
    pass  # Access rules to be defined later
else:
    print(f"Access granted to {user}")
```

Difference from continue:

pass	continue
Does nothing	**Skips iteration**
No flow change	**Alters loop flow**

The return Statement

Definition: The return statement **exits a function** and optionally sends a value back to the caller.

Syntax:

```
return
return value
```

Example1:

```
def add(a, b):
  return a + b
```

Example2:

```
def validate(x):
if x < 0:
    return
  print("Valid")
```

Comparison of Transfer Control Statements

Statement	Scope	Effect
break	Loop	Terminates loop
continue	Loop	Skips iteration
pass	Block	No action
return	Function	Exits function

CHAPTER 4 — FUNCTIONS, SCOPES, AND ABSTRACTION

4. FUNCTIONS, SCOPES, AND ABSTRACTION

Modern software systems demand **modularity, scalability, and maintainability**. Python achieves these objectives through the strategic use of **functions**, which allow developers to decompose complex problems into smaller, manageable, and reusable units.

This chapter establishes a strong conceptual and practical foundation in:

- Functional abstraction
- Argument passing
- Variable scope management
- Anonymous functions
- Functional programming utilities

4.1 FUNDAMENTALS OF FUNCTIONS

Purpose of Functions: Functions serve as the building blocks of structured programming. They encapsulate logic, promote reusability, and improve program organization.

Strategic Benefits:

- Reduced development effort through reuse
- Improved readability via modular design
- Lower maintenance cost
- Faster debugging cycles
- Team scalability in enterprise projects **Example:** $def\ greet()$:

$print("Welcome\ to\ Python\ Programming")$

Types of Programming Languages

In the content of Functions, we have Two Types of programming Languages. They are

a) Un-Structured Programming Language

b) Structured Programming Language

a) Un-Structured Programming Language:

- In Un-Structured Programming Lang, We don't have the concept of FUNCTIONS.
- Since Un-Structured Programming Lang does not contains the concept of FUNCTIONS and Hence whose Applications having the following Limitations.

1. Application development Time is More
2. Application takes More memory Space
3. Application execution time is More
4. Application Performance Degraded
5. Redundancy (Duplication or Replication) of the Code is More.

b) Structured Programming Languages:

- In Structured Programming Lang, We have the concept of FUNCTIONS.
- Since Structured Programming Lang contains the concept of FUNCTIONS and Hence whose Applications having the following Advantages.

1. Application development Time is Less
2. Application takes Less memory Space
3. Application execution time is Less

4. Application Performance Enhanced (Improved)
5. Redundancy (Duplication or Replication) of the Code is Minimized.

Definition of a Function

Formal Definition

A function is a named block of reusable code that performs a specific task and can be executed whenever required.

- Functions act as **sub-programs** within a larger program.

Parts of a Function

At the time Developing functions in real time, we must ensure that, there must exist 2 Parts. they are

1. Function Definition: It consists of:

- def keyword
- Function name
- Parameters
- Function body
- Optional return statement

2. Function Calls:

- Invokes the function for execution
- Every Function Definition Exists Only Once.
- Every Function call must contain a Function Definition Otherwise we get NameError.
- Function Definition will execute when we call by using function calls OR Without calling the Function by using Function Calls PVM will not execute Function Definition.

Phases in Functions

At the time Defining the functions, the Programmer must ensure that there must exist the following Phases.

1. **Input Phase** – Accepting data
2. **Processing Phase** – Performing operations
3. **Output Phase** – Returning results **Example:** def $square(x)$:

$return\ x * x$

Syntax for Defining Functions:

$def\ functionname(list\ of\ parameters\ if\ any)$:
  """$doc\ string$"""
  $statement - 1$
  $statement - 2$
$return\ value$

Explanation :-

1. here 'def' is a keyword, which is used for defining Programmer-defined Functions.
2. "functionname" represents a valid variable name and treated as function name and every function name is an object of type <class, 'function'>
3. "list of formal params" represents list of variable names used in function heading and they are used for storing or holding the input(s) coming from function call(s).
4. """doc string""" represents documentation string and it used for giving or writing the description about functionality of function.
5. The statement-1,statement-2 .... statement-n indentation block of statements and it is processing the input or logic for problem solving and it is known Business Logic.

6. In the Function Body, we use some special variables, and they are called Local Variables and whose purpose is to store the temporary results.
7. The values of Formal Params and Local Variables can be accessed only inside corresponding Function Definition but not possible to access in other part of the program and in Other Function Definitions(Scope of Formal params and Local Var) **Approaches to Define Functions:**

Functions can be defined in multiple ways:

Approach	Parameters	Return
Type 1	No	No
Type 2	Yes	No
Type 3	No	Yes
Type 4	Yes	Yes

Execution Flow of a Function:

4.2 ARGUMENTS AND PARAMETERS

Functions become powerful only when they can **accept data dynamically**. This is achieved through **parameters** (in the function definition) and **arguments** (in the function call). Python supports multiple argument-passing mechanisms, enabling flexibility, readability, and scalability in program design.

Formal Parameters vs Arguments

Formal Parameters

- Variables declared in the function definition.
- The variables used in Function Heading are called Formal Parameters and They are used for Storing the inputs coming Function Calls.
- The Variables Used in Function Body are called Local Variables / Parameters and They are used for Storing Temporary Results / Function Processing Logic Results.
- The Values Formal Parameters and Local parameters can be accessed within corresponding Function Definition but not possible to access in Other Part of the Program • Example: $def\ add(a, b)$: $\#\ a\ and\ b\ are\ parameters$
  $return\ a + b$

Arguments

- Arguments are the variables / Values which are used as Variables in Function Calls.
- Actual values passed during the function call.
- Example: $add(10, 20)$ $\#\ 10\ and\ 20\ are\ arguments$

The relationship between Arguments and Parameters is that all the Values of arguments are passing to Parameters. This Mechanism is called Arguments Passing.

TYPES OF ARGUMENTS IN PYTHON

As we Know that all the Values of arguments are passing to Parameters.
Based on Passing the values of Arguments from Function to Formal Parameters of Function Definition, The arguments are classified into 5 Types. They are:

1. Positional Arguments.
2. Default Arguments
3. Keyword Arguments
4. Keyword Variables Length Arguments(**kwargs)
5. Variables Length Arguments

1. Positional Arguments.

- The Concept of Positional Parameters (or) arguments says that "The Number of Arguments of Function Call must be equal to the number of formal parameters in Function Heading".
- This Parameter mechanism also recommends following Order and Meaning of Parameters for Higher accuracy.
- Pass the Specific Data from Function Call to Function Definition then we must take Positional Argument Mechanism.
- The default Argument Passing Mechanism is Positional Arguments (or) Parameters.
- Syntax for Function Definition :

```
def functionname(param1,param2....param-n):
    # code
```

- Syntax for Function Call: functionname(arg1,arg2....arg-n)

- Here the values of arg1,arg2...arg-n are passing to param-1,param-2...param-n respectively.
- PVM gives First Priority to Positional Arguments.

Example:

```
def subtract(a, b):
    return a − b

subtract(10,5)      # Output: 5
```

2. Default Arguments

- When there is a Common Value for family of Similar Function Calls then Such type of Common Value(s) must be taken as default parameter with common value (But not recommended to pass by using Positional Parameters)
- Syntax for Function Definition with Default Parameters:

```
def functionname(param1,param2, . . . . param − n − 1 = Val1,Param − n = Val2):
    # code
```

- Here param-n-1 and param-n are called "default Parameters". and param1,param-2... are called "Positional parameters".

Example:

```
def greet(name = "User"):
    print("Hello",name)
greet()          # Hello User
greet("Alex")  # Hello Alex
```

Note: When we use default parameters in the function definition, they must be used as last Parameter(s) otherwise we get Error(SyntaxError: non-default argument (Positional) follows default argument).

Invalid: $def\ demo(a = 10, b)$: # SyntaxError

3. Keyword Arguments

- In some of the circumstances, we know the function name and formal parameter names, and we do not know the order of formal Parameter names and to pass the data / values accurately we must use the concept of Keyword Parameters (or) arguments.
- The implementation of Keyword Parameters (or) arguments says that all the formal parameter names used as arguments in Function call(s) as keys.
- Syntax for function definition:-

  $def\ functionname(param1, param2 \ldots param - n)$:

  $\#\ Code$
- Syntax for function call:-

  $functionname(param - n = val - n, param1 = val1, param - n - 1 = val - n - 1, \ldots)$

- Here param-n=val-n,param1=val1,param-n-1=val-n-1,..... are called Keywords arguments
- When we specify Keyword arguments before Positional Arguments in Function Calls(s) then we get SyntaxError: positional argument follows keyword argument.
- **Advantages** ○ Order does not matter. ○ Improves readability.
  ○ Reduces logical errors.
- **Example:**

```
def divide(a, b):
    return a / b

divide(b = 2, a = 10)
```

4. Keyword Variables Length Arguments(**kwargs)

- When we have family of multiple function calls with Variable number of values / arguments then with normal python programming, we must define multiple function definitions. This process leads to more development time.
- Overcome this process, we must use the concept of Variable length Parameters .
- To Implement, Variable length Parameters concept, we must define single Function Definition and takes a formal Parameter preceded with a symbol called asterisk (* param) and the formal parameter with asterisk symbol is called Variable length Parameters and whose purpose is to hold / store any number of values coming from similar function calls and whose type is <class, 'tuple'>.
- Syntax for function definition with Variables Length Parameters:

  $def\ functionname(list\ of\ formal\ params, *param1, param2 = value)$: #

  $Code$

- Here *param1 is called Variable Length parameter and it can hold any number of argument values (or) variable number of argument values and *param1 type is <class, 'tuple'>
- **Internal Representation:** Data becomes a dictionary.

Example:

```
def info(** data):
    print(data)          # {'name': 'Alex','age': 25,'city': 'Delhi'}
```

$$info(name = "Alex", age = 25, city = "Delhi")$$

- Rule:- The *param1 must always written at last part of Function Heading and it must be only one (but not multiple)
- Rule:- When we use Variable length and default parameters in function Heading, we use default parameter as last and before we use variable length parameter and in function calls, we should not use default parameter as Key word argument because Variable number of values are treated as Positional Argument Value(s) .

5. Variables Length Arguments

- When we have family of multiple function calls with Key Word Variable number of values / arguments then with normal python programming, we must define multiple function definitions. This process leads to more development time.
- =>To overcome this process, we must use the concept of Keyword Variable length Parameters .
- =>To Implement, Keyword Variable length Parameters concept, we must define single Function Definition and takes a formal Parameter preceded with a symbol called double asterisk (** param) and the formal parameter with double asterisk symbol is called Keyword Variable length Parameters and whose purpose is to hold / store any number of (Key, Value) coming from similar function calls and whose type is <class, 'dict'>.
- Syntax for function definition with Keyword Variables Length Parameters:

  $def \quad functionname(list\ of\ formal\ params, *param)$: # $Code$

- Here **param is called Keyword Variable Length parameter and it can hold any number of Key word argument values (or) Keyword variable number of argument values and **param type is <class,'dict'>
- Rule:- The **param must always written at last part of Function Heading and it must be only one (but not multiple)
- Final Syntax for defining a Function:
  $def \ \ funcname(PosFormal\ parms, *Varlenparams, default\ params, **kwdvarlenparms)$:
  # $Code$

Example:

```
def total(*nums):
    print(sum(nums))          # 60
total(10, 20, 30)
```

COMBINING DIFFERENT ARGUMENT TYPES

Order Rule: Positional → Default → *args → **kwargs **Example:**

$def\ demo(a, b = 5, *args, **kwargs)$:

$print(a, b, args, kwargs)$

4.3 VARIABLE SCOPE AND GLOBAL CONTROL

In any programming language, variables represent data stored in memory. However, **where** a variable can be accessed and **how** it behaves depends on its **scope**. Python enforces strict scope rules to ensure predictable execution, data safety, and modular program design.

Improper scope management can lead to:

- Data corruption
- Logical ambiguity

- Unintended side effects
- Hard-to-debug programs

This section provides a **deep, systematic exploration** of variable scope in Python, including local and global variables, the global keyword, the globals() function, and professional best practices.

Concept of Variable Scope

Formal Definition: Variable scope refers to the region of a program in which a variable is accessible, valid, and recognized by the interpreter.

In Python, scope determines:

- Variable visibility
- Lifetime of variables
- Memory access rules
- Name resolution behaviour

Python primarily uses two fundamental scopes:

1. **Local Scope**
2. **Global Scope**

(Advanced scopes such as Enclosing and Built-in are discussed later in functional programming contexts.)

Why Scope Control Is Necessary

Without scope control:

- Variables could overwrite each other
- Functions would not be independent
- Data security would be compromised
- Debugging would become chaotic **Scope enables:**
- Encapsulation
- Data isolation
- Modular programming
- Predictable execution

In enterprise systems, strict scope control is critical for:

- Multi-module applications
- Concurrent processing
- Secure data handling
- Large codebases

LOCAL VARIABLES

Definition: A local variable is a variable declared inside a function and accessible only within that function.

```
def sample():
    x = 10   # Local variable
    print(x)
```

Here, x exists only while sample() is executing.

Lifetime of Local Variables:

Local variables:

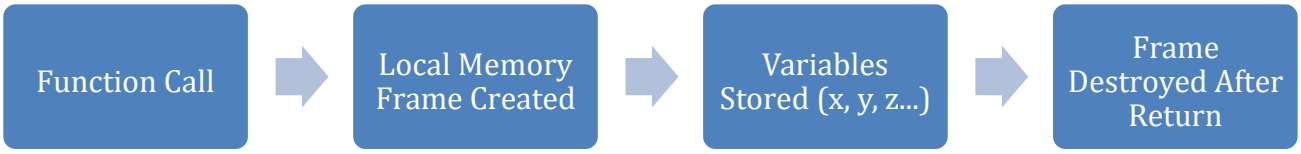
- Are created when the function is called
- Exist in the function's **stack frame** • Are destroyed after the function finishes

```
def demo():
```

```
    y = 50
demo()
print(y)  # NameError
```

Memory Perspective

Each function call creates a **new memory frame**:



This ensures:

- Data isolation
- No interference between function calls **Purpose of Local Variables** Local variables

are used to:

- Store intermediate results
- Maintain function independence
- Prevent data leakage
- Improve modularity

Example:

```
def calculate():
    result = 5 * 10
    return result
```

GLOBAL VARIABLES

Definition: A **global variable** is declared outside all functions and is accessible throughout the program.

```
total = 100 def
show():
    print(total)
```

Global Variable:

- Exist for the entire program execution
- Are stored in the **global namespace**
- Can be accessed from any function (read-only by default)

Reading Global Variables:

Python allows reading global variables inside functions without any special declaration.

```
count = 25
def display():
    print(count)
```

This is safe and common for:

- Configuration values
- Constants
- Read-only data

Name Shadowing (Ambiguity):

If a local variable has the same name as a global variable:

```
x = 10
```

```
def test():
    x = 20
    print(x)
```

Output: 20 #The local variable shadows the global one.

Why Shadowing Can Be Dangerous Shadowing:

- Creates confusion
- Makes debugging harder
- Hides global data unintentionally Best practice:

Use **distinct names** for local and global variables.

MODIFYING GLOBAL VARIABLES — global KEYWORD

Why global Is Required

By default, Python assumes any variable assigned inside a function is **local**.

```
x = 10
def change():
    x = 50  # Local variable, not global
```

The global x remains unchanged.

Correct Usage of global

```
x = 10
def change():
    global x
    x = 50
change()
print(x)
```

Output: 50

How global Works Internally The global keyword:

- Tells Python to link the variable to the **global namespace**
- Prevents local variable creation
- Allows modification of global memory

When global Should Be Avoided Excessive use of global:

- Breaks modularity
- Increases coupling
- Causes unpredictable behaviour
- Complicates testing Prefer:

```
def update(x):
    return x + 1
```

Instead of:

```
global x
```

THE globals() FUNCTION

Definition: globals() returns a **dictionary of all global variables**.

$print(globals())$

Accessing Global Variables Dynamically:

$x = 100$
$print(globals()["x"])$

Modifying Global Variables

$globals()["x"] = 500$

Now x becomes 500.

Use Cases of globals()

- Debugging
- Framework configuration
- Dynamic variable management
- Reflection

Should be used cautiously.

COMMON SCOPE ERRORS

UnboundLocalError:

$x = 10$
$def\ demo():$
  $print(x)$
  $x = 5$

Python assumes x is local because of assignment.

NameError:

$def\ test():$
  $y = 10$
$print(y)$

Accidental Global Modification:

Using global without caution can corrupt application state.

4.4 ANONYMOUS (LAMBDA) FUNCTIONS

In Python, not all functions require a formal name or a multi-line definition. For simple, short-lived operations, Python provides **anonymous functions**, commonly known as **lambda functions**. These functions enable concise, inline functional programming while preserving the expressive power of Python.

Lambda functions are widely used in:

- Data processing
- Functional programming
- Sorting and filtering
- Event handling
- API callbacks
- Mathematical transformations

This section provides a comprehensive exploration of lambda functions, their syntax, behaviour, limitations, and professional usage patterns.

What Is an Anonymous (Lambda) Function?

Formal Definition: A lambda function is a small, unnamed function defined using the lambda keyword, capable of containing a single expression and returning its result implicitly.

Unlike regular functions, lambda functions:

- Do not use the def keyword
- Do not have a function name
- Contain only one expression
- Return values automatically

Why Lambda Functions Exist

Lambda functions are designed to:

- Reduce boilerplate code
- Improve readability for simple logic
- Enable functional programming patterns
- Support inline operations
- Avoid unnecessary named functions Example comparison:

Normal Function

$def\ square(x)$:
    $return\ \boldsymbol{x} * \boldsymbol{x}$

Lambda Function

$square = lambda\ x{:}x\ * x$

Syntax of Lambda Functions

General Syntax: $lambda\ parameters{:}\ expression$

Key Characteristics:

- No return keyword
- Expression result is returned automatically
- Can take multiple parameters
- Only one expression allowed

Execution Model of Lambda Functions Internally, lambda functions:

1. Are treated as function objects
2. Are stored in memory like normal functions
3. Can be assigned to variables
4. Can be passed as arguments
5. Can be returned from functions

Example:
$f = lambda\ x{:}\ x + 10$
$print(f(5))$ # 15

Lambda vs Normal Functions

Feature	Lambda	Normal Function
Name	Optional	Required
Body	Single expression	Multiple statements
Return	Implicit	Explicit
Readability	Best for small logic	Best for complex logic
Debugging	Harder	Easier

Multiple Parameters in Lambda

$add = lambda\ a, b{:}\ a + b$
$print(add(10, 20))$

Lambda with Conditional Logic

$$max\_value = lambda\ a, b: a\ if\ a > b\ else\ b$$

This uses Python's ternary operator inside a lambda.

Lambda with No Parameters

$$greet = lambda: "Hello"$$
$$print(greet())$$

Memory Perspective of Lambda Functions

Lambda functions:

- Are objects stored in memory
- Have no fixed name unless assigned
- Exist only as long as references exist Example:

$$func = lambda\ x: x\ *\ 2$$

Once func is deleted, the lambda is removed by the garbage collector.

Lambda in Functional Programming

Lambda functions enable **functional programming**, where:

- Functions are treated as data
- Functions can be passed as arguments
- Functions can be returned
- Operations are expression-based Example:

$$def\ operate(f, x):$$
$$\quad return\ f(x)$$
$$print(operate(lambda\ y: y\ *\ y, 5))$$

Lambda with Built-in Functions

Lambda functions are most powerful when used with:

- map()
- filter()
- reduce()
- sorted() Example:

$$numbers = [1,2, 3, 4]$$
$$squares = list(map(lambda\ x: x * x, numbers))$$

Lambda with Sorting

$$students = [("John",85), ("Alex",92), ("Bob",78)]$$
$$students.sort(key = lambda\ x: x[1]) \qquad \#\ Sort\ by\ marks$$

Limitations of Lambda Functions Lambda functions:

- Cannot contain multiple statements
- Cannot include loops
- Cannot contain assignments
- Cannot include try-except
- Cannot use return

They are **not a replacement** for normal functions.

Common Errors with Lambda

Trying to use multiple statements

$$lambda\ x: print(x); x + 1$$

Complex logic in lambda

$$lambda\ x: x + 1\ if\ x > 0\ else\ x - 1\ if\ x < 0\ else\ 0 \quad \#\ This\ reduces\ readability.$$

Best Practices for Lambda Functions

- Use lambda for **simple expressions only**
- Avoid complex logic
- Prefer readability over brevity
- Use named functions for large logic
- Document when needed

4.5 SPECIAL FUNCTIONAL UTILITIES IN PYTHON (filter(), map(), reduce())

Python supports a functional programming paradigm through several powerful built-in utilities that operate on **iterable data structures**. Among these, filter(), map(), and reduce() are foundational tools for transforming, selecting, and aggregating data efficiently.

These functions enable:

- Declarative programming
- Cleaner code
- Higher-order function usage
- Pipeline-style data processing
- Improved readability for data transformations They are widely used in:
- Data science
- Machine learning pipelines
- Financial systems
- API data processing
- Backend services
- ETL workflows

Functional Programming Overview

Formal Definition: Functional programming is a programming paradigm where computation is treated as the evaluation of mathematical functions and avoids changing state or mutable data.

Key principles:

- Functions as first-class objects
- Immutability
- Higher-order functions
- Expression-based logic

Python supports functional programming alongside procedural and object-oriented paradigms.

THE filter() FUNCTION

Purpose of filter() **filter()** is used for "Filtering out some elements from list of elements by applying to function". **Formal Definition:** filter() returns an iterator containing only those elements for which the function returns True.

Syntax: $varname = filter(functionname,$ iterable_object)

- Here 'varname' is an object of type <class, 'filter'> and we can convert into any iterable object by using type casting functions.
- "functionname" represents either Normal function or anonymous functions.
- "iterable_object" represents Sequence, List, set and dict types.
- The execution process of filter() is that "Each Value of Iterable object sends to Function Name. If the function returns True, then the element will be filtered. if the Function returns False then that element will be neglected/not filtered ". This process will be continued until all elements of Iterable object completed.

Working Mechanism

For each element:

1. The function is applied
2. If result is True → element kept 3. If False → element

discarded **filter() with Normal Function**

$$def\ is\_even(x):$$
$$\quad return\ x\ \%\ 2 == 0$$
$$nums = [1, 2, 3, 4,5, 6]$$
$$result = list(filter(is\_even, nums))$$

Output: [2,4, 6]

filter() with Lambda Function

$$result = list(filter(lambda\ x: x\ \%\ 2 == 0, nums))$$

Limitations of filter()

- Only selects, does not transform
- Requires Boolean condition
- Less readable for complex logic

THE map() FUNCTION

Purpose of map()

- map() is used for obtaining new Iterable object from existing iterable object by applying old iterable elements to the function.
- In other words, map() is used for obtaining new list of elements from existing list of elements by applying old list elements to the function.

Formal Definition: map() applies a function to every element and returns a new iterable of results.

Syntax: $varname = map(Functionname, Iterable\_object)$

- Here 'varname' is an object of type <class, map'> and we can convert into any iterable object by using type casting functions.
- "Functionname" represents either Normal function or anonymous functions.
- "Iterable_object" represents Sequence, List, set and dict types.
- The execution process of map() is that "map() sends every element of iterable object to the specified function, process it and returns the modified value(result) & new list of elements will be obtained". This process will be continued until all elements of Iterable_object completed.

Working Mechanism

For each element:

1. Apply function
2. Store returned value

map() with Normal Function

$$def\ square(x):$$
$$\quad return\ x * x$$
$$nums = [1, 2, 3]$$
$$result = list(map(square, nums))$$

Output: [1, 4, 9]

map() with Lambda Function

$$result = list(map(lambda\ x{:}x\ * x, nums))$$

Multiple Iterable in map()

$$a = [1, 2, 3]$$
$$b = [4, 5, 6]$$
$$result = list(map(lambda\ x, y{:}\ x + y, a, b))$$

Limitations of map()

- Can reduce readability
- Debugging is harder
- Better replaced by list comprehensions sometimes

THE reduce() FUNCTION

Purpose of reduce()

reduce() is used for obtaining a single element / result from given iterable object by applying to a function.

Formal Definition: reduce() applies a function cumulatively to elements of an iterable, reducing it to one value.

Syntax:
$$from\ functools\ import\ reduce$$
$$varname = reduce(function - name, iterable - object)$$

- here varname is an object of int, float, bool, complex, str only •
  The reduce() belongs to a pre-defined module called "functools".

Working mechanism

Step-1:-Initially, reduce() selects First Two values of Iterable object and place them in First var and Second var .

Step-2:-The function-name(lambda or normal function) utilizes the values of First var and Second var and applied to the specified logic and obtains the result.

Step-3:- reduce () places the result of function-name in First variable and reduce()the succeeding element of Iterable object and places in second variable.

Step-4: Repeat Step-2 and Step-3 until all elements completed in Iterable object and returns the result of First Variable.

reduce() with Normal Function

```
from functools import reduce def
add(x, y):
  return x + y
nums = [1, 2, 3, 4]
result = reduce(add, nums)
```

reduce() with Lambda Function

```
result = reduce(lambda x, y: x + y, nums)
```

Limitations of reduce()

- Less readable
- Can be replaced by sum(), max()
- Not beginner-friendly

4.6 ABSTRACTION THROUGH FUNCTIONS

Abstraction is a foundational principle of computer science that enables the design of complex systems through the separation of **interface** and **implementation**. It allows programmers to work with high-level concepts without being burdened by low-level operational details. In Python, abstraction is primarily achieved through **functions**, which serve as the first formal mechanism for hiding complexity.

This section explores abstraction from a **conceptual, architectural, and theoretical perspective**, emphasizing its role in software design, system scalability, and cognitive efficiency.

Conceptual Meaning of Abstraction

Abstraction is derived from the Latin word *abstrahere*, meaning *to draw away*. In computing, abstraction refers to the process of **removing unnecessary details** to focus on essential features.

Formally:

Abstraction is the intellectual process of modelling a system by emphasizing relevant properties while suppressing irrelevant implementation details.

This principle allows developers to:

- Think in terms of *what* a system does
- Rather than *how* it does it

Abstraction as a Cognitive Tool

Human cognition has limited capacity. Managing every low-level detail of a system is not feasible for large-scale software.

Abstraction reduces **cognitive load** by:

- Simplifying mental models
- Enabling layered thinking
- Allowing conceptual decomposition
- Supporting problem solving at different levels By using abstraction, programmers can:
- Focus on problem logic
- Ignore machine-level operations
- Work efficiently in teams

Abstraction in Software Architecture

In software engineering, abstraction is used to create **layered architectures**: 1.

User Interface Layer

2. Business Logic Layer
3. Data Access Layer
4. Hardware / System Layer Each layer:

- Exposes an abstract interface
- Hides internal implementation

Functions operate similarly at the micro-level by abstracting individual tasks.

Interface vs Implementation

Every abstract system consists of two parts:

Component	Description
Interface	What the system exposes
Implementation	How the system works

Functions provide:

- **Interface** → Function name + parameters
- **Implementation** → Function body

Users interact only with the interface.

Information Hiding Principle

Abstraction is closely related to **information hiding**, a principle proposed by David Parnas.

Information hiding means:

- Internal logic should not be exposed
- Only necessary information should be shared Functions enforce this by:
- Restricting access to local variables
- Encapsulating logic inside the function scope

Abstraction and Modularity

Abstraction enables **modular system design**, where:

- Each module performs one responsibility
- Modules communicate via interfaces
- Internal logic remains isolated This improves:
- Maintainability
- Scalability
- Testability
- Fault isolation

Functions are the smallest modular units in Python.

Levels of Abstraction

Abstraction exists at multiple levels:

Level	Example
Hardware	CPU instructions
OS	File systems

Language	Functions
OOP	Classes
Framework	APIs

Chapter 4 focuses on **function-level abstraction**.

Functional Abstraction vs Object Abstraction

Aspect	**Functional**	**Object-Oriented**
Unit	Function	Class
Focus	Action	Entity
Data	External	Encapsulated
Interface	Parameters	Methods

Functional abstraction models **behavior**, Object abstraction models **entities**.

Abstraction and Reusability

Abstraction enables code reuse because:

- Interfaces remain stable
- Implementations can change
- Users remain unaffected This allows:
- Rapid system evolution
- Backward compatibility
- Reduced maintenance cost

Abstraction and Software Evolution

As software evolves:

- Requirements change
- Technologies improve
- Performance needs increase Abstraction ensures:
- Minimal disruption
- Isolated modifications
- Stable user interaction

Functions act as **evolutionary boundaries**.

Abstraction and Testing

Abstraction improves testing because:

- Functions can be tested independently
- Mock interfaces can replace implementations
- Complex logic can be isolated This supports:
- Unit testing
- Integration testing
- Regression testing

Abstraction and Security

Security-sensitive logic should be abstracted to:

- Prevent misuse
- Hide internal mechanisms
- Control access For example:
- Authentication functions
- Encryption functions
- Payment processing functions

Users should never access internal security logic directly.

Abstraction and Team Collaboration

In large teams:

- Developers work on different modules
- Interfaces define responsibilities
- Implementations evolve independently Functions act as **contracts** between team members.

Abstraction and Maintainability

Maintainable software relies on:

- Clear interfaces
- Hidden complexity
- Stable abstractions Functions support this by:
- Limiting scope
- Encapsulating logic
- Reducing dependencies

Risks of Poor Abstraction

Poor abstraction leads to:

- Overexposed logic
- Tight coupling
- Fragile systems
- Difficult maintenance
- Reduced scalability Examples:
- Large functions with mixed responsibilities
- Ambiguous function names
- Excessive parameter usage

Design Principles Supporting Abstraction

Abstraction aligns with:

- Single Responsibility Principle
- Separation of Concerns
- Open/Closed Principle
- Interface Segregation

These principles guide professional software design.

Abstraction in Python Ecosystem

Python libraries heavily rely on abstraction:

- NumPy abstracts array operations
- Pandas abstracts data manipulation
- Flask abstracts web routing
- TensorFlow abstracts ML computation

Users interact with high-level functions, not internal algorithms.

CHAPTER 5 — Object-Oriented Programming Essentials

5. Object-Oriented Programming Essentials

5.1 Introduction to Object-Oriented Programming

Software development has evolved dramatically since the early days of computing. Initially, programs were small, simple, and written for highly specific tasks. These early programs consisted primarily of sequences of instructions that were executed in linear order. As computing needs expanded and systems became more sophisticated, developers began building increasingly complex applications involving graphical interfaces, networking, databases, distributed systems, artificial intelligence, and large-scale simulations.

As program complexity increased, developers encountered a fundamental challenge:

How can large software systems be designed so they remain understandable, maintainable, and extensible?

Without a structured approach, large programs become difficult to manage. Their logic becomes tangled, dependencies multiply, and even minor modifications can cause unexpected failures. Such poorly organized programs are often referred to as spaghetti code, a metaphor describing software whose structure is so intertwined that it is nearly impossible to trace or modify safely.

Object-Oriented Programming (OOP) was developed as a disciplined methodology to address this challenge. It provides a conceptual framework that allows programmers to construct complex systems from smaller, well-defined, reusable components. Instead of writing programs as long sequences of instructions, OOP organizes software as collections of interacting entities called objects. Each object represents a conceptual unit of the system and is responsible for managing its own data and behavior.

Thus, Object-Oriented Programming represents not merely a coding technique, but a **design philosophy** that promotes clarity, modularity, scalability, and maintainability in software development.

Definition of Object-Oriented Programming

Object-Oriented Programming, commonly abbreviated as **OOP**, is a fundamental programming paradigm that structures software systems around the concept of *objects*. Unlike earlier programming methodologies that focus primarily on instructions and procedures, OOP organizes programs as collections of interacting entities that represent real-world objects or conceptual components within a system.

In formal academic terminology:

Object-Oriented Programming is a programming paradigm in which software is designed as a system of interacting objects, where each object encapsulates data, behavior, and identity into a unified unit.

This definition emphasizes that an object is not merely a data container or a function. Instead, it is a self-contained computational entity that combines both information and the operations that manipulate that information.

Conceptual Foundation

To understand OOP intuitively, it is useful to consider how humans perceive and interpret the world. Humans naturally understand complex systems in terms of entities rather than instructions. For example, when thinking about a university system, one does not imagine a sequence of instructions but rather a collection of interacting components such as students, teachers, courses, and classrooms.

Each of these entities has:

- properties describing its state
- actions describing its behaviour

Object-Oriented Programming mirrors this natural cognitive model. Instead of instructing a computer step by step, OOP enables developers to describe a system as a network of interacting objects that resemble real-world entities.

Thus, OOP is not merely a coding technique but a conceptual framework for modelling problems.

Fundamental Properties of an Object

Every object in an object-oriented system possesses three essential characteristics:

1. State

The state of an object refers to the data it stores. This data represents the current condition or status of the object.

Example

A student object may store:

- name
- age
- grade

2. Behaviour

Behaviour refers to the operations that an object can perform. These operations are implemented as methods associated with the object.

Example

A student object may perform actions such as:

- register for course
- calculate grade
- display profile

3. Identity

Identity distinguishes one object from all other objects, even if they contain identical data. Each object occupies a unique location in memory and therefore possesses a distinct identity.

Structural Components of OOP

Object-Oriented Programming is built upon two primary structural components:

Class — The Blueprint

A **class** is a user-defined data type that serves as a template or blueprint for creating objects. It defines:

- attributes (data members)
- methods (functions)

A class specifies what an object should look like and what it should be able to do, but it does not itself represent a real entity.

Logical Nature of a Class

A class has **logical existence only**. It exists as a definition or description and does not allocate memory for instance data until objects are created from it.

Object — The Instance

An **object** is an instance of a class that exists in memory during program execution. When an object is created, memory is allocated to store its data.

Physical Nature of an Object

An object has **physical existence** because it occupies memory and can interact with other objects.

Analogy: Blueprint and Building To clarify the relationship:

Concept	Real-World Analogy
Class	Architectural blueprint
Object	Actual building

The blueprint defines structure, but the building is the real entity.

Real-World Example

Consider designing software for a banking system.

Real-world entities:

- Account
- Customer
- Transaction

In OOP, each of these can be modelled as a class.

Example conceptual class:

$Class$: $Account\ Attributes$:

$account\_number$

$balance\ Methods$:

$deposit()$

$withdraw()$

When a specific account is created:

$Account\ object \rightarrow Account$ #12345

This object now holds actual data such as:

- account_number = 12345
- balance = 5000

The object represents a real account, not just an abstract idea.

Difference Between Procedural and Object-Oriented Definition

In procedural programming:

A program is defined as a sequence of instructions.

In object-oriented programming:
A program is defined as a system of interacting objects.
This difference represents a fundamental shift in programming philosophy.
Comparative Perspective

Procedural Definition	OOP Definition
Program = Steps	Program = System
Focus = Actions	Focus = Entities
Data separate from logic	Data + logic unified

Data and Behaviour Integration

One of the defining characteristics of OOP is that it integrates data and operations into a single unit.
In procedural programming:
$Data \rightarrow separate\ Functions$
$\rightarrow separate$
In OOP:
$Object = Data + Methods$
This integration ensures that operations affecting data remain associated with that data, improving program reliability and preventing accidental misuse.

Formal Characteristics of Object-Oriented Programming

A programming language is considered object-oriented if it supports a set of defining principles. Although simplified descriptions often mention four principles, a comprehensive academic definition includes seven essential characteristics:

1. **Classes** — blueprints for objects
2. **Objects** — runtime instances
3. **Encapsulation** — data protection
4. **Abstraction** — complexity hiding
5. **Inheritance** — reuse of features
6. **Polymorphism** — multiple forms
7. **Message Passing** — communication mechanism

Together, these characteristics define the theoretical foundation of OOP.

Message Passing Concept

In an object-oriented system, objects interact by sending messages to one another. A message is a request for an object to perform a specific operation.
Conceptually:
Object A requests Object B to perform an action
This communication model resembles interactions between real-world entities and allows complex systems to be constructed from independent components.

Message Passing Concept

In an object-oriented system, objects interact by sending messages to one another. A message is a request for an object to perform a specific operation.
Conceptually:
Object A requests Object B to perform an action

This communication model resembles interactions between real-world entities and allows complex systems to be constructed from independent components.

Need for Object-Oriented Programming

The development of Object-Oriented Programming was not accidental; it arose from practical challenges encountered in earlier programming paradigms. As computing systems evolved from small utilities to large-scale applications involving millions of lines of code, traditional procedural approaches proved increasingly inadequate for managing complexity, ensuring maintainability, and supporting collaborative development. Object-Oriented Programming emerged as a systematic solution to these limitations, offering a structured methodology for designing software that remains organized even as it grows and sophistication.

To understand why OOP became necessary, it is essential to analyse the problems that earlier programming techniques faced and examine how object-oriented principles address these issues.

Limitations of Traditional Programming Approaches

Early programming languages primarily followed procedural or function-oriented paradigms. In these approaches, a program is viewed as a sequence of instructions executed in order. While this method works effectively for small programs, it becomes problematic when applied to complex systems.

Major limitations include:

1. Poor Scalability

Procedural programs tend to become difficult to manage as their size increases. When additional features are introduced, existing logic often must be modified extensively, increasing the risk of errors.

2. Code Duplication

Without structured mechanisms for reuse, programmers frequently repeat similar logic in multiple locations. This redundancy leads to:

- increased program size
- higher memory usage
- greater maintenance effort

3. Tight Coupling Between Components

In procedural programming, different parts of the program often depend heavily on one another. A change in one function may require changes in several other functions, making the system fragile and error prone.

4. Difficulty in Maintenance

Large procedural programs can become extremely difficult to maintain because:

- logic is scattered across many functions
- global variables affect multiple modules
- dependencies are unclear

As a result, debugging and updating such programs becomes time-consuming and risky. **5. Lack of Data Security**

In procedural programs, data is frequently stored in global variables accessible to many parts of the program. This unrestricted access increases the possibility of accidental modification or corruption.

6. Limited Modularity

Procedural design does not inherently enforce modular structure. Although functions provide some modularity, they do not combine data and behaviour into a single unit. This separation often leads to disorganized program architecture.

Complexity Growth in Modern Software

Modern software systems differ significantly from early programs. Contemporary applications often include:

- graphical interfaces
- networking capabilities
- database interaction
- concurrent processing
- distributed components

Such systems require a design methodology capable of managing multiple interacting components simultaneously. Procedural programming lacks the structural mechanisms necessary to handle such complexity effectively.

Concept of Software Complexity Management

Software complexity is one of the central challenges in computer science. As systems grow larger, the number of possible interactions between components increases dramatically. Without proper organizational principles, complexity grows faster than developers can manage.

Object-Oriented Programming addresses this problem by introducing structured abstraction layers. These layers allow developers to focus on one part of the system at a time without needing to understand every detail of the entire program.

This ability to manage complexity is one of the primary reasons OOP became widely adopted.

OOP as a Structured Solution

Object-Oriented Programming provides mechanisms that directly address the limitations of procedural programming. These mechanisms are not arbitrary language features; they are carefully designed principles that promote disciplined software construction. Key structural solutions provided by OOP include:

Encapsulation

Encapsulation restricts access to an object's internal data. Instead of allowing unrestricted modification, data is accessed through controlled interfaces. This prevents accidental corruption and improves program safety.

Abstraction

Abstraction allows programmers to hide complex implementation details and expose only essential functionality. This simplifies system design and improves readability.

Modularity

In OOP, a program is divided into independent components called classes. Each class is responsible for a specific task. This separation makes programs easier to understand and maintain.

Reusability

Classes can be reused across multiple programs or projects. This reduces development time and promotes consistency.

Extensibility

Object-oriented systems can be extended easily by creating new classes based on existing ones. This enables systems to evolve without requiring major redesign.

Importance in Professional Software Development

In real-world software development environments, programs are rarely written by a single individual. Instead, teams of developers collaborate on large projects. Without a structured design methodology, collaboration becomes extremely difficult because developers may unintentionally interfere with each other's code.

Object-Oriented Programming supports collaborative development by:

- defining clear interfaces
- separating responsibilities
- isolating modules
- enabling parallel development

For this reason, most large-scale software systems used in industry are built using object-oriented principles.

Real-World Analogy: Organizational Structure

The necessity of OOP can be understood through an organizational analogy.

Imagine a company where every employee performs every task. Such an organization would quickly become chaotic and inefficient. Instead, companies divide responsibilities into departments:

- finance department
- human resources
- marketing
- operations

Each department handles its own tasks while interacting with others when necessary. This structured approach ensures efficiency, clarity, and scalability.

Object-Oriented Programming applies the same principle to software design. Classes act like departments, and objects act like employees performing specialized tasks.

Conceptual Benefits of OOP

The need for OOP can also be understood in terms of conceptual advantages it provides:

Conceptual Benefit	Explanation
Clarity	Programs become easier to understand
Organization	Structure improves readability
Flexibility	Systems adapt easily to changes
Reliability	Encapsulation reduces errors
Maintainability	Modifications remain localized
Reusability	Existing code can be reused

Academic Perspective

From a theoretical standpoint, Object-Oriented Programming represents a higher level of abstraction compared to procedural programming. It allows developers to reason about software systems using conceptual models rather than low-level instructions. This abstraction reduces cognitive load and enables programmers to design systems systematically.

Thus, OOP is not simply a programming technique but a conceptual framework for organizing computational logic.

Industrial Perspective

In industry, OOP is essential because modern software systems must satisfy requirements such as:

- long-term maintainability
- extensibility
- portability
- scalability
- robustness

Procedural programming alone cannot meet these requirements efficiently. Object-oriented design provides the structural foundation necessary to achieve them.

Procedural vs Object-Oriented Paradigm

Programming paradigms define the fundamental style or philosophy according to which programs are structured and executed. They influence how programmers think about problems, how solutions are designed, and how code is organized. Among the many paradigms that exist in computer science, two of the most historically significant and widely studied are **Procedural Programming** and **Object-Oriented Programming**. Understanding the distinction between these paradigms is essential for appreciating the conceptual importance of OOP and recognizing why it has become the dominant methodology for modern software development.

Meaning of a Programming Paradigm

A programming paradigm can be defined as:

> **A systematic approach or model for structuring programs and solving computational problems.**

Different paradigms encourage different methods of thinking about computation. Some paradigms emphasize sequences of instructions, while others emphasize data structures, logic rules, or mathematical functions. Procedural programming and object-oriented programming represent two fundamentally different ways of conceptualizing software systems.

Procedural Programming Paradigm

Procedural programming is one of the earliest and most straightforward programming methodologies. In this paradigm, a program is structured as a sequence of instructions organized into procedures or functions. The focus is primarily on *what operations must be performed* and *in what order they should be executed*.

Core Characteristics

Procedural programming is characterized by:

- Step-by-step execution
- Linear control flow
- Centralized data storage
- Emphasis on algorithms
- Function-based structure

In procedural design, data and functions are typically separate. Functions operate on data that may be stored globally or passed between functions. The programmer's primary concern is constructing the correct sequence of operations that will produce the desired result.

Conceptual Model

The procedural paradigm can be visualized conceptually as:

$$Input \rightarrow Processing\ Steps \rightarrow Output$$

Each step modifies data and passes it to the next step until the final result is obtained.

Example Conceptual Scenario

Suppose a program is required to calculate student grades. In a procedural approach, the program might consist of functions such as:

- read_input()
- calculate_average()
- assign_grade()
- print_result()

The program executes these functions sequentially. The emphasis is on the order of operations rather than the representation of entities such as "Student" or "Course."

Advantages of Procedural Programming

Procedural programming remains useful because it offers:

- simplicity of structure
- ease of implementation for small tasks
- efficient execution for straightforward algorithms
- minimal overhead

For simple programs or tasks with limited complexity, procedural programming is often sufficient and efficient.

Limitations of Procedural Programming

However, procedural programming exhibits significant limitations when applied to large or complex systems:

1. Programs become difficult to maintain as they grow.
2. Modifications may require changes in many functions.
3. Data security is weak due to global variables.
4. Code reuse is limited.
5. Systems lack structural organization.

These limitations motivated the development of alternative paradigms capable of managing complexity more effectively.

Object-Oriented Programming Paradigm

Object-Oriented Programming represents a fundamentally different approach to software design. Instead of organizing programs as sequences of instructions, OOP organizes them as collections of interacting objects that represent entities within the system.

In this paradigm, the primary unit of organization is not the function but the **object**.

Object-Oriented Programming Paradigm

Object-Oriented Programming represents a fundamentally different approach to software design. Instead of organizing programs as sequences of instructions, OOP organizes them as collections of interacting objects that represent entities within the system.

In this paradigm, the primary unit of organization is not the function but the **object**.

Conceptual Model

The object-oriented paradigm can be represented as:

Objects → Interactions → Behaviour → Result

The focus shifts from instructions to entities and their relationships.

Example Conceptual Scenario

Consider the same student grading system described earlier. In an object-oriented approach, the system would be modelled using classes such as:

- Student
- Course
- Grade

Each class would contain both:

- data describing the entity
- methods describing behaviour

Instead of writing a sequence of functions, the programmer designs a system of interacting objects that represent real-world entities.

Advantages of Object-Oriented Programming

Object-oriented programming provides numerous benefits for large-scale software development:

- modular program structure
- improved readability
- easier maintenance
- greater code reuse
- better scalability
- enhanced data protection
- natural real-world modelling

Because of these advantages, OOP is widely used in modern software engineering.

Philosophical Difference Between Paradigms

The difference between procedural and object-oriented paradigms is not merely technical; it is philosophical.

Procedural programming asks:

What steps must be executed to solve this problem?

Object-oriented programming asks:

What entities exist in this problem, and how do they interact?

This distinction represents a shift from algorithm-centred thinking to model-centred thinking.

Structural Comparison

The contrast between the two paradigms can be summarized as follows:

Aspect	**Procedural Programming**	**Object-Oriented Programming**
Primary Unit	Function	Object
Focus	Operations	Entities
Design Style	Linear	Modular
Data Handling	Separate	Encapsulated
Reusability	Limited	Extensive

Scalability	Difficult	Easier
Maintenance	Complex	Simpler

Real-World Analogy

The distinction between paradigms can be illustrated through a real-world analogy.
Imagine constructing a city transportation system.
A procedural approach would describe:

- how vehicles move
- when signals change
- how passengers board

An object-oriented approach would instead model:

- vehicles
- traffic lights
- passengers
- roads

Each entity would have its own properties and behaviours. The system would function through interactions between these entities rather than through a rigid sequence of instructions.

When to Use Each Paradigm

Procedural programming is most appropriate for:

- small programs
- simple algorithms
- mathematical computations
- scripts

Object-oriented programming is more suitable for:

- large applications
- complex systems
- collaborative projects
- scalable software

Thus, the choice of paradigm depends on the nature of the problem being solved.

Hybrid Nature of Modern Languages

Many modern programming languages, including Python, support multiple paradigms simultaneously. Such languages are known as **multi-paradigm languages**. They allow developers to choose the most appropriate approach for each task. Python, for example, supports:

- procedural programming
- object-oriented programming
- functional programming

This flexibility allows programmers to combine paradigms when designing software.

Academic Interpretation

From a theoretical standpoint, procedural programming represents a lower level of abstraction because it focuses on instructions. Object-oriented programming represents a higher level of abstraction because it focuses on system modelling.
Higher abstraction levels generally lead to:

- better conceptual clarity

- improved design
- easier maintenance

For this reason, OOP is often taught after procedural programming in computer science curricula.

Real-World Modeling Concept

One of the most powerful and defining characteristics of Object-Oriented Programming is its ability to model real-world systems directly and intuitively. Unlike earlier programming paradigms that focus primarily on instructions or algorithms, object-oriented design centres on representing realworld entities as software constructs. This capability makes OOP not only a programming methodology but also a conceptual modelling framework that bridges human reasoning and computational logic.

The real-world modelling concept refers to the process of analysing a problem domain, identifying its key entities, and representing those entities as objects within a program. Each object corresponds to a real or conceptual element of the system and contains both data and behaviour related to that element.

Natural Human Thinking and Modelling

Human beings naturally interpret complex systems by breaking them into identifiable components. For example, when describing a hospital, a person typically thinks in terms of:

- doctors
- patients
- nurses
- rooms
- equipment

Rather than describing the hospital as a sequence of actions, we describe it as a collection of interacting entities. Object-Oriented Programming leverages this natural cognitive tendency. Instead of forcing programmers to think like machines, OOP allows them to design software in a way that mirrors how humans perceive the real world.

Thus, OOP aligns software design with natural human reasoning patterns, making programs easier to understand and conceptualize.

Natural Human Thinking and Modelling

Human beings naturally interpret complex systems by breaking them into identifiable components. For example, when describing a hospital, a person typically thinks in terms of:

- doctors
- patients
- nurses
- rooms
- equipment

Rather than describing the hospital as a sequence of actions, we describe it as a collection of interacting entities. Object-Oriented Programming leverages this natural cognitive tendency. Instead of forcing programmers to think like machines, OOP allows them to design software in a way that mirrors how humans perceive the real world.

Thus, OOP aligns software design with natural human reasoning patterns, making programs easier to understand and conceptualize.

Entities, Attributes, and Behaviours

Real-world modelling in OOP is based on identifying three key components of a system:

Entities

Entities are the distinct objects or concepts present in the system.

Example entities in a library system:

- Book
- Member
- Librarian
- Transaction

Attributes

Attributes represent properties or characteristics of an entity.

Example attributes of a Book:

- title
- author
- ISBN
- price

Behaviours

Behaviours represent actions or operations associated with an entity.

Example behaviours of a Book:

- issue
- return
- display details

In OOP, each entity becomes a class, attributes become data members, and behaviors become methods. This direct mapping from real-world concepts to program structure is what makes object-oriented programming intuitive and powerful.

Mapping Real-World Systems to Software

The modelling process typically involves the following steps:

1. Identify the problem domain
2. Determine relevant entities
3. Define attributes for each entity
4. Define behaviours for each entity
5. Establish relationships between entities
6. Implement classes and objects

This systematic approach allows programmers to transform real-world problems into structured software solutions.

Example: Online Shopping System Real-world entities:

- Customer
- Product
- Order
- Payment

Mapping to OOP:

Real Entity	Software Representation
Customer	Customer class

Product	Product class
Order	Order class
Payment	Payment class

Each class contains relevant attributes and methods. Objects created from these classes represent actual customers, products, or orders.

Advantages of Real-World Modelling

The ability to model real-world systems provides several major advantages in software development.

1. Intuitive System Design

Because classes represent real-world entities, developers can design systems using concepts they already understand.

2. Improved Communication

Design discussions become clearer because system components correspond to real-world objects.

3. Easier Maintenance

When software structure mirror's real-world structure, modifications become easier to implement.

4. Better Problem Understanding

Modelling forces developers to analyse the problem domain carefully, leading to more accurate and effective solutions.

Relationships Between Real-World Entities

Real-world systems consist not only of individual entities but also of relationships between those entities. OOP supports modelling such relationships through mechanisms such as:

- association
- aggregation
- composition
- inheritance

For example, in a university system:

- A student enrols in a course
- A course belongs to a department
- A department is part of a university

These relationships can be directly represented in object-oriented design, allowing software systems to reflect real-world structures accurately.

Abstraction in Modelling

Real-world systems are extremely complex. Attempting to represent every detail would make software unnecessarily complicated. Therefore, modelling requires abstraction, which involves selecting only relevant features while ignoring irrelevant ones.

For example, when modelling a car in software:

Relevant attributes:

- speed
- fuel level
- engine status

Irrelevant attributes:

- number of paint scratches
- microscopic surface texture

Abstraction ensures that models remain simple while still representing essential functionality.

Conceptual Example: Banking System Model

Consider designing software for a banking system. Instead of writing a long list of instructions, an object-oriented approach models the system as interacting objects:

Classes:

- Account
- Customer
- Transaction
- Bank

Objects:

- Specific customer accounts
- Individual transactions

Each object performs tasks related to its role. For instance:

- Account object manages balance
- Customer object manages personal data
- Transaction object records activity

The system functions through interaction between these objects rather than through a single monolithic procedure.

Modelling and Software Architecture

Real-world modelling is not limited to small programs; it also plays a central role in large-scale software architecture. Enterprise-level systems are typically designed using object-oriented models that reflect real organizational structures.

For example, an airline reservation system may include classes such as:

- Flight
- Passenger
- Ticket
- Reservation
- Aircraft

Each class corresponds to a real-world component of the airline system. This alignment between software architecture and real-world structure makes the system easier to design, understand, and extend.

Cognitive Benefits of Real-World Modelling

Modelling software after real-world systems reduces cognitive complexity. When programmers work with familiar concepts, they can reason more effectively about system behaviour. This improves:

- comprehension
- debugging ability
- design accuracy
- problem-solving efficiency

Thus, real-world modelling is not only a technical technique but also a cognitive strategy for managing complexity.

Advantages of OOP

Object-Oriented Programming has become the dominant paradigm in modern software development not merely because it introduces new language features, but because it provides substantial structural, conceptual, and practical advantages over earlier programming methodologies. These advantages address fundamental challenges in software engineering such as complexity management, maintainability, scalability, and collaboration.

The strengths of OOP arise directly from its foundational principles, including encapsulation, abstraction, inheritance, polymorphism, and modular design. When applied correctly, these principles transform software systems into organized, reusable, and adaptable architectures. The following subsections examine the major advantages of OOP in detail.

Modularity

One of the most significant advantages of OOP is modularity. In object-oriented design, a program is divided into independent components called classes. Each class is responsible for a specific part of the system's functionality.

This modular organization provides several benefits:

- Improved clarity of program structure
- Separation of responsibilities
- Easier debugging
- Independent development of components

Because each class encapsulates its own data and methods, developers can focus on individual modules without needing to understand the entire system simultaneously.

Reusability

Object-Oriented Programming promotes code reuse through mechanisms such as inheritance and class composition. Once a class is defined, it can be reused in multiple programs without rewriting its logic.

Reusability reduces:

- development time
- programming effort
- duplication of code
- potential for errors

For example, a well-designed class representing a bank account can be reused across multiple banking applications with minimal modification.

Maintainability

Software maintenance involves correcting errors, improving performance, or adding new features after a program has been deployed. OOP enhances maintainability by organizing code into clearly defined modules.

Because classes encapsulate their internal implementation details, modifications within a class generally do not affect other parts of the system, provided the external interface remains consistent. This localized impact of changes makes object-oriented systems significantly easier to maintain compared to large procedural programs.

Scalability

As software systems grow, maintaining structure becomes increasingly difficult. OOP supports scalability by allowing developers to extend existing systems without rewriting them entirely. New functionality can be added by:

- creating new classes
- extending existing classes
- overriding specific behaviours

This ability to build upon existing structures makes object-oriented systems adaptable to evolving requirements.

Encapsulation and Data Security

Encapsulation restricts direct access to an object's internal data. Instead of exposing internal variables, access is controlled through defined methods.

This provides:

- protection against accidental modification
- improved data integrity
- controlled access to sensitive information

Encapsulation enhances system reliability by preventing unauthorized or unintended changes to critical data.

Abstraction and Reduced Complexity

Abstraction allows programmers to hide implementation details while exposing only essential functionality. This reduces cognitive overload and simplifies system interaction.

For example, when using a graphical interface library, the programmer does not need to understand how pixels are rendered on the screen. Instead, the programmer interacts with high-level objects such as windows and buttons.

Abstraction therefore enables developers to work at higher levels of conceptual understanding.

Improved Real-World Representation

OOP allows software systems to closely resemble real-world structures. This natural mapping improves system design because entities in the program correspond directly to entities in the problem domain.

Such alignment between software architecture and real-world systems enhances:

- clarity
- communication
- intuitive design

Enhanced Collaboration

In large software projects, multiple developers often work simultaneously on different components. OOP supports collaborative development through:

- well-defined interfaces
- clear separation of concerns
- modular class structure

Because classes operate independently, different teams can develop different modules without interfering with each other's work.

Reduced Code Redundancy

Through inheritance and polymorphism, OOP eliminates unnecessary repetition of code. Shared behaviours can be defined in base classes and reused in derived classes.
This reduces:

- program size
- redundancy
- maintenance complexity

Flexibility and Extensibility

Object-oriented systems are designed to adapt to change. Because classes can inherit and override behaviours, systems can be modified or extended without restructuring the entire architecture. For example, adding a new type of payment method in an e-commerce system may require creating a new class that extends an existing payment interface, rather than rewriting the entire payment processing logic.

Improved Debugging and Testing

OOP simplifies debugging and testing because individual classes can be tested independently. Unit testing becomes easier when components are modular and self-contained.
Errors can often be traced to specific classes rather than to large blocks of procedural code.

Conceptual Organization of Software

Object-oriented systems are organized around entities rather than instructions. This entity-based structure improves conceptual organization and makes software easier to understand.
Developers can reason about systems in terms of interacting objects rather than complex control flows.

Terminology Overview

A clear understanding of Object-Oriented Programming requires familiarity with its core terminology. These terms form the conceptual vocabulary of OOP and are essential for interpreting class structures, reading documentation, designing systems, and communicating effectively in professional software environments.
This section provides precise definitions and contextual explanations of the fundamental terms that recur throughout object-oriented design.

Class

A **class** is a user-defined blueprint or template used to create objects. It defines the structure and behaviour that objects created from it will possess.
Formally:
A class is a logical construct that groups related data and functions into a single cohesive unit.
A class specifies:

- attributes that define the state
- methods that define behaviour Important characteristics:
- A class has logical existence only.
- It does not occupy memory for instance data until objects are created.
- It defines a new data type in the program.

Conceptually, a class describes *what an object should be* but does not represent a concrete instance.

Object

An **object** is an instance of a class that exists in memory during program execution.
Formally:
An object is a runtime entity that contains state and behaviour as defined by its class. Key properties of an object:

- It occupies memory.
- It contains actual data values.
- It can invoke methods.
- It has a unique identity.

Objects represent specific realizations of a class definition.

Attribute (Data Member)

An **attribute**, also called a data member, represents the data associated with an object.
Attributes define the state of an object. For example, in a Student class, attributes may include:

- name
- roll number
- grade

Attributes can be:

- instance attributes, unique to each object
- class attributes, shared across all objects

Attributes are essential for representing real-world characteristics within software systems.

Method

A **method** is a function defined inside a class that describes behaviour associated with that class.
Formally:
A method is an operation that acts upon the data of an object.
Methods:

- define what an object can do
- access and modify attributes
- implement system logic

Methods ensure that behaviour remains closely tied to the data it operates on, which is a central principle of OOP.

Encapsulation

Encapsulation is the mechanism of bundling data and methods into a single unit and restricting direct access to internal components.
Formally:
Encapsulation is the process of combining data and behaviour into a single entity while controlling access to internal details.
Encapsulation:

- protects data from unintended modification
- enforces controlled interaction
- enhances system reliability

It is one of the foundational principles of OOP.

Abstraction

Abstraction refers to hiding implementation details while exposing only essential functionality.

Formally:
Abstraction is the process of representing essential features without revealing internal complexity.
Through abstraction:

- complexity is reduced
- cognitive load is minimized
- system interaction becomes simplified

Abstraction allows users of a class to focus on what an object does rather than how it does it.

Inheritance

Inheritance is the mechanism by which one class acquires the properties and behaviours of another class.
Formally:
Inheritance allows a new class to reuse and extend the functionality of an existing class. Inheritance promotes:

- code reuse
- hierarchical classification
- extensibility

It models real-world "is-a" relationships, such as a Savings Account being a type of Account.

Polymorphism

Polymorphism means "many forms." It allows a single interface to represent multiple underlying implementations.
Formally:

Polymorphism enables objects of different classes to be treated as objects of a common superclass.

Polymorphism:

- enhances flexibility
- supports dynamic behaviour
- enables interchangeable components

It allows the same method name to behave differently depending on the object that invokes it.

Message Passing

In object-oriented systems, objects communicate by sending messages to one another. A message typically involves:

- the name of the method
- required parameters This communication model:
- promotes modular interaction
- reduces direct dependency
- supports distributed system design

Message passing is fundamental to understanding object collaboration.

Instance

An **instance** is another term for an object created from a class.
When a class is defined, it describes potential instances. When an object is created, it becomes an instance of that class.

Identity

Each object possesses a unique identity that distinguishes it from all other objects, even if their data is identical.

Identity ensures:

- separate memory allocation
- independent existence
- distinct behaviour

Interface

An **interface** represents the publicly accessible methods of a class.
It defines:

- how external components interact with the class
- what services are available

The interface hides implementation details and promotes abstraction.

Constructor

A **constructor** is a special method invoked when an object is created. Its purpose is to initialize the object's attributes.

Destructor

A **destructor** is a method executed when an object is destroyed or removed from memory. It is typically used for cleanup tasks.

5.2 Core Principles of OOP

Before studying the technical details of classes, objects, and their implementation in Python, it is essential to understand that Object-Oriented Programming is not merely a collection of syntax rules or programming constructs. Rather, it is a conceptual framework built upon a set of foundational principles that define how software systems should be structured, organized, and maintained. These principles form the theoretical backbone of object-oriented design. They provide the guidelines that ensure software systems remain modular, scalable, reusable, and logically structured even as they grow in complexity. Without a clear understanding of these core principles, a programmer may be able to write object-oriented code syntactically but will not be able to design objectoriented systems effectively.
In academic literature, these principles are often referred to as the **fundamental pillars of ObjectOriented Programming**. While different sources may group or name them slightly differently, the universally accepted conceptual foundation includes the following:

- Class — the blueprint that defines structure and behaviour
- Object — the runtime instance of a class
- Encapsulation — the bundling of data and methods with controlled access
- Abstraction — the hiding of internal complexity while exposing essential functionality
- Inheritance — the mechanism for reusing and extending existing class features
- Polymorphism — the ability for a single interface to represent multiple behaviours

These principles are not independent features; rather, they work together as an integrated system. Each principle supports and reinforces the others. For example, encapsulation strengthens abstraction, inheritance promotes reusability, and polymorphism enhances flexibility. Understanding how these principles interact is crucial for mastering object-oriented design.
It is also important to recognize that these principles are **design concepts**, not language-specific features. Although programming languages provide syntax to implement them, the principles themselves are universal and apply across object-oriented languages such as Python, Java, C++, and

C#. Therefore, learning these principles develops not only programming skill but also software design thinking.

Class

The concept of a **class** lies at the very foundation of Object-Oriented Programming. It serves as the structural blueprint from which objects are created and defines both the data and behavior that characterize those objects. Without classes, the object-oriented paradigm would not exist, because objects are always created from class definitions. Understanding the nature, purpose, and structure of a class is therefore essential for mastering object-oriented design.

Formal Definition

A class can be formally defined as:

A class is a user-defined data type that encapsulates data members and member functions into a single logical unit.

In simpler terms, a class specifies what attributes an object should possess and what operations it should be able to perform. It describes the structure and behaviour common to all objects of that type.

Conceptual Interpretation

A class is not an actual entity but a description of an entity. It represents a concept rather than a physical instance. When a programmer defines a class, they are defining a model or specification that can later be used to create multiple objects.

Thus, a class can be understood as:

- a template
- a pattern
- a blueprint
- a prototype

All objects created from the same class share the same structure but may contain different data values.

Formal Definition

A class can be formally defined as:

A class is a user-defined data type that encapsulates data members and member functions into a single logical unit.

In simpler terms, a class specifies what attributes an object should possess and what operations it should be able to perform. It describes the structure and behaviour common to all objects of that type.

Conceptual Interpretation

A class is not an actual entity but a description of an entity. It represents a concept rather than a physical instance. When a programmer defines a class, they are defining a model or specification that can later be used to create multiple objects.

Thus, a class can be understood as:

- a template
- a pattern
- a blueprint
- a prototype

All objects created from the same class share the same structure but may contain different data values.

Components of a Class

A class typically consists of two primary components:

1. Data Members (Attributes)

These represent the properties or characteristics of objects. They store information describing the state of an object.

Examples of attributes for a Student class:

- name
- age
- roll number

2. Member Functions (Methods)

These represent the actions or operations that objects of the class can perform.

Examples:

- display details
- calculate grade
- update record

Together, attributes and methods define the complete structure and behaviour of objects created from the class.

Syntax of a Class in Python

In Python, a class is defined using the keyword:

class

General syntax:

```
class ClassName:
    # attributes
    # methods
```

Example:

```
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def display(self):
        print(self.name, self.age)
```

In this example:

- Student is the class
- name and age are attributes
- display() is a method

Class as a User-Defined Data Type

A class creates a new data type defined by the programmer rather than built into the language. Built-in data types such as integers or strings represent basic forms of data, whereas classes allow developers to define complex data structures representing real-world entities. For example, while Python provides built-in types such as:

- int

- float
- str a programmer can define a custom type such as:

Car
Employee
BankAccount

These user-defined types behave like native types but represent more complex structures.

Real-World Analogy

The relationship between a class and its objects can be understood through a real-world analogy. Consider a blueprint used to construct houses. The blueprint specifies:

- number of rooms
- layout
- structure
- design

However, the blueprint itself is not a house. It merely defines how houses should be built. Each house constructed from that blueprint represents an object. Thus:

Blueprint	**Class**
House	Object

Purpose of Classes

Classes serve several important purposes in software design:

- define structure of data
- organize program logic
- promote modular design
- support code reuse
- simplify complex systems

By grouping related data and operations together, classes provide a natural way to represent realworld entities in software.

Advantages of Using Classes

The use of classes provides multiple benefits:

Structured Organization

Classes allow code to be organized into logical sections.

Reusability

Once defined, a class can be used repeatedly to create multiple objects.

Maintainability

Changes to a class automatically apply to all objects created from it.

Abstraction

Classes allow programmers to hide internal details and expose only essential features.

Class as a Design Unit

In object-oriented design, the class is considered the fundamental unit of software architecture. Rather than thinking in terms of functions, object-oriented programmers think in terms of classes that represent system components.

For example, when designing a library management system, a developer might begin by identifying classes such as:

- Book

- Member
- Librarian
- Transaction

Each class represents a distinct concept in the system.

Class Namespace

Every class maintains its own namespace. This means variables and methods defined inside a class belong specifically to that class and do not interfere with those of other classes.
Namespaces prevent naming conflicts and allow different classes to use identical method names without collision.

Relationship Between Class and Object Creation

A class is only a definition until objects are created from it. The process of creating an object from a class is called **instantiation**.
Conceptually:

$$Class \rightarrow Blueprint$$
$$Instantiation \rightarrow Construction$$
$$Object \rightarrow Finished\ Product$$

Without objects, a class remains unused. Without a class, objects cannot be created.

Object

If a class is the blueprint of an entity, then an **object** is the realization of that blueprint. In ObjectOriented Programming, objects represent the fundamental runtime entities through which programs operate. While classes define structure and behaviour conceptually, objects bring those definitions into existence during execution.
Understanding objects is critical because OOP is not merely about defining classes; it is about creating and managing objects that interact to produce meaningful outcomes.

Formal Definition

An object can be formally defined as:
An object is an instance of a class that occupies memory and contains actual values for attributes along with access to the class's methods. An object is therefore:

- a runtime entity
- a concrete realization of a class
- a container of state and behaviour

Conceptual Nature of an Object

An object represents a specific entity in the system. While a class describes a general category, an object represents a particular example within that category.
For example:
Class → Student
Objects → Rahul, Anita, Mohan
Each object represents a distinct student with unique data, even though all are instances of the same class.

Physical Existence

Unlike classes, which have logical existence only, objects have **physical existence** in memory. When an object is created:

- memory is allocated
- attributes are initialized
- identity is assigned

The object remains in memory until it is destroyed or removed by the garbage collector.

Three Fundamental Properties of an Object

Every object possesses three defining characteristics:

1. State

The state of an object refers to the data it holds at a given moment.

Example:

A bank account object may store:

- account number
- balance
- account holder name

The balance may change over time, meaning the object's state is dynamic.

2. Behaviour

Behaviour represents the actions an object can perform.

Example:

The bank account object may perform:

- deposit
- withdraw
- display balance

Behaviour is implemented through methods.

3. Identity

Each object has a unique identity that distinguishes it from all other objects, even if their states are identical.

Two student objects with the same name and age are still separate objects because they occupy different memory locations.

Object Creation in Python

Objects are created through a process called **instantiation**.

General syntax:

$$\boldsymbol{object\_name} = \boldsymbol{ClassName}(\boldsymbol{arguments})$$

Example:

$$\begin{aligned}
&class\ Student:\\
&\quad def\ \boldsymbol{__init__}(\boldsymbol{self}, \boldsymbol{name}):\\
&\quad\quad \boldsymbol{self}.name = \boldsymbol{name}\\
&\boldsymbol{student1} = \boldsymbol{Student}(\text{"}Rahul\text{"})\\
&\boldsymbol{student2} = \boldsymbol{Student}(\text{"}Anita\text{"})
\end{aligned}$$

In this example:

- Student is the class
- student1 and student2 are objects
- Each object has its own state

Memory Allocation Process

When an object is created:

1. Memory space is allocated.
2. The constructor method initializes attributes.
3. A reference to the object is stored in a variable.

The variable does not store the object directly; it stores a reference to the object in memory.
Conceptually:
$student1 \rightarrow reference \rightarrow object\ in\ memory$

Object as a Self-Contained Unit

An object combines:

- Data (attributes)
- Functions (methods)

This integration ensures that behaviour affecting data is encapsulated within the same entity.
In procedural programming:

- Data is separate.
- Functions operate externally. In OOP:
- Data and behaviour belong together.

Multiple Objects from a Single Class

One of the most powerful aspects of OOP is the ability to create multiple objects from a single class definition.
Example:

$\boldsymbol{account1} = \boldsymbol{Account}(101,5000)$
$\boldsymbol{account2} = \boldsymbol{Account}(102,10000)$
$\boldsymbol{account3} = \boldsymbol{Account}(103,7500)$ Each

object:

- Shares the same structure.
- Contains different data.
- Operates independently.

This allows scalable modelling of real-world systems.

Interaction Between Objects

Objects do not function in isolation. They interact with one another through method calls. This interaction is known as **message passing**.
Example:
A Customer object may call a method of an Account object to perform a transaction.
This communication mechanism allows complex systems to be constructed from simple, interacting units.

Object Lifecycle

An object typically goes through the following stages:

1. Creation
2. Initialization
3. Usage
4. Destruction In Python:

- Creation occurs during instantiation.

- Destruction occurs when no references remain and garbage collection removes the object.

Real-World Analogy

Consider a car manufacturing company.
Class → Car design specification
Object → A specific car with registration number
Even if two cars have identical specifications, they are still distinct objects because they exist independently.

Importance of Objects in OOP

Object-Oriented Programming is fundamentally about objects. Classes exist to create objects, and programs execute by manipulating objects.
In fact, in Python:

- numbers are objects
- strings are objects
- lists are objects
- functions are objects

Everything in Python is treated as an object internally.

Conceptual Comparison: Class vs Object

Aspect	**Class**	**Object**
Nature	Logical	Physical
Memory Allocation	No	Yes
Role	Blueprint	Instance
Existence	At design time	At runtime

Design Perspective

In object-oriented design, the programmer first identifies relevant classes. Then, objects are created to represent actual entities within the system. The program operates by manipulating these objects and allowing them to interact.
This design approach transforms software development from writing procedural steps into constructing an ecosystem of interacting entities.

Encapsulation

Encapsulation is one of the fundamental principles of Object-Oriented Programming. It refers to the mechanism of bundling data and the methods that operate on that data into a single unit, typically a class, while restricting direct access to certain components. Encapsulation plays a crucial role in maintaining system integrity, protecting data, and promoting modular software design.
In simple terms, encapsulation ensures that the internal representation of an object is hidden from the outside world and can only be accessed through well-defined interfaces.

Formal Definition

Encapsulation can be formally defined as:
Encapsulation is the process of combining data and the operations that manipulate that data within a single entity while controlling access to the internal state of that entity.
This definition highlights two key aspects:

1. Data and behaviour are bundled together.

2. Access to internal data is controlled or restricted.

Purpose of Encapsulation

The primary objectives of encapsulation are:

- Protect internal data from accidental modification.
- Ensure controlled access through defined methods.
- Prevent external components from depending on implementation details.
- Improve maintainability and reliability of software systems.

Encapsulation enforces disciplined interaction between different parts of a program.

Data Hiding Concept

Encapsulation introduces the concept of **data hiding**. Data hiding means that certain internal variables of a class should not be directly accessible from outside the class. Instead, they should be accessed or modified only through public methods.

For example, consider a bank account. The account balance should not be directly modified by external code. Instead, operations such as deposit or withdrawal should be performed through specific methods.

Without data hiding, any part of the program could arbitrarily change critical data, leading to inconsistencies and errors.

Encapsulation in Python

Python supports encapsulation primarily through:

- Naming conventions
- Access control mechanisms
- Method-based data access

In Python, attributes can be categorized as:

Public Members

Accessible from anywhere in the program.

Protected Members

Indicated by a single underscore prefix. Intended for internal use, though not strictly enforced.

Private Members

Indicated by double underscores. These undergo name mangling to restrict direct access. Example:

```
class Account:
  def __init__(self, balance):
    self.__balance = balance   # private variable
def deposit(self, amount):      self.__balance +=
amount    def get_balance(self):
    return self.__balance
```

In this example:

- *__balance* is hidden from direct external modification.
- Access is provided through controlled methods.

Controlled Access Through Methods

Encapsulation does not mean data becomes inaccessible. Instead, it ensures that access occurs through well-defined interfaces.

Common access methods include:

- Getter methods to retrieve values.
- Setter methods to modify values under controlled conditions.

This ensures validation and consistency.
For example:

$$\textbf{def } set\_balance(self, amount):$$
$$\quad \textbf{if } amount >= 0:$$
$$\quad\quad self.\_balance = amount$$

Such validation would not be possible if the attribute were directly accessible.

Real-World Analogy

Encapsulation can be understood through the example of a medical capsule.
A capsule contains medicine inside a protective outer layer. The internal chemical composition is hidden from the user. The user interacts only with the capsule as a whole.
Similarly, in OOP:

- Internal data is hidden.
- External interaction occurs through defined methods.

Another analogy is a car engine. A driver operates the car using the steering wheel and pedals without needing to understand internal engine mechanisms.

Benefits of Encapsulation

Encapsulation offers numerous advantages in software development.
1. Improved Security
Sensitive data is protected from unauthorized access.
2. Reduced System Complexity
Users of a class do not need to understand its internal implementation.
3. Increased Maintainability
Internal implementation can change without affecting external code.
4. Improved Code Reliability
Validation logic prevents invalid states.
5. Clear Separation of Responsibilities
Each class manages its own data and behaviour.

Encapsulation vs Abstraction

Encapsulation and abstraction are closely related but distinct concepts.
Encapsulation focuses on: •
Protecting data

- Controlling access Abstraction focuses on:
- Hiding complexity
- Exposing essential functionality

Encapsulation is a mechanism that helps achieve abstraction.

Encapsulation and Modular Design

Encapsulation contributes significantly to modular design. When internal data is hidden, modules become independent units that interact only through defined interfaces.
This modularity:

- Reduces coupling between components
- Enhances flexibility
- Simplifies testing and debugging

Encapsulation and Object Integrity

Object integrity refers to maintaining a consistent and valid state for an object. Encapsulation ensures that:

- Only authorized methods can modify data.
- Invalid states are prevented.
- System stability is maintained.

Without encapsulation, object integrity can easily be compromised.

Practical Design Guidelines

When designing classes:

- Avoid exposing internal variables directly.
- Use methods to control access.
- Validate inputs before modifying state.
- Keep internal implementation details hidden.

Following these guidelines improves robustness.

Abstraction

Abstraction is one of the most essential and intellectually profound principles of Object-Oriented Programming. It enables programmers to manage complexity by focusing on relevant features of an object while deliberately hiding unnecessary implementation details. Through abstraction, software systems become easier to understand, easier to design, and easier to maintain.

In object-oriented design, abstraction is not merely a programming technique but a cognitive strategy that allows developers to model complex systems in simplified forms. By exposing only the essential characteristics of an entity and suppressing internal complexity, abstraction allows programmers to work at higher levels of conceptual thinking rather than being overwhelmed by lowlevel details.

Formal Definition

Abstraction can be formally defined as:

Abstraction is the process of representing essential features of an object while hiding the underlying implementation details.

This definition emphasizes two complementary aspects:

1. Essential information is exposed.
2. Irrelevant or complex details are hidden.

Abstraction therefore acts as a filter that allows users to interact with objects through simplified interfaces without needing to understand internal mechanisms.

Conceptual Meaning of Abstraction

Abstraction is based on selective representation. In real-world thinking, humans naturally abstract complex systems by focusing only on what is necessary for a particular purpose.

For example, when driving a car, a driver does not need to understand:

- engine combustion cycles
- transmission mechanics
- fuel injection algorithms

Instead, the driver interacts with simple controls such as:

- steering wheel
- accelerator

- brake

This simplified interface represents abstraction. The driver uses the car effectively without knowing how its internal components work.
Similarly, in programming, abstraction allows users of a class to interact with objects without needing to understand their internal implementation.

Purpose of Abstraction

The primary objectives of abstraction are:

- Reduce complexity
- Improve clarity
- Simplify interaction
- Enhance maintainability
- Promote modular design

Without abstraction, programmers would need to understand every internal detail of every component they use, which would make large software systems nearly impossible to manage.

Abstraction in Object-Oriented Programming

In OOP, abstraction is achieved by designing classes that expose only essential functionality through public methods while hiding internal logic.
A class typically provides:

- public interface for interaction
- hidden internal implementation Example conceptual

structure:

$Class\ Interface \rightarrow Visible\ to\ user$
$Internal\ Logic \rightarrow Hidden\ from\ user$

The user interacts only with the interface.

Interface vs Implementation

Abstraction creates a clear separation between two layers:

- **Interface Layer**

Defines what an object can do.

- **Implementation Layer**

Defines how the object performs those actions.
This separation provides flexibility. Internal implementation can change without affecting external code, as long as the interface remains unchanged.

Real-World Analogy

Consider an ATM machine. A user interacts with it using simple operations:

- insert card
- enter PIN
- withdraw money

The user does not need to understand:

- database queries
- encryption algorithms
- transaction validation logic

The ATM provides an abstract interface that hides complex internal processes. In the same way, classes provide abstract interfaces to users of objects.

Levels of Abstraction

Abstraction can occur at multiple levels in software design:

- **Data Abstraction**

Hiding the representation of data while exposing operations on that data.

- **Procedural Abstraction**

Hiding algorithmic details inside functions or methods.

- **System-Level Abstraction**

Representing complex systems as interacting components rather than detailed instructions. Higher levels of abstraction allow programmers to reason about systems conceptually rather than mechanically.

Abstraction in Python

Python supports abstraction through:

- classes
- abstract base classes
- method interfaces
- encapsulation

Example conceptual abstraction:

```
class Shape:
    def area(self):
        pass
```

This defines what every shape must do, but not how it does it. Specific shapes implement their own logic.

Advantages of Abstraction

Abstraction offers several important benefits in software engineering.

1. **Reduced Complexity:** Developers focus only on relevant information.
2. **Increased Readability:** Code becomes easier to understand.
3. **Flexibility:** Implementation can change without affecting usage.
4. **Maintainability:** Systems can be modified without rewriting dependent code.
5. **Scalability:** Large systems can be built from abstract components.

Abstraction vs Encapsulation

Although abstraction and encapsulation are related, they serve different purposes.

Concept	Focus
Encapsulation	Protecting data
Abstraction	Simplifying complexity

Encapsulation hides data.
Abstraction hides complexity.
Encapsulation is a mechanism.
Abstraction is a design principle.
Encapsulation often supports abstraction by restricting access to internal details.

Role in Software Architecture

Abstraction plays a central role in software architecture. Complex systems are typically designed in layers, where each layer abstracts the functionality of the layer below it. Example layered structure:

- User Interface Layer
- Application Logic Layer
- Database Layer

Each layer interacts with others through abstract interfaces rather than direct internal access.

Cognitive Importance

From a cognitive perspective, abstraction is essential because human working memory is limited. By hiding unnecessary detail, abstraction reduces mental load and allows programmers to reason about systems more effectively.
Thus, abstraction is not only a programming principle but also a mental strategy for managing complexity.

Inheritance

Inheritance is one of the central pillars of Object-Oriented Programming. It provides a mechanism through which a new class can acquire the properties and behaviours of an existing class. By enabling reuse and extension of previously defined structures, inheritance promotes modularity, reduces redundancy, and supports hierarchical classification of entities.
Inheritance models real-world relationships in which one entity is a specialized version of another. Through this mechanism, object-oriented systems can represent "is-a" relationships, allowing structured organization of related classes.

Formal Definition

Inheritance can be formally defined as:
Inheritance is the mechanism by which a new class derives attributes and methods from an existing class, enabling code reuse and hierarchical classification.
In this context:

- The existing class is called the **base class**, **parent class**, or **superclass**.
- The new class is called the **derived class**, **child class**, or **subclass**.

Conceptual Meaning

Inheritance allows one class to build upon another. Instead of redefining common functionality repeatedly, a subclass automatically acquires the characteristics of its parent class and can extend or modify them as needed.
For example:

- A Vehicle class may define attributes such as speed and fuel.
- A Car class may inherit from Vehicle and add attributes like number_of_doors.

Here, a Car "is a" Vehicle, meaning it inherits general vehicle properties while introducing specific characteristics.

Purpose of Inheritance

The primary purposes of inheritance include:

- Code reusability
- Hierarchical classification
- Reduction of redundancy
- Improved maintainability
- Logical relationship modelling

Inheritance prevents duplication of common features and promotes systematic organization.

Real-World Analogy

Consider biological classification. A Mammal class may include characteristics such as:

- warm-blooded
- gives birth
- has hair

A Dog class inherits from Mammal and adds specific traits such as:

- breed
- bark

The Dog automatically possesses all characteristics of Mammal while having additional specialized behaviour.

Thus:

$$Dog \rightarrow is\ a \rightarrow Mammal$$

$$Mammal \rightarrow is\ a \rightarrow Animal$$

This hierarchical relationship mirrors inheritance in OOP.

Syntax of Inheritance in Python

In Python, inheritance is defined using parentheses after the class name.

General syntax:

$$\textbf{\textit{class ChildClass}}(ParentClass):$$

$$\textbf{\textit{pass}}$$

Example:

$$\textbf{\textit{class Vehicle}}:$$

$$\quad \textbf{\textit{def}}\ start(self):$$

$$\quad\quad print("Vehicle\ started")$$

$$\textbf{\textit{class Car}}(Vehicle):$$

$$\quad \textbf{\textit{def}}\ drive(self):$$

$$\quad\quad print("Car\ is\ driving")$$

In this example:

- Car inherits the start() method from Vehicle.
- Car can also define additional methods such as drive().

Types of Inheritance

Object-Oriented Programming supports several types of inheritance structures.

1. Single Inheritance: One child class inherits from one parent class.

2. Multilevel Inheritance: A class inherits from a child class, forming a chain.

3. Hierarchical Inheritance: Multiple child classes inherit from the same parent class.

4. Multiple Inheritance: A class inherits from more than one parent class.

Python supports all these types.

Inheritance and Code Reusability

Inheritance significantly reduces code duplication. Instead of rewriting common functionality in multiple classes, a base class can define shared methods once, and derived classes automatically reuse them. This promotes:

- consistency
- maintainability
- efficiency

If changes are needed in shared functionality, modifying the base class updates all derived classes automatically.

Overriding in Inheritance

Inheritance allows subclasses to override parent class methods. This means a child class can provide a different implementation of a method defined in the parent class.
Example:

```
class Animal:
  def speak(self):
    print("Animal sound")
class Dog(Animal):
def speak(self):
    print("Bark")
```

The Dog class overrides the speak() method.
Overriding enables customization while preserving structural hierarchy.

The super() Function

In Python, the super() function allows a subclass to access methods of its parent class.
Example:

```
class Car(Vehicle):
def start(self):
    super().start()
    print("Car ready")
```

The super() function ensures proper invocation of parent class methods.

Advantages of Inheritance

Inheritance provides several benefits:

1. **Code Reuse:** Common logic is written once.
2. **Logical Hierarchy:** Classes reflect real-world relationships.
3. **Extensibility:** Systems can grow without rewriting existing code.
4. **Maintainability:** Shared behaviour can be modified centrally.

Potential Drawbacks

While inheritance is powerful, improper use can create:

- overly complex hierarchies
- tight coupling
- difficult debugging

Therefore, inheritance should be applied thoughtfully.

Inheritance vs Composition

Inheritance models “is-a” relationships.
Composition models “has-a” relationships.
Example:

- A Car is a Vehicle → Inheritance.
- A Car has an Engine → Composition.

Understanding this distinction is important in object-oriented design.

Polymorphism

Polymorphism is one of the most powerful and flexible principles of Object-Oriented Programming. The term originates from Greek words meaning "many forms." In the context of programming, polymorphism refers to the ability of a single interface to represent multiple underlying implementations.

Polymorphism allows objects of different classes to respond to the same method call in different ways. It promotes flexibility, extensibility, and loose coupling within software systems. Through polymorphism, object-oriented programs achieve dynamic behaviour while maintaining a unified structure.

Formal Definition

Polymorphism can be formally defined as:

> ***Polymorphism is the ability of different objects to respond to the same method name or interface in different ways.***

This means that a single method call can produce different behaviours depending on the object invoking it.

Conceptual Meaning

Polymorphism enables one interface to control multiple behaviours. Instead of writing separate code for each specific type, a common interface is used, and each object determines its own behaviour.

For example, consider a drawing application with different shapes:

- Circle
- Rectangle
- Triangle

Each shape has a method called draw(). Although the method name is the same, each shape draws itself differently.

The user of the system does not need to know which specific shape is being drawn. The same draw() call works for all shapes.

Real-World Analogy

Consider the concept of a remote control.

A remote control may operate different devices:

- Television
- Air conditioner
- Speaker system

The same button labelled "Power" performs different actions depending on the device. This is polymorphism in action: one interface, multiple behaviours.

Another example is a person speaking in different languages. The action "speak" exists universally, but its form varies.

Types of Polymorphism

Polymorphism can generally be categorized into two types:

1. Compile-Time Polymorphism:

Also known as static polymorphism. It occurs when method resolution happens during compilation. Examples include method overloading and operator overloading.

2. Runtime Polymorphism:

Also known as dynamic polymorphism. It occurs when method resolution happens during execution. It is achieved through method overriding and inheritance.

Python primarily supports runtime polymorphism. **Polymorphism**

Through Method Overriding

Method overriding occurs when a subclass provides a specific implementation of a method already defined in its parent class.
Example:

```
class Animal:
  def speak(self):
    print("Animal sound")
class Dog(Animal):
def speak(self):
    print("Bark")

class Cat(Animal):
def speak(self):
    print("Meow")
```

Here:

- The speak() method exists in all classes.
- Each class defines its own behaviour.

Calling **speak()** on different objects produces different results.

Polymorphism Through Duck Typing

Python supports a flexible form of polymorphism known as duck typing.
The principle is:
If an object behaves like a duck, it is treated as a duck.
Example:

```
class Bird:
  def fly(self):
    print("Flying")

class Airplane:
  def fly(self):
    print("Airplane flying")
def make_it_fly(entity):
  entity.fly()
```

Both Bird and Airplane objects can be passed to **make_it_fly()**, because they both implement a **fly()** method.
This demonstrates interface-based polymorphism.

Operator Overloading

Operator overloading is another form of polymorphism where operators behave differently based on operand types.
Example:

- performs addition for integers.
- performs concatenation for strings.

This is polymorphism at the operator level.

Purpose of Polymorphism

The primary purposes of polymorphism include:

- Increasing flexibility
- Reducing conditional logic
- Promoting extensibility
- Supporting open-closed principle
- Enhancing code readability

Instead of writing multiple conditional statements to handle different types, polymorphism delegates behaviour to objects themselves.

Advantages of Polymorphism

Polymorphism offers several advantages:

1. **Flexibility:** New classes can be introduced without modifying existing code.
2. **Maintainability:** Behaviour changes can be localized within specific classes.
3. **Reduced Complexity:** Eliminates extensive conditional branching.
4. **Scalability:** Systems can grow by adding new subclasses.

Polymorphism and Inheritance

Polymorphism often works in combination with inheritance. A base class defines a general interface, and subclasses provide specific implementations.

For example:

$$\textbf{\textit{def}}\ process\_shape(shape):$$
$$shape.draw()$$

Any subclass of Shape that implements **draw()** can be passed to this function. Thus, polymorphism leverages inheritance to achieve dynamic behaviour.

Polymorphism and Abstraction

Polymorphism supports abstraction by allowing users to interact with objects through generalized interfaces without needing to know their specific types.

For example, calling a payment process method does not require knowledge of whether the payment is via credit card, debit card, or digital wallet.

Conceptual Comparison

Principle	**Focus**
Encapsulation	Protect data
Abstraction	Hide complexity
Inheritance	Reuse structure
Polymorphism	Flexible behaviour

Polymorphism ensures that objects sharing a common interface can behave differently while maintaining structural unity.

Design Perspective

From a software architecture standpoint, polymorphism promotes loose coupling. Components interact through interfaces rather than specific implementations. This reduces dependency and increases system adaptability.

Polymorphism aligns with modern design principles such as:

- Open-Closed Principle
- Interface Segregation
- Dependency Inversion

Relationships Between Principles

While each core principle of Object-Oriented Programming can be studied independently, their true power emerges only when they are understood as an integrated system. Classes, objects, encapsulation, abstraction, inheritance, and polymorphism do not operate in isolation. They function together as interdependent design mechanisms that collectively define the object-oriented paradigm. This section examines how these principles interact, reinforce one another, and form a cohesive architectural framework for modern software systems.

Conceptual Integration of Principles

The core principles of OOP are often presented as separate pillars. However, they are better understood as components of a single unified model.

At a high level:

- **Class and Object** provide structural foundation.
- **Encapsulation and Abstraction** provide protection and simplification.
- **Inheritance and Polymorphism** provide extensibility and flexibility.

Together, they create systems that are modular, scalable, and maintainable.

Class and Object as Foundational Units

All other principles depend upon the existence of classes and objects.

- Encapsulation occurs within classes.
- Abstraction is implemented through class interfaces.
- Inheritance extends classes.
- Polymorphism operates on objects derived from classes.

Without classes and objects, the remaining principles cannot exist. Thus, class and object form the structural base of OOP.

Encapsulation and Abstraction: Complementary Mechanisms

Encapsulation and abstraction are closely related but serve distinct purposes.

Encapsulation:

- Protects internal data.
- Controls access to object state.
- Enforces integrity. Abstraction:
- Hides complexity.
- Exposes essential features.
- Simplifies interaction.

Encapsulation provides the mechanism that makes abstraction possible. By restricting access to internal components, abstraction can present a clean, simplified interface to users.

Thus:

Encapsulation → Enables → Abstraction

Inheritance and Polymorphism: Extensibility Pair

Inheritance and polymorphism work together to create extensible systems.
Inheritance:

- Establishes hierarchical relationships.
- Reuse's structure and behaviour.
- Promotes code sharing.

Polymorphism:

- Allows multiple implementations of a common interface.
- Enables dynamic behaviour.
- Promotes flexibility.

Inheritance provides structural relationships, while polymorphism enables behavioural diversity within that structure.
Thus:
Inheritance → Provides Structure
Polymorphism → Provides Behavioural Flexibility
Together, they allow new classes to extend existing ones without braking system integrity.

Hierarchical Relationship Between Principles

The principles can be visualized conceptually in hierarchical form:

1. Classes define structure.
2. Objects instantiate that structure.
3. Encapsulation protects object state.
4. Abstraction simplifies object interaction.
5. Inheritance extends structure.
6. Polymorphism diversifies behaviour.

Each principle builds upon the previous ones.

Interaction in a Real System

Consider a payment processing system.
Base class:

- Payment

Derived classes:

- CreditCardPayment
- DebitCardPayment
- DigitalWalletPayment Encapsulation:
- Protects transaction details.

Abstraction:

- Exposes a **process_payment()** method. Inheritance:
- Derived classes reuse base structure.

Polymorphism:

- Each subclass implements **process_payment()** differently.

Here, all principles operate together seamlessly.

Complementary Nature of Principles

These principles reinforce each other rather than compete.

- Encapsulation strengthens abstraction.
- Abstraction simplifies polymorphic use.
- Inheritance enables polymorphism.
- Polymorphism reduces dependency on specific implementations.
- Classes provide the framework within which all principles operate.

This interdependence creates cohesive design.

System Design Perspective

From a software architecture viewpoint:
Encapsulation reduces coupling.
Abstraction reduces complexity.
Inheritance promotes reuse.
Polymorphism increases flexibility. Combined,
they achieve:

- Modular design
- Extensible architecture
- Stable system behaviour
- Adaptability to change

Alignment with Design Principles

The integration of OOP principles aligns with broader software engineering guidelines such as:

- Open-Closed Principle
- Single Responsibility Principle
- Liskov Substitution Principle

Polymorphism supports substitution.
Encapsulation supports responsibility separation.
Inheritance supports extension without modification.

Conceptual Dependency Map

The principles can be conceptualized as follows:
Class → Object
Encapsulation → Protects Object
Abstraction → Simplifies Object Interface
Inheritance → Extends Class
Polymorphism → Enhances Object Behaviour
Each arrow represents dependency and reinforcement.

Balanced Application

Effective object-oriented design requires balanced application of these principles.
Excessive inheritance may create rigid hierarchies.
Insufficient encapsulation may compromise integrity.
Improper abstraction may oversimplify essential details. Therefore,
thoughtful integration is crucial.

Unified Perspective

Object-Oriented Programming is best understood not as a collection of isolated techniques but as a unified conceptual ecosystem.

- Classes define structure.

- Objects bring structure to life.
- Encapsulation protects integrity.
- Abstraction reduces cognitive load.
- Inheritance promotes reuse.
- Polymorphism enables flexibility.

Together, they transform software into structured, interacting systems.

5.3 Classes

In the previous section, we examined the theoretical foundation of Object-Oriented Programming by exploring its core principles. We discussed how classes and objects form the structural base of OOP, and how encapsulation, abstraction, inheritance, and polymorphism collectively enable modular and scalable software design.

However, understanding the conceptual meaning of a class is only the first step. To develop mastery in object-oriented programming, it is necessary to examine classes from a structural and internal perspective. This section transitions from theoretical understanding to detailed structural analysis.

A class in Python is more than a blueprint. It is a runtime construct, a namespace container, a memory structure, and even an object itself. Python treats classes as first-class objects, meaning that they can be created dynamically, passed as arguments, stored in variables, and inspected during execution. Therefore, a deeper understanding of classes requires examination of:

- How classes are defined syntactically
- How they are structured internally
- How namespaces are organized
- How attributes and methods differ
- How memory is allocated and managed

This section will analyse the anatomy of a class in detail, including its definition, internal structure, namespace behaviour, and memory model. The goal is to move beyond surface-level syntax and develop an architectural understanding of how Python implements object-oriented structures. By the end of this section, the reader will understand not only what a class is conceptually, but also how it behaves internally within the Python runtime environment.

Definition of Class

In Section 5.1, a class was introduced as a conceptual blueprint for creating objects. While that definition is correct from a theoretical standpoint, it is insufficient for deep technical understanding. In this section, we will redefine the class from structural, runtime, and memory perspectives to fully appreciate its role in Python's object model.

A class is not merely a syntactic construct. It is a runtime entity, a namespace container, a user-defined type, and even an object itself within Python.

Formal Technical Definition

A class can be formally defined as:

A class is a user-defined data type that encapsulates attributes and methods within a distinct namespace and serves as a blueprint for creating objects.

This definition highlights four essential properties:

1. It defines a new data type.
2. It groups related attributes and methods.
3. It creates a separate namespace.
4. It enables object instantiation.

Class as a User-Defined Data Type

In Python, built-in data types include:

- int
- float
- str
- list
- dict

When a programmer defines a class, they create a new data type.

For example: class

Student: pass

Here, Student becomes a new type. Just as int represents integer values, Student represents student entities.

We can verify this concept:

$print(type(Student))$

Output:

$< class\ 'type' >$

This demonstrates a critical fact:

A class itself is an object of type ***type****.*

Thus, classes are first-class objects in Python.

Logical vs Runtime Perspective

A class exists in two perspectives:

1. **Logical Perspective:** From a design standpoint, a class defines structure and behaviour.
2. **Runtime Perspective:** When the Python interpreter executes a class definition:
    a. A class object is created.
    b. A namespace dictionary is constructed.
    c. The class name is bound to that object.

Thus, defining a class creates an object in memory representing that class.

Class as a Namespace Container

Every class maintains its own namespace.

A namespace is a mapping between names and objects.

When you define attributes and methods inside a class, they are stored in the class's namespace dictionary.

Example:

$\boldsymbol{class\ Sample}$:
  $x = 10$
  $\boldsymbol{def}\ display(self)$:
    $print("Hello")$

Internally, Python stores these inside:

$Sample.\_\_dict\_\_$

This dictionary contains:

- x
- display
- other internal attributes

Thus, a class is fundamentally a namespace object.

Class Object Creation Process

When Python encounters a class definition, it performs the following steps:

1. Creates a temporary namespace dictionary.
2. Executes the class body inside that namespace.
3. Creates a class object using type().
4. Assigns the class name to the created class object.

Conceptually:

class definition → namespace creation → class object creation → name binding

Therefore, a class definition is executable code.

Class vs Object Distinction Revisited

It is important to refine the earlier distinction:

Concept	Description
Class	A blueprint and type definition
Object	Instance created from the class

However, from Python's internal model:

3. A class is also an object.
4. Instances reference their class.
5. The class maintains shared attributes.

This layered structure makes Python highly dynamic.

Class as a Template

The traditional definition describes a class as a blueprint.

This remains valid because:

- It defines attribute structure.
- It defines method behaviour.
- It allows creation of multiple objects.

Example:

$$\boldsymbol{class\ Car}:$$
$$\quad \boldsymbol{def}\ \_init\_(self, brand):$$
$$\quad\quad self.brand = brand$$

Here, **Car** defines the template for car objects.

Class as a Behavioural Contract

A class also defines a behavioural contract.

By defining methods, a class specifies:

- What operations objects can perform.
- How external components may interact with them.

This contract ensures consistent behaviour across instances.

Class as a Structural Abstraction

A class abstracts real-world entities into computational models.

For example:

Real World → Bank Account

Software Model → Account class Attributes

represent state.

Methods represent actions.

Thus, a class bridges real-world concepts and software implementation.

Class as a Factory

A class acts as a factory for creating objects.
When a class is called:

$$obj = ClassName()$$

Python:

1. Allocates memory.
2. Initializes the object.
3. Returns the reference.

Thus, classes manufacture instances.

Key Properties of a Class

A class has the following technical properties:

1. It is an object.
2. It is created at runtime.
3. It contains a namespace dictionary.
4. It defines instance structure.
5. It supports inheritance.
6. It supports method binding.
7. It can be passed as an argument.

Syntax of Class Creation

After understanding the conceptual and structural meaning of a class, the next step is to study how classes are defined syntactically in Python. The syntax of class creation specifies how programmers declare a new class, define its attributes, and implement methods that describe its behavior. Python provides a clean and readable syntax for defining classes. Because Python is designed to emphasize simplicity and clarity, the syntax for class creation closely resembles natural language constructs while still supporting powerful object-oriented features.
This section explains the complete syntax of class creation, including naming conventions, structural rules, indentation requirements, and internal components.

Basic Syntax of Class Definition

In Python, a class is defined using the keyword: ***class***
General syntax:

class ClassName:
    # class body

Explanation:

- class is the keyword used to declare a class.
- ***ClassName*** is the identifier representing the name of the class.
- The colon : indicates the start of the class body.
- The class body contains attributes and methods.

Example:

class Student:
    pass

In this example:

- Student is the name of the class.

- pass is used as a placeholder when the class body contains no statements.

Naming Conventions for Classes

Although Python does not strictly enforce naming rules beyond standard identifier requirements, certain conventions are widely followed to improve readability and maintain consistency.
Common guidelines include:

- Class names typically follow **PascalCase**.
- Each word in the class name begins with a capital letter.
- Class names should represent nouns that describe entities.

Examples of valid class names:

$Student$
$BankAccount$
$EmployeeRecord$
$VehicleManager$

Examples of poor naming practices:

$studentdata$
$data123\ temp$

Following consistent naming conventions improves code clarity and maintainability.

Class Body Structure

The class body contains the internal definition of the class. It may include several elements such as:

- class attributes
- instance attributes
- methods
- constructors
- docstrings Example:

$\boldsymbol{class\ Student}$:
$school = "Central\ School"$
$\boldsymbol{def}\ \_init\_(self, name)$:
$self.name = name$ $\boldsymbol{def}$
$display(self)$:
$print(self.name)$

In this structure:

- school is a class attribute.
- __init__() is the constructor.
- display() is an instance method.

The Role of Indentation

Python uses indentation rather than braces or keywords to define code blocks. The class body must therefore be properly indented.
Example:
Correct syntax:

$\boldsymbol{class\ Car}$:
$\boldsymbol{def}\ start(self)$:
$print("Car\ started")$

Incorrect syntax:

```
class Car:
def start(self):
print("Car started")
```

Improper indentation leads to syntax errors.
Thus, indentation is essential for defining the boundaries of the class body.

The pass Statement

Sometimes a programmer may wish to create an empty class as a placeholder. In such cases, Python requires at least one statement inside the class body.
The pass statement serves this purpose.
Example:

```
class EmptyClass:
  pass
```

The pass statement performs no operation but satisfies the syntactic requirement that a class body cannot be empty.

Class with Attributes

Attributes represent data associated with the class.
Example:

```
class Laptop:
  brand = "Dell"
  price = 50000 Here:
```

- brand and price are attributes defined at class level.
- These attributes belong to the class itself.

Class with Methods

Methods represent functions defined within the class.
Example:

```
class Calculator:
  def add(self, a, b):
    return a + b
```

Explanation:

- add() is a method of the class.
- self represents the instance calling the method.

Methods define the behaviour associated with class objects.

Class with Constructor

A constructor is a special method used to initialize object attributes during object creation. In Python, the constructor method is named:

```
__init__
```

Example:

```
class Person:
  def __init__(self, name):
self. name = name     def
greet(self):
    print("Hello",self. name)
```

When an object is created:

$p = Person("Rahul")$

The constructor automatically initializes the attribute name.

Class with Multiple Methods

A class may contain multiple methods to represent different behaviours.

Example:

```
class BankAccount:
    def __init__(self, balance):
        self.balance = balance
    def deposit(self, amount):
    self.balance += amount
    def display(self):
        print("Balance:", self.balance)
```

Here:

- deposit() modifies the account balance.
- display() prints the current balance.

Execution of Class Definition

It is important to understand that a class definition is executable code.

When Python executes a class definition:

1. A namespace dictionary is created.
2. The class body is executed.
3. A class object is created.
4. The class name is bound to that object.

Thus, the class definition is processed during program execution rather than during compilation.

Common Syntax Errors

Several mistakes frequently occur when defining classes.

Missing Colon Incorrect:

```
class Student
```

Correct:

```
class Student:
```

Improper Indentation

Incorrect indentation breaks class structure.

Missing self Parameter Example:

Incorrect:

```
def show():
  print("Hello")
```

Correct:

```
def show(self):
  print("Hello")
```

The self-parameter allows methods to access instance attributes.

Extended Syntax with Inheritance

Classes can also inherit from other classes.

General syntax:

class ChildClass(*ParentClass*):
pass

Example:

class Animal:
  pass
class Dog(*Animal*):
  pass

This syntax establishes a parent-child relationship between classes.

Class Structure

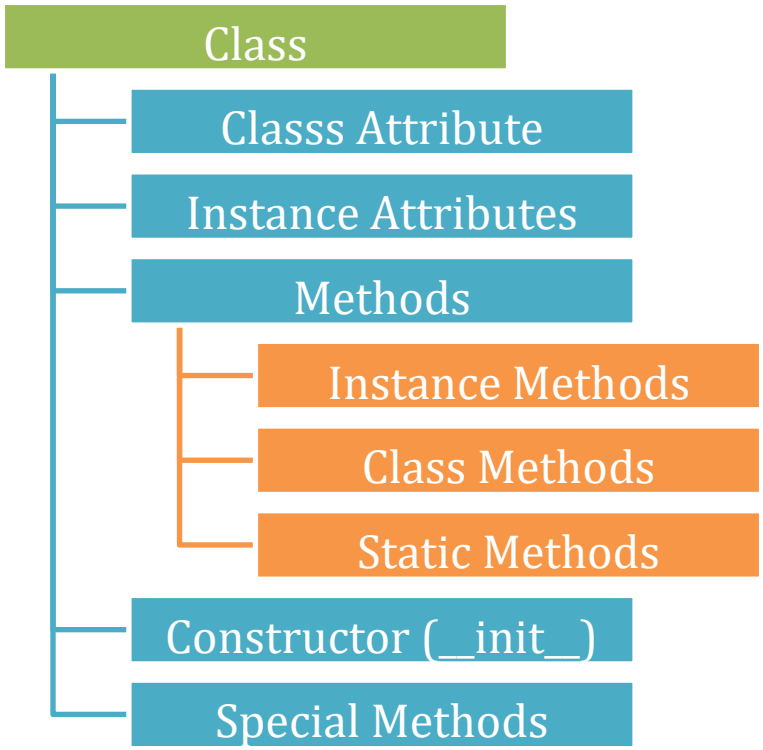
After learning the syntax of class creation, the next step is to understand the **internal structure of a class**. The class structure defines how data and behavior are organized inside a class. Understanding this structure is essential for designing well-organized object-oriented programs. A class is not simply a container of functions. It is a structured unit composed of multiple components that together define the characteristics and operations of objects created from it. In general, the internal structure of a class contains the following elements:

1. Class Attributes (Class-Level Data Members)
2. Instance Attributes (Object-Level Data Members)
3. Methods (Functions inside the class)
4. Constructor
5. Destructor (optional)
6. Special Methods

Each of these components plays a specific role in defining the behaviour and state of objects.

Conceptual Layout of a Class

The structure of a class can be conceptually represented as follows:



This layered structure organizes data and behaviour logically.

Class Attributes

Class attributes are variables defined directly inside the class but outside any method. They represent **properties that are shared by all objects of the class**.

Example:

```
class Student:
    school_name = "Central School"
```

In this example:

- school_name is a class attribute.
- All objects of the Student class share the same value.

Characteristics of class attributes:

- Stored in class namespace
- Shared among all instances
- Accessed using class name or object Example access:

```
print(Student.school_name)
```

Instance Attributes

Instance attributes are variables that belong to a specific object rather than the class itself. They are usually defined inside the constructor using the self reference.

Example:

```
class Student:
    def __init__(self, name):
        self.name = name
```

Here:

- name is an instance attribute.

- Each object can have a different value. Example:

$student1.name \rightarrow Rahul$

$student2.name \rightarrow Anita$

Characteristics:

- Stored in object namespace
- Unique for each object
- Defined using self

Methods

Methods are functions defined inside a class. They describe the **behaviour** of objects.

Example:

```
class Student:
    def display(self):
        print("Student Name")
```

Methods operate on object data and perform actions. Types of methods include:

- Instance methods
- Class methods
- Static methods

Instance Methods

Instance methods operate on object-specific data.

Example:

```
class Student:
    def show(self):
        print("Hello")
```

Here self refers to the current object.

Class Methods

Class methods operate on class-level data rather than object data. They use the decorator:

```
@classmethod
```

Example:

```
class Student:
    school = "Central School"
    @classmethod
    def get_school(cls):
        return cls.school
```

Here cls refers to the class itself.

Static Methods

Static methods do not depend on instance data or class data.

They use the decorator:

@staticmethod Example:

```
class MathTools:
@staticmethod    def
add(a, b):       return
a + b
```

Static methods behave like normal functions but are grouped inside the class.

Constructor

The constructor is a special method used to initialize objects when they are created. In Python, the constructor method is:

```
__init__()
```

Example:

```
class Employee:
  def __init__(self, name, salary):
    self.name = name
    self.salary = salary
```

When an object is created:

```
emp = Employee("Rahul",50000)
```

The constructor automatically initializes attributes.

Key properties of constructors:

- Executed automatically during object creation
- Initializes instance attributes
- Helps establish initial object state

Destructor

A destructor is a special method that executes when an object is destroyed. In Python, the destructor is:

```
__del__()
```

Example:

```
class Test:
  def __del__(self):
    print("Object destroyed")
```

The destructor runs when the object is removed from memory.

However, Python relies on **garbage collection**, so destructors are rarely used directly.

Special Methods

Python classes may contain special methods (also called **magic methods**) that control object behaviour.

Examples include:

Method	**Purpose**
init	Constructor
del	Destructor
str	String representation
len	Length behavior

add	Operator overloading

Example:

```
class Book:
  def __str__(self):
    return "Book Object"
```

These methods enable Python objects to interact with built-in operations.

Interaction Between Components

The elements of a class work together as follows:

- Attributes store data.
- Methods manipulate data.
- Constructors initialize objects.
- Special methods customize behaviour.

Together they define the **complete functionality of objects**.

Example of Complete Class Structure

```
class Student:
  school = "Central School"
def __init__(self, name):
self. name = name      def
display(self):
    print("Name: ", self. name)
  @classmethod
  def school_name(cls):
    return cls. school
@staticmethod   def
info():
    print("Student class")
```

This class contains:

- class attribute
- instance attribute
- instance method
- class method
- static method

Class Namespace

In object-oriented programming, a **namespace** plays a critical role in organizing and managing identifiers such as variable names, function names, and class names. When defining classes in Python, each class maintains its own namespace that stores attributes and methods belonging to that class. Understanding the concept of a class namespace is essential for understanding how Python resolves names, manages memory, and organizes object-oriented structures internally. A namespace acts as a **mapping between names and objects**. It ensures that identifiers used within a particular scope refer to the correct objects without conflicting with identifiers in other scopes.

Definition of Namespace

A namespace can be formally defined as:

> ***A namespace is a collection of name-to-object mappings that Python uses to organize and manage identifiers within a program.***

In simple terms, a namespace is like a **dictionary where names are keys and objects are values**.

For example:

$$name \rightarrow object$$

Python internally manages several namespaces during program execution.

Types of Namespaces in Python

Python maintains different namespaces for different contexts.

The primary namespaces include:

1. **Built-in Namespace**
2. **Global Namespace**
3. **Local Namespace**
4. **Class Namespace**
5. **Instance Namespace**

Each namespace exists independently and prevents naming conflicts between variables defined in different scopes.

Built-in Namespace

This namespace contains built-in functions and exceptions provided by Python. Examples include:

$$print$$
$$len$$
$$type\ int$$

These names are automatically available in every Python program.

Global Namespace

The global namespace contains names defined at the top level of a program or module. Example:

$$\boldsymbol{x} = 10$$

Here x belongs to the global namespace.

Local Namespace

Local namespaces are created when functions execute.

Example:

```
def display():
    y = 5
```

Here y exists only inside the function.

Class Namespace

The class namespace stores all attributes and methods defined inside the class.

Example:

```
class Student:
    school = "Central School"
    def display(self):
        print("Student")
```

Inside the class namespace:

$$school$$
$$display$$

are stored as entries.

Instance Namespace

When objects are created, they have their own namespace for storing instance attributes.

Example:

$$\boldsymbol{student1}.name = "Rahul"$$

Here name belongs to the instance namespace.

Class Namespace Creation

When Python encounters a class definition, it performs the following steps:

1. Creates a temporary namespace dictionary.
2. Executes the class body within this namespace.
3. Stores attributes and methods inside the namespace.
4. Creates the class object using this namespace.

Example:

$$class\ Test:$$
$$\quad \boldsymbol{x} = 10$$
$$\quad def\ \boldsymbol{show}(\boldsymbol{self}):$$
$$\quad\quad \boldsymbol{print}("Hello")$$

Internally Python stores:

$$\{$$
$$'x': 10,$$
$$'show': function,$$
$$'\_\_module\_\_': '\_\_main\_\_'$$
$$\}$$

This dictionary becomes the class namespace.

Accessing Class Namespace

Python provides special attributes to inspect namespaces.

The most used attribute is: **__dict__** Example:

$$class\ Sample:$$
$$\quad \boldsymbol{x} = 5$$
$$\quad def\ \boldsymbol{show}(\boldsymbol{self}):$$
$$\quad\quad pass$$
$$\boldsymbol{print}(\boldsymbol{Sample}.\_\_dict\_\_)$$

Output will show all attributes stored inside the class namespace.

Example result:

$$\{$$
$$'x': 5,$$
$$'show': < function >,$$
$$'\_\_module\_\_': '\_\_main\_\_'$$
$$\}$$

Thus, __dict__ exposes the internal namespace mapping.

Relationship Between Class and Instance Namespace

Both classes and objects maintain separate namespaces.

Example:

$$class\ Student:$$
$$\quad \boldsymbol{school} = "ABC\ School"$$

$s1 = Student()$
$s1.name = "Rahul"$

Namespaces:

Class Namespace: school
Instance Namespace: name

This separation ensures that instance data does not interfere with class-level data.

Attribute Lookup Mechanism

When accessing an attribute through an object, Python follows a specific lookup order.
The search sequence is:

1. Instance Namespace
2. Class Namespace
3. Parent Class Namespace (if inheritance exists)
4. Built-in Namespace Example:

$class\ Student:$
$\quad school = "ABC\ School"$
$s1 = Student()$
$print(s1.school)$

Python searches:

1. Instance namespace → not found
2. Class namespace → found

Thus, the value is retrieved from the class.

Shadowing of Class Attributes

If an instance defines an attribute with the same name as a class attribute, the instance attribute overrides the class attribute.
Example:

$class\ Student:$
$\quad school = "ABC\ School"$
$s1 = Student()$
$s1.school = "XYZ\ School"$
$print(s1.school)$

Output:

$XYZ\ School$

Here the instance attribute **shadows** the class attribute.

Importance of Class Namespace

The class namespace is essential for several reasons:

- Organizes class attributes and methods
- Prevents naming conflicts
- Enables attribute lookup mechanism
- Supports inheritance structures
- Enables introspection and reflection

Without namespaces, large programs would suffer from severe naming collisions.

Attributes vs Methods

In object-oriented programming, classes encapsulate both **data** and **behaviour**. These two components are represented by **attributes** and **methods** respectively. Understanding the distinction between attributes and methods is fundamental to understanding how objects store information and perform operations.

While attributes represent the **state of an object**, methods represent the **actions that an object can perform**. Together, they define the complete functionality of a class.

This section examines the conceptual, structural, and functional differences between attributes and methods, as well as their roles in the object-oriented architecture of Python.

Definition of Attributes

Attributes are variables associated with a class or an object. They store information that represents the state or characteristics of an entity.

Formally:

Attributes are data members of a class that store the state of an object or shared data among objects.

Attributes can be categorized into two types:

1. **Class Attributes**
2. **Instance Attributes**

Class Attributes

Class attributes belong to the class itself and are shared by all objects created from that class.

Example:

```
class Student:
    school = "Central School"
```

Here:

- school is a class attribute.
- All objects of the Student class share this attribute.

Accessing class attributes:

```
print(Student.school)
        or
s1 = Student()
print(s1.school)
```

Both references access the same class-level value.

Instance Attributes

Instance attributes belong to individual objects. Each object can have its own values for these attributes.

Example:

```
class Student:
    def __init__(self, name):
        self.name = name
```

Here:

- name is an instance attribute.
- Each object stores its own value. Example usage:

```
s1 = Student("Rahul")
s2 = Student("Anita") Object
```

states:

$s1.name \rightarrow Rahul$

$s2.name \rightarrow Anita$

Thus, instance attributes represent object-specific data.

Definition of Methods

Methods are functions defined within a class that describe the behaviour of objects. Formally:

Methods are functions associated with a class that operate on the data stored within objects.

Methods enable objects to perform actions such as:

- modifying data
- displaying information
- performing computations Example:

```
class Student:
    def display(self):
        print("Student Information")
```

Here **display()** is a method belonging to the class.

Types of Methods

Python supports three primary types of methods.

Instance Methods

Instance methods operate on instance attributes.

Example:

```
class Student:
    def show(self):
        print(self.name)
```

self refers to the object invoking the method.

Class Methods

Class methods operate on class attributes.

They are defined using the decorator: **@classmethod** Example:

```
class Student:
    school = "Central School"
    @classmethod
    def get_school(cls):
        return cls.school cls
```

represents the class itself.

Static Methods

Static methods do not depend on instance or class data.

Example:

```
class MathTools:
@staticmethod    def
add(a, b):
        return a + b
```

Static methods are utility functions logically grouped within the class.

Structural Difference

Attributes and methods differ structurally within a class.

Attributes are data variables, whereas methods are executable functions.

Example class:

```
class Car:
    brand = "Toyota"
    def start(self):
        print("Car started")
```

Structure:

Component	Type
brand	Attribute
start()	Method

Functional Difference

Attributes and methods serve different purposes in object-oriented programming.

Feature	Attributes	Methods
Purpose	Store data	Define behaviour
Type	Variable	Function
Execution	Not executed	Executed when called
Role	Represent state	Perform actions

Attributes hold information, while methods manipulate that information.

Callable vs Non-Callable Members

One key distinction between attributes and methods is whether they are **callable**.
Attributes are **non-callable objects**, meaning they cannot be executed.
Example: $student.name$
Methods are **callable objects**.
Example: $student.display()$
The parentheses indicate that the method is executed.

Binding of Methods to Objects

When a method is accessed through an object, Python automatically binds the object to the method.
Example:

```
class Test:
    def show(self):
        print("Hello")
t = Test()
t.show()
```

Internally, Python converts this call into: $Test.show(t)$
The object reference t becomes the self-parameter. This process is called **method binding**.

Storage of Attributes and Methods

Attributes and methods are stored differently in memory.
Attributes may be stored in:

- class namespace

- instance namespace

Methods are stored in the **class namespace**.
Objects access methods through their class.
Example: $Object \rightarrow Class \rightarrow Method$
This design reduces memory usage because methods are shared by all objects.

Example Demonstrating Difference

Example class:

```
class Laptop:
  brand = "Dell"
  def show_brand(self):
    print(self.brand)
```

Usage:

```
l1 = Laptop()
print(l1.brand)     # attribute
l1.show_brand()     # method
```

Here:

- brand stores data.
- show_brand() performs an action.

Conceptual Analogy

The relationship between attributes and methods can be compared to real-world objects. Consider a **Car**:
Attributes (State):

- color
- speed
- fuel level Methods (Behaviour):
- start()
- accelerate()
- stop()

Attributes describe **what the object is**, while methods describe **what the object does**.

Design Importance

Separating attributes and methods provides several advantages:

- Clear organization of data and behaviour
- Improved code readability
- Better maintainability
- Enhanced modular design

This separation allows objects to encapsulate both state and functionality within a single unit.

Class Memory Model

To fully understand how classes operate in Python, it is important to examine how they are represented in memory during program execution. While earlier sections discussed the conceptual and structural aspects of classes, this section focuses on the **memory model of classes and objects**.
The class memory model explains how Python stores:

- class definitions
- class attributes
- instance attributes

- methods
- object references

Understanding this internal structure helps programmers write more efficient code and better understand how Python manages objects at runtime.

Concept of Memory Allocation in Python

When a Python program runs, memory is dynamically allocated to store variables, objects, and class structures. Python manages memory automatically using a combination of reference counting and garbage collection.

For classes, memory allocation occurs in two primary stages:

1. **Class Creation**
2. **Object Instantiation**

During these stages, Python allocates memory for both class-level and object-level components.

Memory Allocation During Class Creation

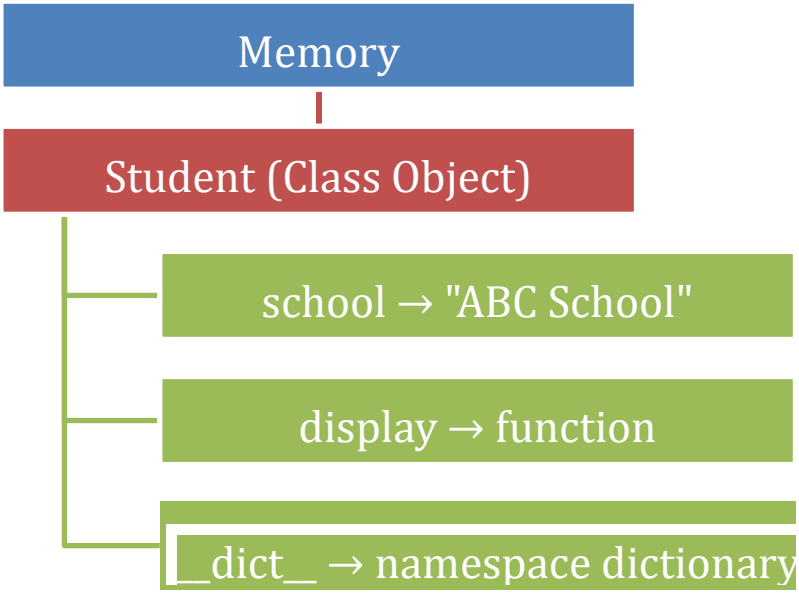
When Python executes a class definition, it creates a **class object** in memory. This class object contains the class namespace, which stores all attributes and methods defined inside the class.

Example:

```
class Student:
    school = "ABC School"
    def display(self):
        print("Student")
```

When Python executes this class definition, it performs the following steps:

1. Creates a temporary namespace dictionary.
2. Executes the class body.
3. Stores attributes and methods inside the namespace.
4. Creates the class object.
5. Binds the class name Student to the class object. Conceptual memory representation:



At this stage, **no objects have been created yet**.

Memory Allocation During Object Creation

Objects are created from classes through a process called **instantiation**.

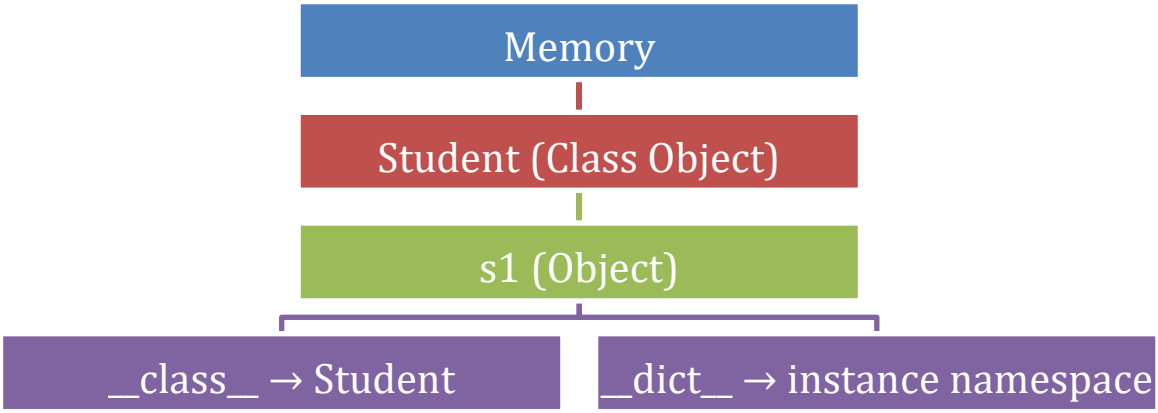
Example:

$s1 = Student()$

When this statement executes, Python performs the following operations:

1. Allocates memory for the object.
2. Creates an instance namespace.
3. Links the object to its class.
4. Initializes attributes if a constructor exists.

Memory representation:



The object stores only its instance-specific attributes.

Class Memory vs Instance Memory

Class attributes and instance attributes are stored in different locations.

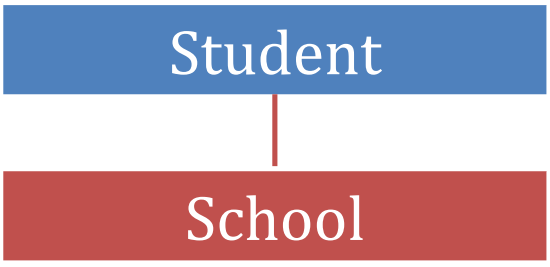
Class Memory

Class attributes are stored once inside the class namespace.

Example:

```
class Student:
    school = "ABC School"
```

Memory:



All objects share this attribute.

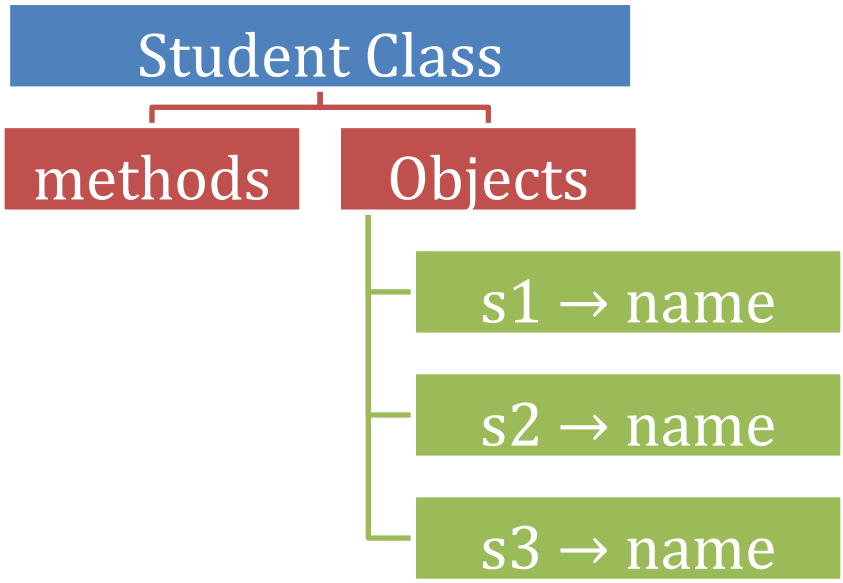
Instance Memory

Instance attributes are stored inside individual objects.

Example:

```
class Student:
    def __init__(self, name):
        self.name = name
```

Memory representation:



Each object maintains its own instance data.

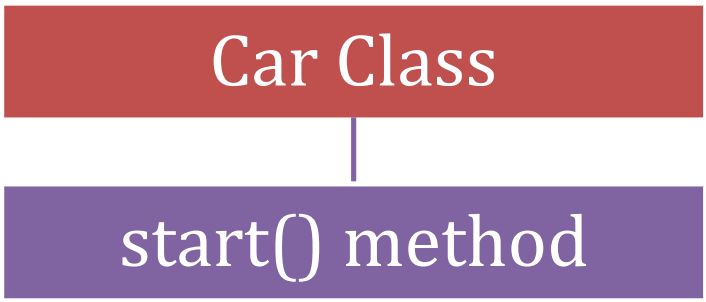
Shared Method Storage

Methods are stored only once inside the class namespace and are shared by all objects.

Example:

```
class Car:
    def start(self):
        print("Car started")
```

Memory structure:



Objects reference this method rather than storing copies.

Example:

$car1 \rightarrow Car.start()$

$car2 \rightarrow Car.start()$

$car3 \rightarrow Car.start()$

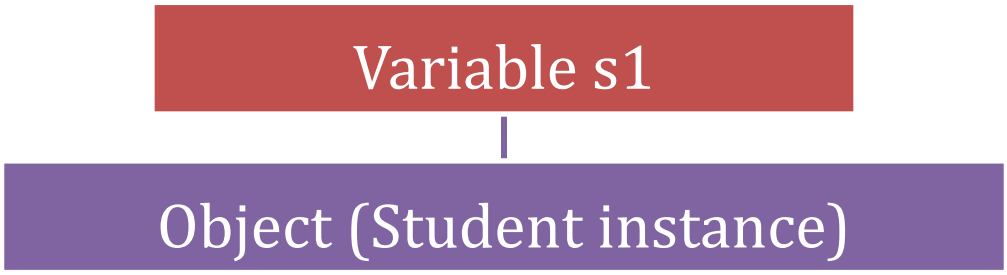
This design improves memory efficiency.

Object Reference Model

Variables in Python do not directly store objects; they store references to objects.

Example:

s1 = Student("Rahul") Memory structure:



Thus, s1 holds a reference to the object rather than the object itself.

Relationship Between Object and Class

Every object contains a reference to the class from which it was created.
This relationship is maintained through the attribute: $\_\_class\_\_$
Example: $print(s1.\ \_\_class\_\_)$
Output: $< class\ 'Student' >$
This reference allows the object to access class attributes and methods.

Attribute Lookup in Memory

When an attribute is accessed through an object, Python follows a specific lookup process.
Example: $s1.\ school$ Lookup order:

1. Instance namespace
2. Class namespace
3. Parent class namespace
4. Built-in namespace Conceptual flow:

$$Object \rightarrow Class \rightarrow Parent\ Class \rightarrow Built-in$$ This search mechanism is called **attribute resolution**.

Memory Efficiency in Class Design

The class memory model is designed for efficiency.
Key features include:

- Methods stored once
- Shared class attributes
- Instance attributes stored only when needed
- Objects referencing class rather than copying data

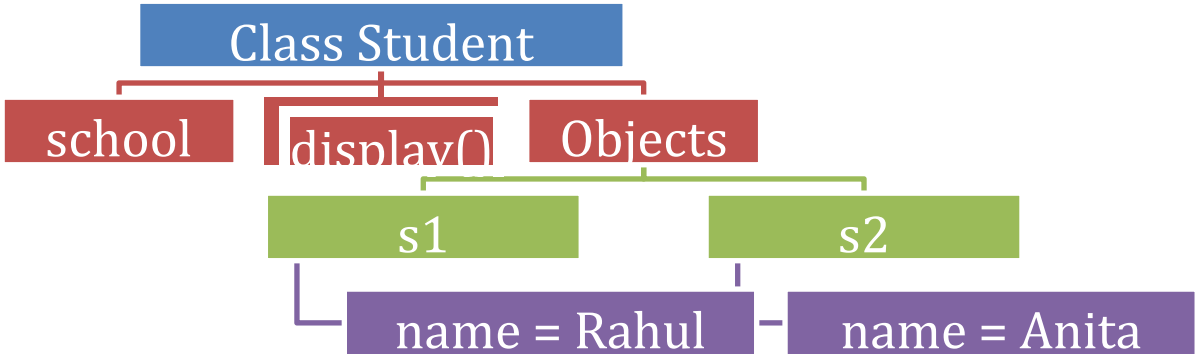
This architecture reduces memory consumption, especially when many objects are created.

Example Demonstrating Class Memory Model

Example program:

```
class Student:
    school = "ABC School"
    def __init__(self, name):
        self.name = name
    def display(self):
        print(self.name)
s1 = Student("Rahul")
s2 = Student("Anita")
```

Memory representation:



Shared:

school

display()

Individual:

name

Garbage Collection

Python automatically manages memory using garbage collection.
When objects are no longer referenced, Python removes them from memory.
Example: ***del*** *s*1
If no references remain, the object becomes eligible for garbage collection. This automatic memory management prevents memory leaks.

5.4 Objects

In the previous section, we examined the concept of **classes** in detail. Classes serve as structural blueprints that define attributes and methods representing the properties and behaviors of entities. However, classes alone do not perform any operations or represent real data until they are instantiated. The true functionality of object-oriented programs emerges when **objects are created from classes**.

Objects are the fundamental runtime entities in object-oriented programming. While a class describes what an entity should look like, an object represents an actual instance of that entity in memory. In other words, objects bring class definitions to life by allocating memory, storing data, and executing methods.

In Python, everything is treated as an object. Numbers, strings, lists, functions, and even classes themselves are objects. This unified object model makes Python highly flexible and dynamic. Understanding objects is therefore essential for mastering Python programming and for fully appreciating how object-oriented systems operate internally.

Objects encapsulate both **state and behavior**. The state of an object is represented by its attributes, while the behavior is represented by its methods. When objects interact with each other by invoking methods, they form the dynamic structure of a program.

This section explores objects from both conceptual and technical perspectives. It explains how objects are defined, how they are created from classes, how Python performs the instantiation process, how objects maintain identity within memory, and how objects evolve throughout their lifecycle during program execution.

Definition of Object

In object-oriented programming, an **object** represents a concrete instance of a class that exists in memory during program execution. While a class defines the structure and behavior of a type of entity, an object embodies that definition by storing actual data and enabling the execution of methods associated with the class.

Objects are the fundamental runtime entities in object-oriented systems. Programs built using object-oriented principles operate primarily through the creation, interaction, and manipulation of objects. Every operation performed by a program ultimately involves objects interacting with one another.

Formal Definition

An object can be formally defined as:

An object is an instance of a class that occupies memory and contains both state (data) and behaviour (methods).

This definition highlights two important aspects:

1. An object is created from a class.
2. An object contains both data and functionality.

Thus, objects serve as the practical realization of class definitions.

Conceptual Meaning of an Object

A class describes a general category, while an object represents a specific member of that category.

For example:

Class →

Student

Objects →

Rahul

Anita

Karan

Each student object has its own values for attributes such as name, age, or roll number, even though all objects share the same structure defined by the class.

Thus:

Class → **Blueprint**

Object → **Real Instance**

Physical Existence of Objects

Unlike classes, which exist conceptually as templates, objects have **physical existence in memory**. When an object is created, Python allocates memory to store its attributes and maintains a reference to the class from which it was created.

Example:

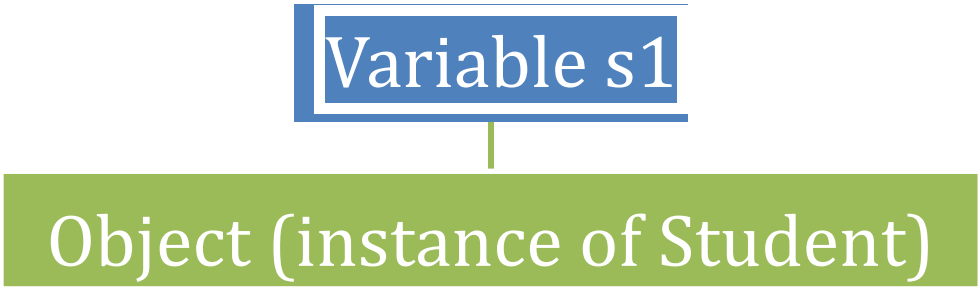
$$\boldsymbol{class\ Student}:$$

$$\quad \boldsymbol{pass}$$

$s1 = Student()$ In

this example:

- Student is the class.
- s1 is an object of that class. Memory representation:



The object occupies memory and can store data or execute methods.

Components of an Object

An object typically consists of three essential components:

1. **Identity**
2. **State**
3. **Behaviour**

These three characteristics define the complete nature of an object. **Identity**
Identity refers to the unique reference of an object in memory. Even if two objects contain identical data, they remain distinct because they occupy different memory locations.
Python provides the id() function to retrieve an object's identity.
Example:

$s1 = Student()$
$print(id(s1))$

Each object has a unique identity value.
State
The state of an object represents the data stored within it. This data is stored in the object's attributes.
Example:

$\boldsymbol{class\ Student}$:
  $\boldsymbol{def}\ \_init\_(self, name)$:
    $self.name = name$

Here name represents the state of the object.
Example object states:
s1.name → Rahul s2.name
→ Anita
Each object maintains its own state.
Behaviour
Behaviour represents the operations that an object can perform. These operations are defined as methods within the class.
Example:

$\boldsymbol{class\ Student}$:
  $\boldsymbol{def}\ greet(self)$:
    $print("Hello\ Student")$

When the method is called: $s1.\ greet()$
The object performs an action defined by the class.

Object as a Self-Contained Entity

Objects encapsulate both data and the methods that operate on that data. This integration creates a **self-contained unit** capable of representing real-world entities.

For example, a bank account object may contain:

Attributes:

- account number
- balance Methods:
- deposit()
- withdraw()
- check_balance()

This design ensures that data and operations are logically grouped together.

Objects in Python's Unified Model

Python follows a unified object model in which nearly everything is treated as an object. Examples include:

- integers
- strings
- lists
- functions
- modules
- classes Example:

$x = 10$

$print(type(x))$

Output:

$< class\ 'int' >$

Even basic data types are implemented internally as objects.

Object Interaction

Objects rarely operate in isolation. Instead, they interact with other objects through method calls. This interaction allows objects to collaborate in performing complex tasks.

Example:

A **Customer object** may interact with an **Account object** to perform financial transactions. This communication mechanism is often referred to as **message passing**.

Object Creation from Classes

Objects are created by calling the class name like a function.

Example:

$\boldsymbol{class\ Car}$:

  $\boldsymbol{pass}$

$c1 = Car()$ Here:

- Car defines the structure.
- $c1$ becomes an instance of that structure.

The object can then store attributes and execute methods defined in the class.

Multiple Objects from One Class

One class can create multiple objects.

Example:

$s1 = Student()$

$s2 = Student()$

$s3 = Student()$ Each

object:

- shares the same class structure
- maintains independent data
- occupies separate memory

This allows efficient modelling of many entities using a single class definition.

Object Creation

After understanding what an object is, the next step is to examine **how objects are created in Python**. Object creation is the process through which a class definition is transformed into a real instance that occupies memory and can store data or perform operations.

In object-oriented programming, classes define the structure of objects, but objects themselves come into existence only when they are explicitly created during program execution. This process allows programs to generate multiple instances from a single class blueprint.

Concept of Object Creation

Object creation refers to the act of **instantiating a class**, which means creating an individual instance of that class. During object creation, Python allocates memory for the object and prepares it for use.

Formally:

> ***Object creation is the process of generating an instance of a class by allocating memory and associating the instance with the class definition.***

Once an object is created, it can access attributes and methods defined in the class.

Basic Syntax for Object Creation

Objects are created by calling the class name followed by parentheses.

General syntax:

$$\boldsymbol{object\_name = ClassName()}$$

Explanation:

- ClassName() invokes the class constructor.
- Python creates an instance of the class.
- The reference to the object is stored in object_name.

Example:

$class\ Student$:

  $pass$

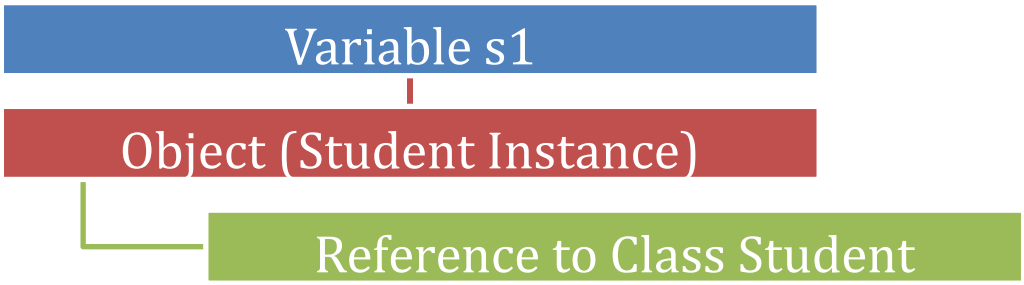
$\boldsymbol{s1 = Student()}$ Here:

- Student is the class.
- s1 is an object (instance) of that class.

Memory Representation of Object Creation

When an object is created, Python allocates memory to store the instance.

Conceptual representation:



Thus:

- The variable stores a **reference** to the object.
- The object stores a reference to its **class**.

Object Creation with Constructor Initialization

Most classes contain a constructor method named __init__(). This method initializes the attributes of an object when it is created.

Example:

$$class\ Student:$$
$$def\ \_\_\boldsymbol{init}\_\_(\boldsymbol{self}, \boldsymbol{name}):$$
$$\boldsymbol{self}.name = \boldsymbol{name}$$

Object creation:

s1 = Student("Rahul") During this process:

1. Memory is allocated for the object.
2. The constructor is executed.
3. The attribute name is initialized.

Object state:

$$s1.name \rightarrow Rahul$$

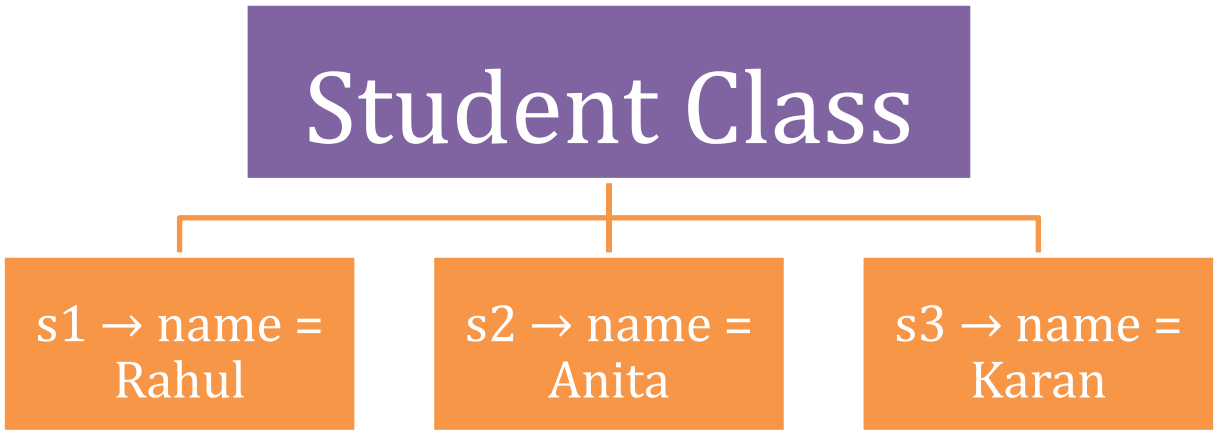
Creating Multiple Objects

A single class can generate multiple objects.

Example:

s1 = Student("Rahul")
s2 = Student("Anita") s3 = Student("Karan")

Memory structure:



Each object maintains its own independent data.

Object Creation Without Constructor

If a class does not define a constructor, Python automatically provides a default constructor.
Example:

```
class Test:
    pass
t1 = Test()
```

Python still creates the object even though no explicit initialization method exists.

Assigning Attributes After Object Creation

Attributes can also be assigned to objects after they are created.
Example:

```
class Student:
    pass
s1 = Student()
s1.name = "Rahul"
s1.age = 20
```

Here attributes are dynamically added to the object.
Object state:

s1.name → Rahul s1.age
→ 20

This demonstrates Python's flexible dynamic object model.

Object Creation and Reference Variables

When an object is created, the variable used in assignment does not store the object itself. Instead, it stores a reference pointing to the object's location in memory.
Example:

```
s1 = Student()
```

Memory representation:

s1 → Reference → Student Object

Multiple variables can reference the same object.
Example: $s2 = s1$

Now both variables point to the same object.

Checking Object Type

Python provides built-in functions to verify object type.
Example:

$$\boldsymbol{print}(\boldsymbol{type}(\boldsymbol{s1}))$$

Output:

$$< class\ '\_\_main\_\_.\ Student' >$$

This confirms that s1 is an instance of the Student class.
Another useful function:

$$\boldsymbol{isinstance}(\boldsymbol{s1},\boldsymbol{Student})$$

Output:

$$True$$

Object Creation Flow

The object creation process follows this sequence:

1. The class name is invoked.
2. Python allocates memory for the object.
3. The constructor initializes attributes.
4. The object reference is returned.
5. The reference is stored in a variable.

Conceptual flow:
Class → Object Creation → Memory Allocation → Initialization → Reference Assignment

Importance of Object Creation

Object creation is essential because it allows programs to represent multiple real-world entities using a single class definition.
For example:
A **Student class** can create hundreds of student objects, each storing unique information such as name, age, and roll number.
This mechanism makes object-oriented systems scalable and reusable.

Instantiation Process

In object-oriented programming, **instantiation** refers to the internal process through which a class is converted into an object during program execution. While the previous section explained how objects are created syntactically by calling a class name, this section focuses on the **internal steps that Python performs behind the scenes when an object is instantiated**.
Instantiation is a multi-stage process involving memory allocation, object initialization, and reference assignment. Understanding this process provides deeper insight into how Python manages objects internally and how constructors participate in preparing objects for use.

Definition of Instantiation

Instantiation can be formally defined as:
Instantiation is the process by which Python creates an object from a class by allocating memory, initializing attributes, and returning a reference to the newly created object. During instantiation, Python transforms the class blueprint into a functional instance that can store data and execute methods.

Basic Instantiation Example

Consider the following class:

$$\textbf{\textit{class}}\ Student:$$
$$\quad \textbf{\textit{def}}\ \_\_init\_\_(self, name):$$
$$\quad\quad self.name = name$$

Object creation:

$$s1 = Student("Rahul")$$

This single statement triggers several internal operations inside Python.

Steps in the Instantiation Process

When a class is called to create an object, Python performs the following sequence of steps:

Step 1: Class Invocation

The class name is called like a function.

Example:

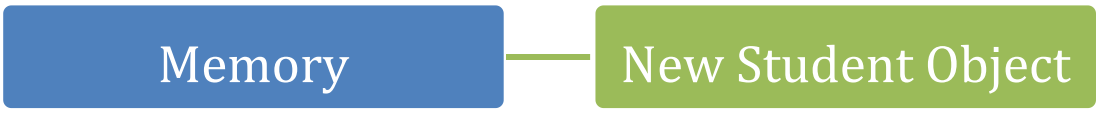
$$Student("Rahul")$$

Python interprets this as a request to create an object of the Student class.

Step 2: Memory Allocation

Python allocates memory space for the new object. This memory will store the object's attributes and internal information.

Conceptual representation:



This step ensures that the object has its own independent storage.

Step 3: Object Creation using __new__()

Internally, Python first calls the special method: $\_\_new\_\_()$ The __new__() method is responsible for **creating the object itself**.

General form:

$$object = ClassName.\_\_new\_\_(ClassName)$$

This method returns a newly created object.

Note: In most programs, programmers do not override __new__() unless advanced behavior is required.

Step 4: Object Initialization using __init__()

After the object is created, Python automatically calls the constructor: $\_\_init\_\_()$ The constructor initializes the attributes of the object.

Example:

$$\textbf{\textit{def}}\ \_\_init\_\_(self, name):$$
$$\quad self.name = name$$

During instantiation:

$$\_\_init\_\_(s1, "Rahul")$$

This assigns the value "Rahul" to the attribute name.

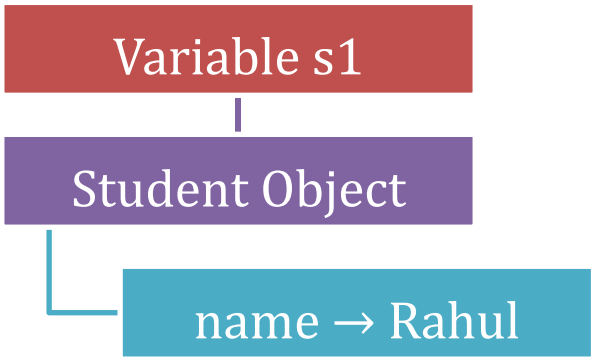
Step 5: Reference Assignment

After initialization, Python returns a reference to the newly created object.

Example:

$$s1 = Student("Rahul")$$

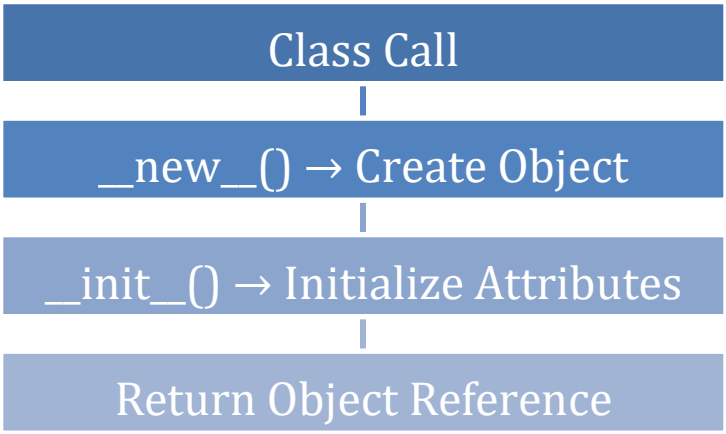
Memory representation:



The variable s1 now points to the object stored in memory.

Complete Instantiation Flow

The entire process can be summarized as follows:



Thus, instantiation converts a class definition into a usable object.

Role of the Constructor in Instantiation

The constructor (__init__) plays an essential role in the instantiation process. It prepares the object by assigning initial values to its attributes.

Example:

```
class Car:
    def __init__(self, brand, model):
        self.brand = brand
        self.model = model
```

Object creation:

$c1 = Car("Toyota", "Camry")$ Object

state:

c1.brand → Toyota c1.model

→ Camry

Without a constructor, objects would not automatically receive initial data.

Instantiation Without Explicit Constructor

If a class does not define a constructor, Python automatically provides a default constructor.

Example:

```
class Test:
    pass
t1 = Test()
```

Python still performs the instantiation process, but no attributes are initialized.

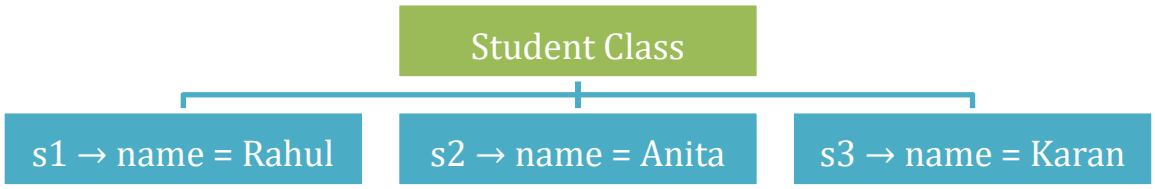
Instantiation of Multiple Objects

Instantiation allows a single class to generate multiple objects.

Example:

```
s1 = Student("Rahul")
s2 = Student("Anita")
s3 = Student("Karan")
```

Memory representation:



Each object is created independently but shares the same class structure.

Internal Relationship Between Class and Object

During instantiation, the object stores a reference to the class from which it was created.

Example:

```
print(s1.__class__)
```

Output:

```
<class 'Student'>
```

This reference allows the object to access class attributes and methods.

Instantiation and Method Binding

After instantiation, methods defined in the class become accessible to the object.

Example:

```
class Student:
    def greet(self):
        print("Hello Student")
s1 = Student()
s1.greet()
```

Internally:

```
Student.greet(s1)
```

This mechanism binds the method to the object instance.

Object Identity

In object-oriented programming, every object created during program execution possesses a unique identity that distinguishes it from all other objects. Even if two objects contain identical data and exhibit the same behaviour, they remain separate entities because they occupy different locations in memory. This unique identification of objects is known as **object identity**.

Object identity plays a critical role in Python's memory model because it allows the interpreter to distinguish between different objects and manage references accurately. Understanding object

identity helps programmers comprehend how Python handles variables, references, comparisons, and memory management.

Definition of Object Identity

Object identity can be formally defined as:

> ***Object identity is the unique reference assigned to an object that distinguishes it from all other objects in memory.***

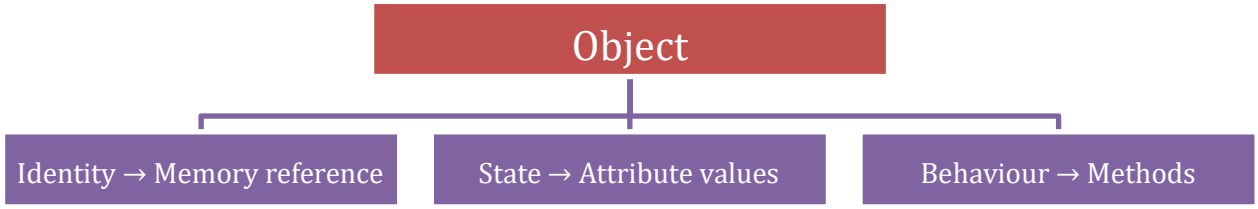
This identity corresponds to the object's memory address or a unique identifier associated with that memory location.

Thus, even if two objects contain the same values, they are considered different objects if their identities differ.

Identity vs State vs Behaviour

An object in Python is characterized by three fundamental properties:

1. **Identity** – the unique reference that identifies the object in memory
2. **State** – the data stored within the object (attributes)
3. **Behaviour** – the operations that the object can perform (methods) These three properties together define the complete nature of an object. Example conceptual representation:



The id() Function

Python provides a built-in function called id() that returns the unique identity of an object.

Syntax: id(object) Example:

```
class Student:
    pass
s1 = Student()
s2 = Student()
print(id(s1))
print(id(s2))
```

Output (example):

140223453121040

140223453121184

Each number represents the unique identity of the object in memory.

Even though both objects belong to the same class, they have different identities.

Identity and Memory Address

In many Python implementations, the value returned by id() corresponds to the memory address of the object. This means the identity directly reflects where the object is stored in memory.

Conceptual representation:

s1 → Object stored at memory location A s2

→ Object stored at memory location B

Since the memory locations differ, the identities differ.

Identity Comparison Using is Operator

Python provides the is operator to compare object identities.

Syntax: ***object1*** *is* ***object2***

This operator returns **True** if both variables refer to the same object.

Example:

$\boldsymbol{a} = [1, 2,3]$

$\boldsymbol{b} = \boldsymbol{a}$

$\boldsymbol{print}(\boldsymbol{a}\ is\ \boldsymbol{b})$

Output: True

Here both variables reference the same object.

Identity vs Equality

It is important to distinguish between **identity** and **equality**.

Concept	**Meaning**
Identity	Whether two variables refer to the same object
Equality	Whether two objects contain the same value

Equality comparison uses the == operator.

Example:

$\boldsymbol{a} = [1, 2,3]$

$\boldsymbol{b} = [1,2, 3]$

$\boldsymbol{print}(\boldsymbol{a} == \boldsymbol{b})$

$\boldsymbol{print}(\boldsymbol{a}\ is\ \boldsymbol{b})$

Output:

True

False Explanation:

- a == b → True because values are equal
- a is b → False because objects are different

Thus, identity and equality represent different comparisons.

Object Identity in Reference Assignment

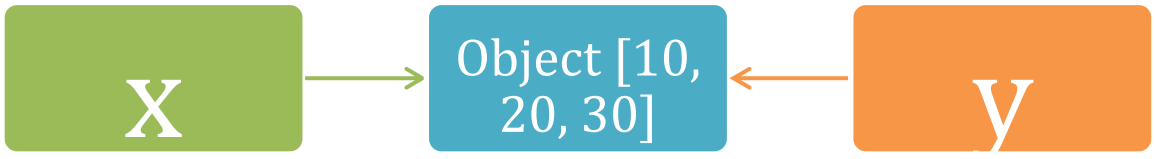
When a variable is assigned to another variable, both variables reference the same object.

Example: x =

[10, 20, 30]

y = x

Memory representation:



Here both x and y share the same identity.

Checking identity:

$\boldsymbol{print}(\boldsymbol{x}\ is\ \boldsymbol{y})$

Output:

True

Identity and Immutable Objects

For immutable objects such as integers and strings, Python may reuse objects internally to optimize memory usage. In such cases, two variables may share the same identity if Python internally caches the object.

Example:

$$\boldsymbol{a} = 10$$
$$\boldsymbol{b} = 10$$
$$\boldsymbol{print}(\boldsymbol{a}\ is\ \boldsymbol{b})$$

In many cases, the output may be: $True$

This occurs because Python reuses small integer objects.

Identity in Object-Oriented Systems

Object identity allows object-oriented systems to manage multiple objects efficiently. Even if many objects belong to the same class and contain identical data, their identities ensure that each object remains distinct.

Example:

$$\boldsymbol{s1} = \boldsymbol{Student}()$$
$$\boldsymbol{s2} = \boldsymbol{Student}()$$

Even though both objects are instances of the same class, they represent different entities.

Importance of Object Identity

Object identity is important for several reasons:

- Enables Python to manage object references
- Distinguishes between different objects in memory
- Supports reference-based variable assignments
- Allows identity comparison using is
- Enables efficient memory management

Without object identity, Python would not be able to track objects accurately during program execution.

Conceptual Example

Consider a student management system.

Two students may have identical details:

Name: Rahul

Age: 20

However, they are still different students. Similarly, in Python:

Student Object 1 → Identity A

Student Object 2 → Identity B

Even if their data is identical, their identities differ.

CHAPTER 6 – Modules, Exceptions, and Error Handling

6. Modules, Exceptions and Error Handling

6.1 Introduction to Modules

A **module** in Python is a file that contains a collection of **functions, variables, classes, and executable statements**, which can be reused in multiple programs.

In simple terms:

A module is just a **.py file** that helps you organize and reuse code.

6.2 Why Modules are Required

In Python programming, as applications grow in size and complexity, writing the entire code in a single file becomes difficult to manage, understand, and maintain.

To overcome these limitations, Python provides **modules**, which allow developers to divide large programs into smaller, manageable, and reusable components.

Modules solve this problem by:

- Breaking large programs into smaller parts
- Improving Readability
- Enabling code reuse
- Making debugging easier
- Supporting team development

6.3 Types of Modules

Definition:

Modules in Python are categorized based on their origin and usage.

Types:

1. Built-in Modules
2. User-defined Modules
3. Third-party Modules

Type	Description	Example
Built-in	Predefined modules provided by Python, available without installation	math, sys
User-defined	Modules created by the programmer to organize and reuse custom code	mymodule.py
Third-party	External modules developed by others, installed using package managers	numpy, pandas

Example:

```
import math
print(math.sqrt(25))  #5.0
```

6.4 Creating and Using Modules

A module in Python is not just a file, but a structured unit of code organization. It allows developers to logically group related functions, variables, and classes into a single file, making the program modular and reusable.

When a module is created, Python treats it as an independent namespace. This means all the variables and functions defined inside the module are stored separately and can be accessed using the module name.
Using modules helps in:

- Reducing code duplication
- Improving maintainability
- Organizing large applications into smaller parts

Step-by-step:
Step-1: Create File
"*mymodule.py*"
Step-2: Add Code

```
def add(a,b):
    return a + b
```

Step-3: Use in another file

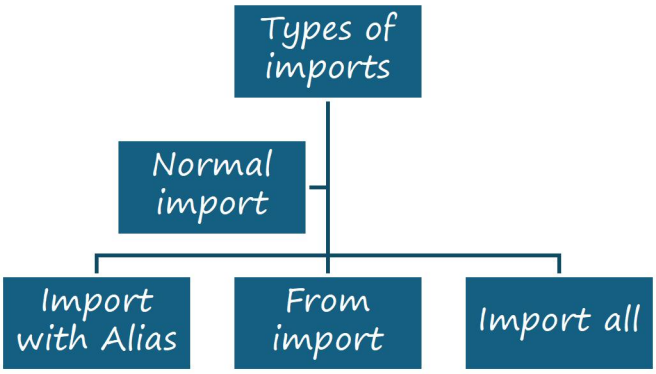
```
import mymodule
print (mymodule.add(10, 20))
```

Key Concepts Table:

Concept	Meaning
Module Name	File name
Function Call	module.function()
Reusability	Use anywhere

6.5 Importing Modules

Importing a module means accessing the functions, variables, or classes defined in another Python file or built-in library so that they can be used in the current program.



1.Normal Import

In this method, the entire module is imported, and its contents are accessed using the module name.

Example:

```
import math
print(math.sqrt(16))
```

2. Import with Alias

In this method, the module is imported with an alternative name (alias) to make it easier to use.

Example:

```
import math as m
print(m.sqrt(35))
```

3. From Import

In this method, specific functions or variables are imported directly from a module.

Example:

```
from math import sqrt
print(sqrt(36))
```

4. Import All

In this method, all functions and variables from a module are imported directly into the current namespace.

Example:

```
from math import  *
print(sqrt(49))
```

6.6 Module Search Path

The Module Search Path is the list of locations (directories) where Python looks for a module when we try to import it.

When you write an import statement, Python does not directly know where the module is stored. It searches for the module in a specific order called the search path

How Python Searches for Modules:

When a module is imported, Python searches in the following order:

1. Current Working Directory

Python first checks the folder where the current program is running.

👉 If your module file is in the same folder, Python will find it immediately.

Example:

```
# mymodule.py is in same folder
  import mymodule
```

2. PYTHON PATH

If the module is not found in the current directory, Python checks the directories listed in the PYTHONPATH.

👉 PYTHONPATH is a list of external folders where modules can be stored.

Real-world idea:

Think of PYTHONPATH like a “custom folder list” where you keep your reusable modules.

3.Standard Library Directories
If still not found, Python checks its built-in modules.

👉 These are modules already provided by Python

Example:

import math
import sys

Viewing Module Search Path

We can see all the paths where Python searches using the *sys* module.

Example:

import sys
print(sys.path)

Example of Error:

import mymodule

If mymodule.py is not present in any search path:
ModuleNotFoundError: No module named 'mymodule'

6.7 Built-in Modules

Definition:

Built-in modules are pre-defined modules that come along with Python installation. These modules provide ready-made functions and classes to perform common tasks such as mathematical operations, random number generation, and date-time handling.

👉 These modules do not require any external installation and can be used directly by importing them

Why Built-in Modules are Important

- Reduce development time
- Avoid writing complex logic manually
- Provide optimized and tested functions
- Improve code readability

1.Math Module

The math module provides functions for performing mathematical calculations such as square root, factorial, power, algorithms, etc.
Common Functions:

Function	Description	Role
sqrt()	Square Root	Sqrt(25) → 5
pow()	Power	pow(2,3) → 8
factorial()	Factorial	Factorial(5) →120

ceil()	Round up	ceil(4.2) → 5
floor()	Round down	floor(4.8) → 4

Example:

```
import math
print(math.sqrt(16))
print(math.factorial(5))
print(math.ceil(4.2))
print(math.floor(4.8))
```

2.Random Module:

The **random module** in Python is a built-in module used to generate random values. These values can be integers, floating-point numbers, or randomly selected elements from sequences such as lists, tuples, and strings.

It is widely used in applications where unpredictability is required, such as games, simulations, sampling, testing, and security-related features.

The random module uses a pseudo-random number generator, which means the numbers appear random but are generated using deterministic algorithms

◇ **Why Random Module is Important**

- Used to simulate real-world randomness
- Helps in testing programs with unpredictable inputs
- Used in gaming logic (dice, cards)
- Used in data sampling and analysis

◇ **Important Functions in Random Module:**

1. Randint(a,b)

Returns a random integer between a and b (both inclusive).

👉 It includes both starting and ending values.

Examples:

(i)
```
import random
print(radom.randint(1,10))
```

(ii)
```
import random
dice = random.randint(1,6)
print("Dice Value:", dice)
```

(iii)
```
import randon
otp = random.randint(1000,9999)
print("OTP:", otp)
```

2.Randrange(start,stop,step)

Returns a random number from a specified range, similar to the *range()* function

👉 It does NOT include the stop value**.**

Examples:

(i) import random

```
chars = "abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ1234567890"
password = ""
for i in range(8):
password += random.choice(chars)
print("Generated Password:", password)
```

(ii) import random

```
rewards = ["100 Coins", "Extra Life", "50 Gems", "Try Again"]
result = random.choice(rewards)
print("Spin Reward:", result)
```

3.Choice

The *choice()* function in the random module is used to select and return a single random element from a sequence such as a list, tuple, or string.The selection is completely random, and every element in the sequence has an equal chance of being selected.

👉 The sequence can contain:

- List
- Tuple
- String

Examples:

(i) import random

```
students = ["Rahul", "Asha", "Kiran", "Sneha", "Vijay"]
selected = random.choice(students)
print("Selected Student:", selected)
```

(ii) import random

```
movies = ["Interstellar", "KGF", "Inception", "Bahubali"]
movie = random.choice(movies)
print("Recommended Movie:", movie)
```

(iii) import random

```
letter = random.choice("ABCDEFGHIJKLMNOPQRSTUVWXYZ")
,print("Random Letter:", letter)
```

4. Choices(Sequence, k=n)

The *choices()* function in the random module is used to select multiple random elements from a sequence such as a list, tuple, or string.

Unlike *choice(),* which returns only one element, *choices()* returns multiple elements at the same time in the form of a list.

👉 The same element can be selected more than once because *choices()* performs random selection with replacement.

The number of elements to be selected is specified using the *k* parameter**.**

Syntax:

random.choices(sequence, k=n)

sequence → Collection of elements

k → Number of random elements to select

Examples:

(i) import random

```
questions = [
        "Python",
        "Java",
        "C++",
        "JavaScript",
        "SQL"
 ]
selected = random.choices(questions, k=3)
print("Selected Questions:", selected)
```

(ii) import random

```
offers = ["10% OFF", "20% OFF", "Free Delivery", "Cashback"]
selected_offers = random.choices(offers, k=3)
print("Today's Offers:", selected_offers)
```

5. Sample(Sequence, k=n)

The *sample()* function in the random module is used to select multiple unique random elements from a sequence such as a list, tuple, or string.

Unlike *choices(),* the *sample()* function does not allow duplicate selections. Once an element is selected, it cannot be selected again in the same sampling process.

The selected elements are returned in the form of a list.

👉 The number of elements to be selected is specified using the *k* parameter.

If the value of *k* is greater than the number of available elements, Python generates a *ValueError.*

Syntax:

random.sample(sequence, k=n)

sequence → Collection of elements

k → Number of unique random elements to select

Examples:

(i) import random

```
participants = ["Rahul", "Asha", "Kiran", "Sneha", "Vijay"]
winners = random.sample(participants, k=2)
print("Selected Winners:", winners)
```

(ii) import random

```
seats = [101,102,103,104,105,106]
allocated = random.sample(seats, k=3)
print("Allocated Seats:", allocated)
```

**** Return Unique random elements****

****Duplicate values are not allowed****

6. Shuffle(sequence)

The *shuffle()* function in the random module is used to randomly rearrange the elements of a mutable sequence such as a list.

It changes the order of the elements directly in the original list, meaning the existing list itself gets modified instead of creating a new list.

The arrangement of elements becomes completely random each time the program is executed.

👉 The *shuffle()* function works only with mutable sequences like lists and does not work with strings or tuples directly because they are immutable**.**

Syntax:

random.shuffle(sequence)

sequence → List whose elements need to be shuffled randomly

Examples:

(i) import random

```
cards = ["Ace", "King", "Queen", "Jack", "10"]
random.shuffle(cards)
print("Shuffled Cards:", cards)
```

(ii)import random

```
questions = [
    "Python Basics",
    "Functions",
    "Modules",
    "OOP",
    "Exception Handling"
]
random.shuffle(questions)
print("Shuffled Questions:")
for q in questions:
print(q)
```

7. Seed(value)

The seed() function in the random module is used to initialize the random number generator with a specific value.

Normally, random numbers generated by Python are different each time the program runs. However, by using seed(), we can make the random module generate the same sequence of random values repeatedly. This is useful when we want reproducible results for testing, debugging, simulations, or scientific experiments.

👉 If the same seed value is used again, Python produces the same random output sequence**.**

Syntax:

random.seed(value)

value → Any integer or valid seed value used to initialize the random generator

Examples:

(i) import random

```
random.seed(20)
rewards = ["Coins", "Weapon", "Extra Life", "Gems"]
print(random.choice(rewards))
```

```
    print(random.choice(rewards))
```
(ii)
```
import random
chars = "ABC123XYZ"
random.seed(7)
password = ""
for i in range(5):
password += random.choice(chars)
print("Generated Password:", password)
```

6.8 User-defined Modules:

User-defined modules are modules created by programmers to organize and reuse their own code.
In Python, any .py file containing functions, variables, or classes can be treated as a module and used in other Python programs.
These modules help developers divide large applications into smaller, manageable, and reusable components.
👉 Instead of writing the same code again and again, we can place commonly used code inside a module and import it whenever required

6.8.1 Why user-defined modules are important:

As software applications grow larger, writing the entire program inside a single file becomes difficult to manage, understand, and maintain. To solve this problem, Python provides user-defined modules, which allow programmers to divide programs into smaller reusable files.
User-defined modules help developers organize related functions and logic separately, making the application more structured and modular.
Instead of rewriting the same code multiple times, developers can create a module once and reuse it in different programs whenever needed.
In real-world software development, large applications are usually divided into multiple modules such as:

- Authentication module
- Payment module
- Database module
- Employee module
- Report generation module

This modular approach improves readability, maintainability, and teamwork.

6.8.2 Creating a User-defined Module:

A user-defined module can be created simply by creating a Python file with the .py extension.
Inside the module file, we can define:

- Functions
- Variables
- Classes
- Executable statements

The module name is automatically taken from the file name.

6.8.3 steps to create a Module

Step 1: Create a python file

calculator.py

Step 2: Write functions inside the module

```
def add(a, b):
return a + b
def sub(a, b):
return a - b
def mul(a, b):
return a * b
```

step 3 : Save the file

Save the module in the same directory where the main program is present.

Using the Module:

Once the module is created, it can be imported into another Python program using the import statement.

After importing, all functions inside the module can be accessed using the module name.

Examples:

```
#main.py
    import calculator
    print(calculator.add(10, 20))          #30
    print(calculator.sub(50, 10))          #40
    print(calculator.mul(5, 4))            #20
```

6.8.4Import Variations

Python provides multiple ways to import modules depending on the requirement.

Different import methods improve readability and simplify function access

1.Import Complete Module

In this method, the entire module is imported into the current program using the import keyword.

After importing the module, all functions, variables, and classes inside the module can be accessed using the module name followed by the dot *(.)* operator.

This is one of the most commonly used and recommended importing methods because it improves readability and clearly shows from which module a function is being accessed

Examples:

(i) #banking.py

```
def deposit(amount):
 print("Amount Deposited:", amount)
 def withdraw(amount):
 print("Amount Withdrawn:", amount)
```

#customer.py

```
 import banking
 banking.deposit(5000)
 banking.withdraw(2000)
```

(ii) # calculator.py

```
def add(a, b):
return a + b
def div(a, b):
return a / b
```

 # app.py

```
import calculator
print("Addition:", calculator.add(10, 20))
print("Division:", calculator.div(50, 5))
```

2. Import Specific Functions:

In this method, only selected functions or variables are imported from the module instead of importing the complete module.

This approach reduces unnecessary imports and allows direct access to functions without using the module name repeatedly.

It is useful when only a few functions are required from a module.

Examples:

(i) #salary.py

```
def bonus(salary):
return salary * 0.20
def tax(salary):
return salary * 0.10
```

 # company.py

```
from salary import bonus, tax
print("Bonus:", bonus(50000))
print("Tax:", tax(50000))
```

(ii) # result.py

```
def total(m1, m2, m3):
return m1 + m2 + m3
def average(total_marks):
return total_marks / 3
```

 # school.py

```
from result import total, average
t = total(80, 75, 90)
print("Total Marks:", t)
print("Average:", average(t))
```

3.Import Module with Alias:

In this method, a module is imported using an alternative name called an alias by using the as keyword.

Alias names are mainly used to simplify long module names and improve code readability.

This method is very useful in large applications where frequently used modules have lengthy names

Examples:

(i) # scientific.py

```
def square(n):
return n * n
def cube(n):
```

```
        return n * n * n
# main.py
    import scientific as sci
    print("Square:", sci.square(5))
    print("Cube:", sci.cube(3))
```

(ii)

```
# shopping.py
    def total(price, quantity):
    return price * quantity
    def discount(amount):
    return amount * 0.10# billing.py
# billing.py
    import shopping as shop
    bill = shop.total(500, 3)
    print("Total Bill:", bill)
    print("Discount:", shop.discount(bill))
```

6.8.5 Advantages of Modules:

Introduction

In Python, modules play an important role in developing structured and organized programs. A module is a file containing Python code such as functions, variables, and classes that can be reused in multiple programs. Instead of writing the same code repeatedly, programmers can store commonly used functionalities inside modules and import them whenever needed.

Modules are especially useful in large software applications where the program contains thousands of lines of code. By dividing the program into smaller manageable files, modules improve readability, maintenance, debugging, and reusability.

1. Code Reusability

One of the biggest advantages of modules is code reusability. A function written in one module can be reused in multiple programs without rewriting the same logic again.

This saves:

- Development time
- Programmer effort
- Memory usage
- Testing effort